ทฤษฎีการคำนวณ

ภาษาไม่พึ่งบริบท เครื่องจักรทัวริง ความตัดสินได้ และความซับซ้อน

เอกสารสรุปบทเรียนภาษาไทย

เรียบเรียงเพื่อใช้ประกอบการอ่าน
Introduction to the Theory of Computation (Sipser, 3rd ed.)

คำนำ

เอกสารนี้เป็นบันทึกสรุปบทเรียนภาษาไทยที่เขียนขึ้นใหม่ทั้งหมด ไม่ใช่คำแปลของหนังสือ ใช้สำหรับทบทวนหัวข้อที่ตรงกับบทที่ 2 (ช่วงหลัง) ถึงบทที่ 7 ของหนังสือ *Introduction to the Theory of Computation* ของ Michael Sipser ตัวอย่างและแผนภาพทั้งหมดในเอกสารนี้จัดทำขึ้นใหม่ ผู้อ่านควรใช้เอกสารนี้ควบคู่กับหนังสือต้นฉบับ ซึ่งมีบทพิสูจน์ฉบับเต็มและแบบฝึกหัด ศัพท์เทคนิคสำคัญจะกำกับภาษาอังกฤษไว้ในวงเล็บเมื่อปรากฏครั้งแรก

สารบัญ

1 ภาษาไม่พึ่งบริบท: รูปแบบปกติ ออโตมาตาแบบกดลง และขีดจำกัด **1**

1.1 รูปแบบปกติของชอมสกี 1

1.1.1 ทบทวนไวยากรณ์ไม่พึ่งบริบท 1

1.1.2 นิยามและทฤษฎีบท 2

1.1.3 ตัวอย่างการแปลง 3

1.1.4 ความยาวของการสืบทอด 3

1.2 ออโตมาตาแบบกดลง 4

1.2.1 แนวคิด 4

1.2.2 นิยามเชิงรูปนัย 5

1.2.3 ตัวอย่าง 6

1.2.4 ความไม่เชิงกำหนดมีผลต่อพลัง 7

1.3 ความสมมูลของ CFG และ PDA 7

1.3.1 จากไวยากรณ์สู่ออโตมาตาแบบกดลง 7

1.3.2 จากออโตมาตาแบบกดลงสู่ไวยากรณ์ 9

1.4 ภาษาที่ไม่ใช่ภาษาไม่พึ่งบริบท 10

1.4.1 บทตั้งการปั๊มสำหรับภาษาไม่พึ่งบริบท 10

1.4.2 การใช้บทตั้งการปั๊ม 11

1.4.3 สมบัติปิดของภาษาไม่พึ่งบริบท 13

สรุปท้ายบท 14

2 เครื่องจักรทัวริงและวิทยานิพนธ์เชิร์ช–ทัวริง **15**

2.1 เครื่องจักรทัวริง 15

2.1.1 ภาพรวมเชิงสัญชาตญาณ 15

2.1.2 นิยามเชิงรูปนัย 16

2.1.3 โครงแบบและความสัมพันธ์ “ให้ผลเป็น” 16

2.1.4 ตัวอย่างระดับสูง: ภาษา $\{0^n1^n2^n\}$ 18

2.1.5 ตัวอย่างละเอียด: ภาษาพาลินโดรม 19

2.2 รูปแบบต่าง ๆ ของเครื่องจักรทัวริง 20

2.2.1 เครื่องที่หัวอ่านอยู่กับที่ได้ 21

2.2.2 เครื่องจักรทัวริงหลายเทป 21

2.2.3 เครื่องจักรทัวริงไม่เชิงกำหนด 22

2.2.4 ตัวแจกแจง 24

2.2.5 ความทนทานของแบบจำลอง 25

2.3 วิทยานิพนธ์เชิร์ช–ทัวริง 25
2.3.1 อัลกอริทึมคืออะไร 25
2.3.2 ระดับของการบรรยายเครื่องจักรทัวริง 26
2.3.3 การเข้ารหัสวัตถุเป็นสตริง 27
2.3.4 ตัวอย่างการบรรยายระดับสูง: กราฟแบ่งสองส่วน 27
สรุปท้ายบท 29

3 ภาษาที่ตัดสินได้ ภาษาที่รู้จำได้ และความตัดสินไม่ได้ 30
3.1 ภาษาที่ตัดสินได้ 30
3.1.1 ปัญหาที่ตัดสินได้เกี่ยวกับภาษาปรกติ 31
3.1.2 ปัญหาที่ตัดสินได้เกี่ยวกับภาษาไม่พึ่งบริบท 33
3.2 ภาษาที่รู้จำได้แบบทัวริง 35
3.2.1 ทบทวนนิยามและความแตกต่างจากความตัดสินได้ 35
3.2.2 ความตัดสินได้ = รู้จำได้ทั้งสองด้าน 36
3.2.3 การจำลองแบบสลับซ้อน (dovetailing) 37
3.3 ความตัดสินไม่ได้ 37
3.3.1 ภาษา A_{TM} และเครื่องจักรทัวริงเอกภพ 37
3.3.2 วิธีแนวทแยง 38
3.3.3 A_{TM} ตัดสินไม่ได้ 40
3.3.4 ภาษาที่ไม่รู้จำได้แบบทัวริงที่เป็นรูปธรรม 41

4 การลดรูปปัญหา (Reducibility) 43
4.1 ปัญหาที่ตัดสินไม่ได้ผ่านการลดรูป 43
4.1.1 แนวคิดของการลดรูป 43
4.1.2 ปัญหาการหยุดทำงาน 43
4.1.3 ปัญหาความว่างของภาษา 44
4.1.4 ปัญหาความปรกติของภาษา 45
4.1.5 ปัญหาความเท่ากันของเครื่องจักรทัวริง 45
4.1.6 ทฤษฎีบทของไรซ์ 45
4.2 การลดรูปผ่านประวัติการคำนวณ 46
4.2.1 ออโตมาตาขอบเขตเชิงเส้น 46
4.2.2 ปัญหาเกี่ยวกับไวยากรณ์ไม่พึ่งบริบท 47
4.3 การลดรูปแบบส่งแผนที่ 47
4.3.1 ฟังก์ชันที่คำนวณได้ 48
4.3.2 นิยามและสมบัติ 48
4.3.3 ภาษาที่ทั้งรู้จำไม่ได้และส่วนเติมเต็มก็รู้จำไม่ได้ 49
4.4 ปัญหาการจับคู่ของโพสต์ 49

5 ทฤษฎีความซับซ้อนเบื้องต้น (Basic Complexity Theory) 51
5.1 การวัดความซับซ้อน 51
5.1.1 เวลาทำงานและสัญกรณ์เชิงเส้นกำกับ 51

5.1.2 ตัวอย่างการวิเคราะห์: ภาษา $\{0^k1^k\}$ 52
5.1.3 ความสัมพันธ์ระหว่างแบบจำลอง 52
5.2 คลาส P 53
5.2.1 ทำไมจึงใช้พหุนาม 53
5.2.2 ตัวอย่างปัญหาใน P 54
5.3 คลาส NP 55
5.3.1 ตัวตรวจสอบและใบรับรอง 55
5.3.2 ตัวอย่างปัญหาใน NP 56
5.3.3 คำถาม P เทียบกับ NP 56
5.4 ความบริบูรณ์เอ็นพี 57
5.4.1 ปัญหาความสอดคล้องได้ 57
5.4.2 การลดรูปเวลาพหุนาม 57
5.4.3 ปัญหา 3SAT และการลดรูปไปยัง CLIQUE 58
5.4.4 นิยามของความบริบูรณ์เอ็นพี 58
5.4.5 ทฤษฎีบทคุก–เลวิน 59
5.4.6 ปัญหาเอ็นพีบริบูรณ์อื่น ๆ 59

บทที่ 1

ภาษาไม่พึ่งบริบท: รูปแบบปกติ ออโตมาตาแบบกดลง และขีดจำกัด

บทนี้ศึกษาภาษาไม่พึ่งบริบท (context-free languages) ต่อจากเรื่องไวยากรณ์ไม่พึ่งบริบทเบื้องต้น เราจะเริ่มจากการจัดไวยากรณ์ให้อยู่ใน**รูปแบบปกติของชอมสกี** ซึ่งช่วยให้การวิเคราะห์ไวยากรณ์ง่ายขึ้นมาก จากนั้นจะแนะนำเครื่องจักรอีกแบบหนึ่งคือ**ออโตมาตาแบบกดลง** ซึ่งเป็นออโตมาตาจำกัดที่มีหน่วยความจำแบบกองซ้อนเพิ่มเข้ามา แล้วแสดงว่าเครื่องจักรนี้มีพลังเท่ากับไวยากรณ์ไม่พึ่งบริบทพอดี สุดท้ายเราจะศึกษา**บทตั้งการปั๊ม**สำหรับภาษาไม่พึ่งบริบท ซึ่งเป็นเครื่องมือหลักในการพิสูจน์ว่าภาษาบางภาษา*ไม่ใช่*ภาษาไม่พึ่งบริบท

1.1 รูปแบบปกติของชอมสกี

1.1.1 ทบทวนไวยากรณ์ไม่พึ่งบริบท

นิยาม 1.1. ไวยากรณ์ไม่พึ่งบริบท (context-free grammar, CFG) คือสิ่งอันดับสี่ $G = (V, \Sigma, R, S)$ โดยที่

(1) V เป็นเซตจำกัดของ**ตัวแปร** (variables) หรือสัญลักษณ์ไม่ปลายทาง

(2) Σ เป็นเซตจำกัดของ**สัญลักษณ์ปลายทาง** (terminals) โดย $V \cap \Sigma = \emptyset$

(3) R เป็นเซตจำกัดของ**กฎ** (rules) แต่ละกฎมีรูป $A \to w$ เมื่อ $A \in V$ และ $w \in (V \cup \Sigma)^*$

(4) $S \in V$ เป็น**ตัวแปรเริ่มต้น** (start variable)

ถ้า $u, v, w \in (V \cup \Sigma)^*$ และ $A \to w$ เป็นกฎ เราเขียน $uAv \Rightarrow uwv$ และกล่าวว่า uAv **ให้ผล** (yields) uwv ในหนึ่งขั้น ถ้ามีลำดับการให้ผลจาก u ไปถึง v ได้ในศูนย์ขั้นขึ้นไป เราเขียน $u \overset{*}{\Rightarrow} v$ และเรียกลำดับนั้นว่า**การสืบทอด** (derivation) ภาษาของไวยากรณ์คือ $L(G) = \{\, w \in \Sigma^* : S \overset{*}{\Rightarrow} w \,\}$ และภาษาใดที่มีไวยากรณ์ไม่พึ่งบริบทสร้างได้ เรียกว่า**ภาษาไม่พึ่งบริบท** (context-free language, CFL) การสืบทอดแต่ละครั้งแทนได้ด้วย**ต้นไม้แจงส่วน** (parse tree) ซึ่งโหนดภายในเป็นตัวแปร และใบเป็นสัญลักษณ์ปลายทางหรือ ε

ไวยากรณ์ทั่วไปอาจมีกฎหน้าตาหลากหลาย เช่น กฎที่ฝั่งขวายาวมาก กฎว่าง $A \to \varepsilon$ หรือกฎที่แค่เปลี่ยนชื่อตัวแปร $A \to B$ ซึ่งทำให้การให้เหตุผลเกี่ยวกับไวยากรณ์ยุ่งยาก รูปแบบปกติที่จะนิยามต่อไปนี้จำกัดรูปของกฎให้เหลือเพียงสองแบบ

1.1.2 นิยามและทฤษฎีบท

นิยาม 1.2. ไวยากรณ์ไม่พึ่งบริบท $G = (V, \Sigma, R, S)$ อยู่ใน**รูปแบบปกติของชอมสกี** (Chomsky normal form, CNF) ถ้าทุกกฎมีรูปแบบใดแบบหนึ่งต่อไปนี้

$$A \to BC \qquad \text{หรือ} \qquad A \to a$$

เมื่อ $A, B, C \in V$, $a \in \Sigma$ และ B, C ไม่ใช่ตัวแปรเริ่มต้น นอกจากนี้อนุญาตให้มีกฎ $S \to \varepsilon$ ได้เพียงกฎเดียวเพื่อรองรับกรณีที่ ε อยู่ในภาษา

ทฤษฎีบท 1.3. *ภาษาไม่พึ่งบริบททุกภาษาถูกสร้างได้ด้วยไวยากรณ์ไม่พึ่งบริบทที่อยู่ในรูปแบบปกติของชอมสกี*

แนวคิดการพิสูจน์. เราแปลงไวยากรณ์ G ใด ๆ ทีละขั้นโดยไม่เปลี่ยนภาษาที่สร้าง แต่ละขั้นกำจัดกฎ “ที่ผิดรูป” ไปหนึ่งชนิด

ขั้นที่ 1 เพิ่มตัวแปรเริ่มต้นใหม่ สร้างตัวแปร S_0 และกฎ $S_0 \to S$ แล้วให้ S_0 เป็นตัวแปรเริ่มต้น ขั้นนี้รับประกันว่าตัวแปรเริ่มต้นจะไม่ปรากฏทางขวาของกฎใดเลย

ขั้นที่ 2 กำจัดกฎว่าง (ε-rules) เลือกกฎ $A \to \varepsilon$ ที่ $A \neq S_0$ แล้วลบทิ้ง จากนั้นสำหรับทุกกฎที่มี A อยู่ฝั่งขวา ให้เพิ่มกฎใหม่ที่ได้จากการลบ A ออกในทุกวิธีที่เป็นไปได้ เช่น จากกฎ $X \to uAvAw$ จะได้กฎเพิ่ม $X \to uvAw$, $X \to uAvw$ และ $X \to uvw$ ถ้ามีกฎ $X \to A$ ให้เพิ่ม $X \to \varepsilon$ (ยกเว้นกรณีที่เคยกำจัด $X \to \varepsilon$ ไปแล้ว) ทำซ้ำจนไม่เหลือกฎว่างนอกจาก $S_0 \to \varepsilon$

ขั้นที่ 3 กำจัดกฎหน่วย (unit rules) เลือกกฎ $A \to B$ แล้วลบทิ้ง จากนั้นสำหรับทุกกฎ $B \to w$ ให้เพิ่มกฎ $A \to w$ (ยกเว้นกฎหน่วยที่เคยกำจัดไปแล้ว) ทำซ้ำจนไม่เหลือกฎหน่วย

ขั้นที่ 4 จัดรูปกฎที่เหลือ สำหรับกฎ $A \to u_1 u_2 \cdots u_k$ ที่ $k \geq 3$ ให้แทนด้วยสายโซ่

$$A \to u_1 A_1, \quad A_1 \to u_2 A_2, \quad \ldots, \quad A_{k-2} \to u_{k-1} u_k$$

โดย $A_1, \ldots, A_{k-2}$ เป็นตัวแปรใหม่ และในทุกกฎที่ฝั่งขวายาวตั้งแต่สองสัญลักษณ์ ถ้ามีสัญลักษณ์ปลายทาง a ปรากฏ ให้แทน a ด้วยตัวแปรใหม่ U_a พร้อมเพิ่มกฎ $U_a \to a$

แต่ละขั้นคงภาษาไว้เหมือนเดิม และเมื่อจบขั้นที่ 4 ทุกกฎจะอยู่ในรูป $A \to BC$, $A \to a$ หรือ $S_0 \to \varepsilon$ □

หมายเหตุ. ลำดับของขั้นตอนมีความสำคัญ ถ้ากำจัดกฎหน่วยก่อนกฎว่าง การกำจัดกฎว่างอาจสร้างกฎหน่วยขึ้นมาใหม่ได้ เช่น จาก $X \to AB$ เมื่อ $B \to \varepsilon$ ถูกลบ จะได้ $X \to A$ ซึ่งเป็นกฎหน่วย

1.1.3 ตัวอย่างการแปลง

ตัวอย่าง 1.4. พิจารณาไวยากรณ์ G_1 ที่มีกฎ

$$S \to aSc \mid B, \qquad B \to bB \mid \varepsilon$$

ไวยากรณ์นี้สร้างภาษา $\{\, a^n b^m c^n : n, m \geq 0 \,\}$ กล่าวคือ a และ c ต้องมีจำนวนเท่ากันและครอบ b ไว้ตรงกลาง เราจะแปลง G_1 ให้อยู่ในรูปแบบปกติของชอมสกี

ขั้นที่ 1 เพิ่มตัวแปรเริ่มต้นใหม่

$$S_0 \to S, \qquad S \to aSc \mid B, \qquad B \to bB \mid \varepsilon$$

ขั้นที่ 2 กำจัด $B \to \varepsilon$ ก่อน ตัวแปร B ปรากฏทางขวาในกฎ $S \to B$ และ $B \to bB$ จึงต้องเพิ่ม $S \to \varepsilon$ และ $B \to b$ ได้

$$S_0 \to S, \qquad S \to aSc \mid B \mid \varepsilon, \qquad B \to bB \mid b$$

ตอนนี้มีกฎว่างใหม่คือ $S \to \varepsilon$ ตัวแปร S ปรากฏทางขวาในกฎ $S_0 \to S$ และ $S \to aSc$ จึงเพิ่ม $S_0 \to \varepsilon$ และ $S \to ac$ ได้

$$S_0 \to S \mid \varepsilon, \qquad S \to aSc \mid ac \mid B, \qquad B \to bB \mid b$$

ขั้นที่ 3 กำจัดกฎหน่วย เริ่มจาก $S \to B$ โดยนำกฎของ B มาให้ S

$$S_0 \to S \mid \varepsilon, \qquad S \to aSc \mid ac \mid bB \mid b, \qquad B \to bB \mid b$$

จากนั้นกำจัด $S_0 \to S$ โดยนำกฎทั้งหมดของ S มาให้ S_0

$$S_0 \to aSc \mid ac \mid bB \mid b \mid \varepsilon, \qquad S \to aSc \mid ac \mid bB \mid b, \qquad B \to bB \mid b$$

ขั้นที่ 4 กฎ aSc ยาวสามสัญลักษณ์ จึงแยกเป็น U_aT กับ $T \to SU_c$ และแทนสัญลักษณ์ปลายทางในกฎยาวด้วย U_a, U_b, U_c ผลลัพธ์สุดท้ายคือ

$$\begin{aligned} S_0 &\to U_aT \mid U_aU_c \mid U_bB \mid b \mid \varepsilon \\ S &\to U_aT \mid U_aU_c \mid U_bB \mid b \\ T &\to SU_c \\ B &\to U_bB \mid b \\ U_a &\to a, \qquad U_b \to b, \qquad U_c \to c \end{aligned}$$

ทุกกฎอยู่ในรูปที่กำหนด และ S_0 ไม่ปรากฏทางขวาของกฎใด

1.1.4 ความยาวของการสืบทอด

ข้อดีอย่างหนึ่งของรูปแบบปกติของชอมสกีคือต้นไม้แจงส่วนเป็น*ต้นไม้ทวิภาค* (binary tree) เสมอ และจำนวนขั้นของการสืบทอดถูกกำหนดแน่นอนโดยความยาวของสตริง

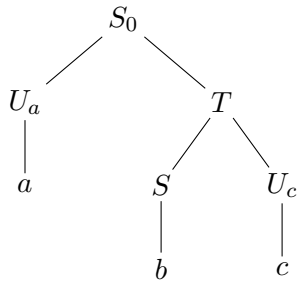
บทตั้ง 1.5. *ถ้า G อยู่ในรูปแบบปกติของชอมสกี และ $w \in L(G)$ มีความยาว $n \geq 1$ แล้วทุกการสืบทอดของ w ใช้กฎทั้งหมด $2n - 1$ ขั้นพอดี*

พิสูจน์. กฎแบบ $A \to BC$ เพิ่มจำนวนตัวแปรในสตริงขึ้นหนึ่ง ส่วนกฎแบบ $A \to a$ เปลี่ยนตัวแปรหนึ่งตัวเป็นสัญลักษณ์ปลายทางหนึ่งตัว การสืบทอดเริ่มจากตัวแปรหนึ่งตัว และต้องจบด้วยสัญลักษณ์ปลายทาง n ตัว จึงต้องใช้กฎแบบ $A \to a$ พอดี n ครั้ง และเพื่อให้มีตัวแปรครบ n ตัวต้องใช้กฎแบบ $A \to BC$ พอดี $n-1$ ครั้ง (กฎ $S_0 \to \varepsilon$ ใช้ไม่ได้เพราะ $n \geq 1$) รวมเป็น $2n-1$ ขั้น □

ตัวอย่าง 1.6. ด้วยไวยากรณ์ผลลัพธ์ในตัวอย่างที่ 1.4 สตริง abc (ยาว 3) มีการสืบทอด

$$S_0 \Rightarrow U_aT \Rightarrow aT \Rightarrow aSU_c \Rightarrow abU_c \Rightarrow abc$$

ซึ่งใช้ $5 = 2 \cdot 3 - 1$ ขั้น ตรงตามบทตั้ง ต้นไม้แจงส่วนแสดงในรูปที่ 1.1



รูปที่ 1.1: ต้นไม้แจงส่วนของ abc ในไวยากรณ์รูปแบบปกติของชอมสกี: โหนดภายในแตกกิ่งเป็นสองเสมอ ยกเว้นโหนดที่อยู่เหนือใบ

ผลที่ตามมาคือ ถ้าต้องการทดสอบว่า w อยู่ใน $L(G)$ หรือไม่ ก็เพียงตรวจการสืบทอดทั้งหมดที่ยาว $2n-1$ ขั้น ซึ่งมีจำนวนจำกัด ข้อเท็จจริงนี้จะถูกใช้อีกครั้งในบทที่ว่าด้วยความตัดสินได้

1.2 ออโตมาตาแบบกดลง

1.2.1 แนวคิด

ออโตมาตาจำกัดมีหน่วยความจำเพียงสถานะซึ่งมีจำนวนจำกัด จึงไม่สามารถ “นับ” ได้ไม่จำกัด เช่น ไม่สามารถจำได้ว่าอ่าน a ไปแล้วกี่ตัวเพื่อเทียบกับจำนวน c ที่ตามมา **ออโตมาตาแบบกดลง** (pushdown automaton, PDA) แก้ข้อจำกัดนี้โดยเพิ่ม**กองซ้อน** (stack) ที่มีขนาดไม่จำกัด แต่เข้าถึงได้เฉพาะด้านบนสุดเท่านั้น

- **กด** (push) คือการวางสัญลักษณ์ใหม่ไว้บนสุด
- **ดึง** (pop) คือการนำสัญลักษณ์บนสุดออก

กองซ้อนทำงานแบบ “เข้าหลังออกก่อน” (last in, first out) ซึ่งเหมาะกับโครงสร้างที่ซ้อนกัน เช่น วงเล็บ หรือรูปแบบ “ครึ่งหน้าสะท้อนครึ่งหลัง” ในแต่ละขั้น PDA ดูสถานะปัจจุบัน สัญลักษณ์อินพุตถัดไป (หรือไม่อ่านก็ได้) และสัญลักษณ์บนสุดของกองซ้อน (หรือไม่ดูก็ได้) แล้วเลือกสถานะถัดไปและสิ่งที่จะวางแทนบนกองซ้อน

ในบทนี้ PDA เป็นแบบ**ไม่เชิงกำหนด** (nondeterministic) โดยปริยาย คือในแต่ละขั้นอาจมีทางเลือกหลายทาง และเครื่องยอมรับสตริงถ้ามี*อย่างน้อยหนึ่ง*ทางเลือกที่นำไปสู่การยอมรับ

1.2.2 นิยามเชิงรูปนัย

เขียน $\Sigma_\varepsilon = \Sigma \cup \{\varepsilon\}$ และ $\Gamma_\varepsilon = \Gamma \cup \{\varepsilon\}$ และ $\mathcal{P}(X)$ แทนเซตกำลังของ X

นิยาม 1.7. ออโตมาตาแบบกดลง คือสิ่งอันดับหก $M = (Q, \Sigma, \Gamma, \delta, q_0, F)$ โดยที่

(1) Q เป็นเซตจำกัดของสถานะ

(2) Σ เป็นเซตจำกัดเรียกว่า**อักษรอินพุต** (input alphabet)

(3) Γ เป็นเซตจำกัดเรียกว่า**อักษรกองซ้อน** (stack alphabet)

(4) $\delta : Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \to \mathcal{P}(Q \times \Gamma_\varepsilon)$ เป็น**ฟังก์ชันการเปลี่ยนสถานะ** (transition function)

(5) $q_0 \in Q$ เป็นสถานะเริ่มต้น

(6) $F \subseteq Q$ เป็นเซตของสถานะยอมรับ

ถ้า $(r, c) \in \delta(q, a, b)$ หมายความว่า เมื่ออยู่ที่สถานะ q อ่านอินพุต a และสัญลักษณ์บนสุดของกองซ้อนคือ b เครื่องสามารถไปที่สถานะ r โดยแทน b ด้วย c บนกองซ้อน ในแผนภาพสถานะเราเขียนป้ายของลูกศรจาก q ไป r ว่า

$$a,\ b \to c$$

ค่าใดค่าหนึ่งในสามตัวอาจเป็น ε ได้ ซึ่งให้ความหมายดังนี้

ป้าย	ความหมาย
$a,\ \varepsilon \to c$	อ่าน a แล้วกด c (ไม่ดึงอะไร)
$a,\ b \to \varepsilon$	อ่าน a แล้วดึง b ออก (ไม่กดอะไร)
$\varepsilon,\ b \to c$	ไม่อ่านอินพุต แทนบนสุด b ด้วย c
$a,\ \varepsilon \to \varepsilon$	อ่าน a โดยไม่แตะกองซ้อน

นิยาม 1.8. PDA M **ยอมรับ** (accepts) สตริง w ถ้าเขียน $w = w_1 w_2 \cdots w_m$ โดย $w_i \in \Sigma_\varepsilon$ ได้ และมีลำดับสถานะ $r_0, r_1, \ldots, r_m \in Q$ กับลำดับสตริง $s_0, s_1, \ldots, s_m \in \Gamma^*$ (แทนเนื้อหากองซ้อน โดยตัวซ้ายสุดคือบนสุด) ซึ่งสอดคล้องกับเงื่อนไขทั้งสามต่อไปนี้

(i) $r_0 = q_0$ และ $s_0 = \varepsilon$ (เริ่มที่สถานะเริ่มต้นด้วยกองซ้อนว่าง)

(ii) สำหรับ $i = 0, \ldots, m-1$ มี $a, b \in \Gamma_\varepsilon$ และ $t \in \Gamma^*$ ที่ $s_i = at$, $s_{i+1} = bt$ และ $(r_{i+1}, b) \in \delta(r_i, w_{i+1}, a)$

(iii) $r_m \in F$

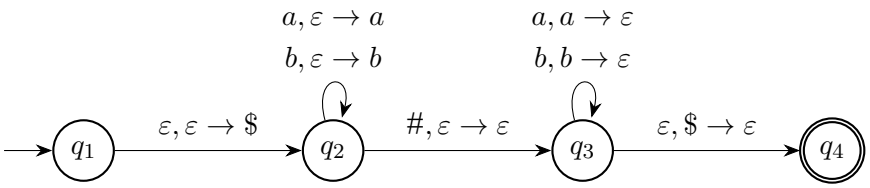
ภาษาของ M คือ $L(M) = \{w : M \text{ ยอมรับ } w\}$

สังเกตว่านิยามไม่มีวิธีตรวจโดยตรงว่ากองซ้อนว่างหรือไม่ เทคนิคมาตรฐานคือกดสัญลักษณ์พิเศษ \$ ลงไปก่อนเป็นอันดับแรก เมื่อใดที่เห็น \$ อยู่บนสุดก็แปลว่ากองซ้อน “ว่าง” ในเชิงตรรกะ และสังเกตด้วยว่าเครื่องยอมรับได้เฉพาะเมื่ออ่านอินพุตหมดแล้วเท่านั้น

1.2.3 ตัวอย่าง

ตัวอย่าง 1.9. ให้ $L_1 = \{\, w\,\#\,w^{\mathcal{R}} : w \in \{a,b\}^*\,\}$ เมื่อ $w^{\mathcal{R}}$ คือ w กลับด้าน เช่น $ab\#ba$, # และ $bba\#abb$ อยู่ใน L_1 แต่ $ab\#ab$ ไม่อยู่ สัญลักษณ์ # ทำหน้าที่เป็นตัวบอกจุดกึ่งกลาง

แนวคิดของเครื่องคือ ก่อนพบ # ให้กดทุกตัวอักษรที่อ่านลงกองซ้อน เมื่อพบ # ให้เปลี่ยนโหมด แล้วเทียบตัวอักษรที่อ่านกับตัวบนสุดของกองซ้อนทีละตัว เนื่องจากกองซ้อนคืนค่าในลำดับกลับกัน การเทียบนี้จึงเทียบ $w^{\mathcal{R}}$ กับ w ที่กลับด้านพอดี เมื่อเหลือแค่ \$ ก็ยอมรับ แผนภาพสถานะแสดงในรูปที่ 1.2 โดยมี $\Sigma = \{a,b,\#\}$, $\Gamma = \{a,b,\$\}$ และ $F = \{q_4\}$



รูปที่ 1.2: PDA สำหรับ $L_1 = \{w\#w^{\mathcal{R}}\}$: สถานะ q_2 เก็บครึ่งแรก และสถานะ q_3 ตรวจครึ่งหลัง

ตารางต่อไปนี้แสดงการทำงานของเครื่องเมื่อรับอินพุต $ab\#ba$ (เนื้อหากองซ้อนเขียนให้ตัวซ้ายสุดเป็นบนสุด)

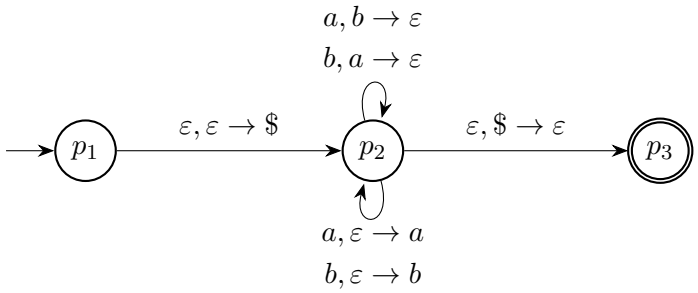
ขั้น	สถานะ	อินพุตที่เหลือ	กองซ้อน
0	q_1	$ab\#ba$	ε
1	q_2	$ab\#ba$	\$
2	q_2	$b\#ba$	a\$
3	q_2	$\#ba$	ba\$
4	q_3	ba	ba\$
5	q_3	a	a\$
6	q_3	ε	\$
7	q_4	ε	ε

อินพุตหมดเมื่อเครื่องอยู่ที่ q_4 ซึ่งเป็นสถานะยอมรับ จึงยอมรับ $ab\#ba$ ในทางกลับกัน สำหรับ $ab\#ab$ เมื่ออยู่ที่ q_3 ตัวบนสุดคือ b แต่อ่านได้ a จึงไม่มีลูกศรให้ไปต่อ เครื่องติดค้างและไม่ยอมรับ

ตัวอย่าง 1.10. ให้ $L_2 = \{\, w \in \{a,b\}^* : w$ มีจำนวน a เท่ากับจำนวน $b\,\}$ เช่น ε, $abba$, $bbaaba$ อยู่ใน L_2 แต่ aab ไม่อยู่ ต่างจาก L_1 ตรงที่ตัวอักษรทั้งสองชนิดสลับกันได้ตามอำเภอใจ

แนวคิดคือ ให้กองซ้อนเก็บ “ส่วนเกิน” ของตัวอักษรชนิดที่นำอยู่ในขณะนั้น เมื่ออ่าน a ถ้าบนสุดเป็น b ก็ให้หักล้างกัน (ดึง b ออก) มิฉะนั้นให้กด a และทำทำนองเดียวกันเมื่ออ่าน b เมื่ออ่านจบ ถ้าบนสุดเป็น \$ แปลว่าไม่มีส่วนเกิน จำนวนจึงเท่ากัน แผนภาพแสดงในรูปที่ 1.3

เครื่องนี้ไม่เชิงกำหนด เพราะเมื่ออ่าน a ขณะที่บนสุดเป็น b เครื่องสามารถเลือกกด a ทับลงไปแทนการหักล้างก็ได้ แต่การเลือกแบบนั้นไม่ทำให้เครื่องยอมรับสตริงผิด เหตุผลมีดังนี้ ทุกครั้งที่อ่านตัวอักษรหนึ่งตัว เครื่องจะกดสัญลักษณ์ชนิดเดียวกันหนึ่งตัว หรือดึงสัญลักษณ์ชนิดตรงข้ามหนึ่งตัว ถ้าเครื่องจบที่ p_3 ได้ กองซ้อนต้องเหลือแค่ \$ ดังนั้นสัญลักษณ์ a ทุกตัวที่ถูกกดถูกดึงออกด้วยการอ่าน b



รูปที่ 1.3: PDA สำหรับ L_2: วงด้านบนหักล้างตัวอักษรต่างชนิดกัน วงด้านล่างกดส่วนเกินลงกองซ้อน

และสัญลักษณ์ b ทุกตัวที่ถูกกดถูกดึงออกด้วยการอ่าน a ถ้าให้ x และ y เป็นจำนวนครั้งที่กด a และกด b ตามลำดับ จะได้ว่าจำนวน a ที่อ่าน $= x + y =$ จำนวน b ที่อ่าน ในทางกลับกัน ถ้า $w \in L_2$ การเลือกหักล้างทุกครั้งที่ทำได้จะนำไปสู่การยอมรับเสมอ

1.2.4 ความไม่เชิงกำหนดมีผลต่อพลัง

สำหรับออโตมาตาจำกัด แบบเชิงกำหนดและแบบไม่เชิงกำหนดมีพลังเท่ากัน แต่สำหรับ PDA ไม่เป็นเช่นนั้น **ออโตมาตาแบบกดลงเชิงกำหนด** (deterministic PDA, DPDA) คือ PDA ที่ในทุกสถานการณ์มีทางเลือกให้ไปต่อได้ไม่เกินหนึ่งทาง (รวมถึงไม่มีความขัดแย้งระหว่างการเคลื่อนที่แบบอ่านอินพุตและแบบ ε) ภาษาที่ DPDA รู้จำได้เรียกว่า**ภาษาไม่พึ่งบริบทเชิงกำหนด** (deterministic CFL)

หมายเหตุ. PDA แบบไม่เชิงกำหนดมีพลังมากกว่าแบบเชิงกำหนดอย่างแท้จริง ตัวอย่างเช่น ภาษา $\{ ww^{\mathcal{R}} : w \in \{a, b\}^* \}$ (พาลินโดรมความยาวคู่) เป็นภาษาไม่พึ่งบริบท เพราะ PDA ไม่เชิงกำหนดสามารถ “เดา” ได้ว่าจุดกึ่งกลางอยู่ตรงไหน โดยใช้เครื่องคล้ายรูปที่ 1.2 แต่แทนลูกศร $\#, \varepsilon \to \varepsilon$ ด้วย $\varepsilon, \varepsilon \to \varepsilon$ อย่างไรก็ตาม สามารถพิสูจน์ได้ว่าไม่มี DPDA ใดรู้จำภาษานี้ได้ เพราะเครื่องเชิงกำหนดไม่มีทางรู้ว่าเมื่อใดควรเปลี่ยนจากการกดเป็นการดึง ภาษา L_1 ในตัวอย่างที่ 1.9 เป็นภาษาไม่พึ่งบริบทเชิงกำหนด เพราะ # บอกจุดกึ่งกลางไว้ให้แล้ว ภาษาไม่พึ่งบริบทเชิงกำหนดมีความสำคัญในทางปฏิบัติ เพราะเป็นพื้นฐานของตัวแจงส่วน (parser) ที่ทำงานได้รวดเร็วในคอมไพเลอร์

1.3 ความสมมูลของ CFG และ PDA

ทฤษฎีบท 1.11. *ภาษาหนึ่งเป็นภาษาไม่พึ่งบริบทก็ต่อเมื่อมีออโตมาตาแบบกดลงที่รู้จำภาษานั้น*

การพิสูจน์แบ่งเป็นสองทิศทาง ซึ่งเราจะแยกเป็นบทตั้งสองบท

1.3.1 จากไวยากรณ์สู่ออโตมาตาแบบกดลง

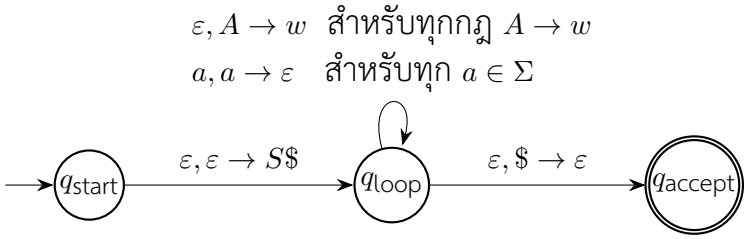
บทตั้ง 1.12. *ถ้า A เป็นภาษาไม่พึ่งบริบท แล้วมี PDA ที่รู้จำ A*

แนวคิดการพิสูจน์. ให้ G เป็นไวยากรณ์ที่สร้าง A เราสร้าง PDA P ที่ *จำลองการสืบทอดซ้ายสุด* (left-most derivation) ของ G โดยใช้กองซ้อนเก็บส่วนของสตริงกลางทาง (sentential form) ที่ยังไม่ได้ตรวจกับอินพุต P ทำงานดังนี้

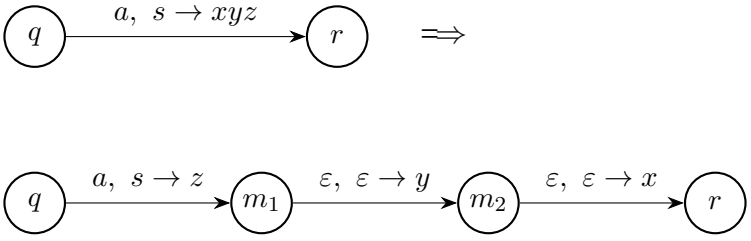
1. กด $ แล้วกดตัวแปรเริ่มต้น S ลงกองซ้อน
2. ทำซ้ำขั้นต่อไปนี้
   - ถ้าบนสุดเป็นตัวแปร A ให้เลือกกฎ $A \to w$ อย่างไม่เชิงกำหนด แล้วแทน A ด้วยสตริง w บนกองซ้อน
   - ถ้าบนสุดเป็นสัญลักษณ์ปลายทาง a ให้อ่านอินพุตหนึ่งตัวและเทียบกับ a ถ้าตรงกันให้ดึงออก ถ้าไม่ตรงให้สาขานี้ล้มเหลว
   - ถ้าบนสุดเป็น $ ให้ไปยังสถานะยอมรับ (ซึ่งจะยอมรับได้จริงก็ต่อเมื่ออ่านอินพุตหมดแล้ว)

สังเกตว่าตัวแปรที่อยู่ซ้ายสุดในสตริงกลางทางจะไปอยู่บนสุดของกองซ้อนเสมอ หลังจากสัญลักษณ์ปลายทางที่นำหน้าถูกเทียบกับอินพุตหมดแล้ว ดังนั้นแต่ละสาขาที่ยอมรับของ P สอดคล้องกับการสืบทอดซ้ายสุดของอินพุตพอดี □

PDA ที่ได้มีเพียงสามสถานะหลัก ดังรูปที่ 1.4 ในรูปนี้ป้าย $\varepsilon, A \to w$ หมายถึงการแทนตัวบนสุด A ด้วย*ทั้งสตริง* w ในขั้นเดียว ซึ่งนิยามเดิมไม่อนุญาต แต่เป็นตัวย่อที่จำลองได้ด้วยสถานะเพิ่มเติม ดังรูปที่ 1.5



รูปที่ 1.4: โครงสร้าง PDA สามสถานะที่จำลองไวยากรณ์ G



รูปที่ 1.5: การแทนตัวย่อ “กดทั้งสตริง xyz” ด้วยสถานะใหม่ m_1, m_2: กดจากตัวท้ายก่อน เพื่อให้ x อยู่บนสุด

ตัวอย่าง 1.13. นำไวยากรณ์ G_1 จากตัวอย่างที่ 1.4 ($S \to aSc \mid B$, $B \to bB \mid \varepsilon$) มาสร้าง PDA ตามโครงสร้างข้างต้น วงที่ q_{loop} จะมีป้ายดังนี้

$$\varepsilon, S \to aSc \qquad \varepsilon, S \to B \qquad \varepsilon, B \to bB \qquad \varepsilon, B \to \varepsilon$$
$$a, a \to \varepsilon \qquad b, b \to \varepsilon \qquad c, c \to \varepsilon$$

เมื่อรับอินพุต abc เนื้อหากองซ้อนในสาขาที่ยอมรับเปลี่ยนไปดังนี้

$$S\$ \ \to\ aSc\$ \xrightarrow{\text{อ่าน } a}\ Sc\$ \ \to\ Bc\$ \ \to\ bBc\$ \xrightarrow{\text{อ่าน } b}\ Bc\$ \ \to\ c\$ \xrightarrow{\text{อ่าน } c}\ \$$$

ซึ่งสอดคล้องกับการสืบทอดซ้ายสุด $S \Rightarrow aSc \Rightarrow aBc \Rightarrow abBc \Rightarrow abc$

1.3.2 จากออโตมาตาแบบกดลงสู่ไวยากรณ์

บทตั้ง 1.14. *ถ้า PDA ใดรู้จำภาษา A แล้ว A เป็นภาษาไม่พึ่งบริบท*

ทิศทางนี้ยากกว่า เริ่มจากปรับ PDA P ให้มีสมบัติสามข้อ (ทำได้เสมอโดยไม่เปลี่ยนภาษา)

(a) มีสถานะยอมรับเพียงสถานะเดียว q_{accept}

(b) ก่อนยอมรับ กองซ้อนต้องว่างเปล่า (เพิ่มสถานะที่วนดึงทุกอย่างออกก่อนเข้า q_{accept})

(c) ทุกการเปลี่ยนสถานะ*กดหนึ่งตัวหรือดึงหนึ่งตัวอย่างใดอย่างหนึ่งเท่านั้น* (การเปลี่ยนที่ทั้งดึงและกดแยกเป็นสองขั้นผ่านสถานะกลาง ส่วนการเปลี่ยนที่ไม่แตะกองซ้อนแทนด้วยการกดสัญลักษณ์หลอกแล้วดึงออกทันที)

แนวคิดการพิสูจน์. สำหรับทุกคู่สถานะ p, q ของ P สร้างตัวแปร A_{pq} โดยตั้งใจให้

> A_{pq} สร้างสตริง x ได้ ก็ต่อเมื่อ x พา P จากสถานะ p ที่กองซ้อนว่าง ไปยังสถานะ q ที่กองซ้อนว่าง

(การเคลื่อนที่เช่นนี้ยังทำได้เหมือนเดิมเมื่อมีของอยู่ใต้กองซ้อน เพราะไม่เคยแตะของเหล่านั้นเลย) ตัวแปรเริ่มต้นคือ $A_{q_0 q_{\text{accept}}}$

พิจารณาการคำนวณใด ๆ ที่อ่าน x แล้วพาจาก p (กองซ้อนว่าง) ไป q (กองซ้อนว่าง) ซึ่งมีอย่างน้อยหนึ่งขั้น เพราะสมบัติ (c) ขั้นแรกต้องเป็นการกด และขั้นสุดท้ายต้องเป็นการดึง มีได้สองกรณี (ดูรูปที่ 1.6)

กรณีที่ 1 สัญลักษณ์ที่ถูกดึงในขั้นสุดท้ายคือตัวเดียวกับที่ถูกกดในขั้นแรก แปลว่ากองซ้อนไม่ว่างเลยระหว่างทาง ให้ขั้นแรกอ่าน a และไปยัง r และขั้นสุดท้ายอ่าน b จาก s ส่วนตรงกลางพาจาก r ไป s โดยไม่แตะสัญลักษณ์ที่กดไว้ จึงสร้างได้จาก A_{rs} เราจึงใส่กฎ

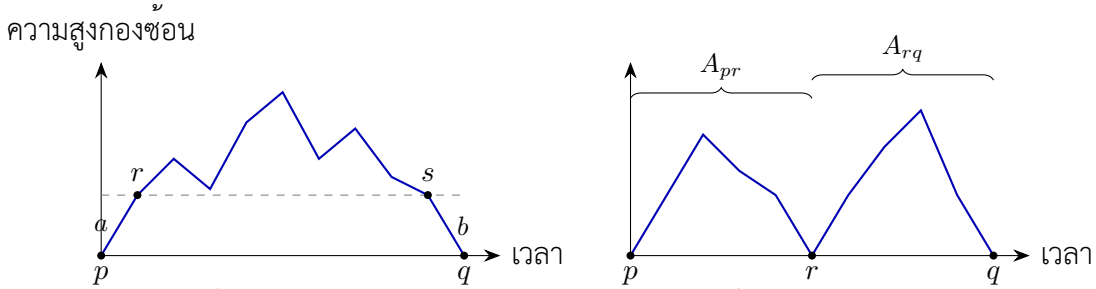
$$A_{pq} \to aA_{rs}b \quad \text{เมื่อ } (r, u) \in \delta(p, a, \varepsilon) \text{ และ } (q, \varepsilon) \in \delta(s, b, u)$$

กรณีที่ 2 กองซ้อนกลับมาว่างที่จุดหนึ่งระหว่างทาง ที่สถานะ r การคำนวณจึงแบ่งเป็นสองช่วง คือจาก p ไป r และจาก r ไป q ทั้งคู่เริ่มและจบด้วยกองซ้อนว่าง เราจึงใส่กฎ

$$A_{pq} \to A_{pr}A_{rq} \quad \text{สำหรับทุก } p, q, r \in Q$$

นอกจากนี้ใส่กฎ $A_{pp} \to \varepsilon$ สำหรับทุก p สำหรับการคำนวณที่ไม่มีขั้นใดเลย การพิสูจน์ว่าไวยากรณ์นี้ถูกต้องทำโดยอุปนัยสองทาง: อุปนัยบนจำนวนขั้นของการสืบทอดเพื่อแสดงว่า ถ้า $A_{pq} \overset{*}{\Rightarrow} x$ แล้ว x พา p ไป q ได้จริง และอุปนัยบนจำนวนขั้นของการคำนวณเพื่อแสดงทิศทางกลับ □

บทตั้งที่ 1.12 และ 1.14 รวมกันพิสูจน์ทฤษฎีบทที่ 1.11 ผลที่ตามมาทันทีคือความสัมพันธ์ระหว่างภาษาปรกติกับภาษาไม่พึ่งบริบท



กรณีที่ 1: $A_{pq} \to aA_{rs}b$ กรณีที่ 2: $A_{pq} \to A_{pr}A_{rq}$

รูปที่ 1.6: ความสูงของกองซ้อนตามเวลาระหว่างการคำนวณจาก p ไป q: ซ้าย กองซ้อนไม่กลับมาว่างระหว่างทาง (สัญลักษณ์แรกที่กดถูกดึงในขั้นสุดท้าย); ขวา กองซ้อนว่างที่สถานะกลาง r

บทแทรก 1.15. *ภาษาปรกติ (regular language) ทุกภาษาเป็นภาษาไม่พึ่งบริบท*

พิสูจน์. ภาษาปรกติทุกภาษามีออโตมาตาจำกัดที่รู้จำ ออโตมาตาจำกัดใด ๆ เป็น PDA ที่ไม่ใช้กองซ้อนเลย (ทุกลูกศรมีป้ายแบบ $a, \varepsilon \to \varepsilon$) จากทฤษฎีบทที่ 1.11 ภาษานั้นจึงเป็นภาษาไม่พึ่งบริบท □

การรวมนี้เป็นการรวมแบบแท้ เพราะ L_1 และ L_2 ในหัวข้อก่อนเป็นภาษาไม่พึ่งบริบทที่ไม่ใช่ภาษาปรกติ

1.4 ภาษาที่ไม่ใช่ภาษาไม่พึ่งบริบท

ภาษาไม่พึ่งบริบทมีพลังมากกว่าภาษาปรกติ แต่ก็ยังมีขีดจำกัด กองซ้อนเพียงอันเดียวจับคู่ได้ทีละ “สองฝั่ง” เช่น a กับ c ใน $a^n b^m c^n$ แต่ไม่สามารถตรวจความสัมพันธ์ที่ต้องจำข้อมูลเดียวกันไว้ใช้หลายครั้งได้ เครื่องมือที่ใช้แสดงขีดจำกัดนี้คือบทตั้งการปั๊ม

1.4.1 บทตั้งการปั๊มสำหรับภาษาไม่พึ่งบริบท

ทฤษฎีบท 1.16 (บทตั้งการปั๊ม (pumping lemma) สำหรับภาษาไม่พึ่งบริบท)**.** *ถ้า A เป็นภาษาไม่พึ่งบริบท แล้วมีจำนวนเต็มบวก p (เรียกว่า**ความยาวปั๊ม**, pumping length) ซึ่งทำให้ทุกสตริง $s \in A$ ที่ $|s| \geq p$ สามารถแบ่งเป็น $s = uvxyz$ ได้ โดยเป็นไปตามเงื่อนไขทั้งสาม*

(1) $uv^i xy^i z \in A$ *สำหรับทุก* $i \geq 0$

(2) $|vy| > 0$

(3) $|vxy| \leq p$

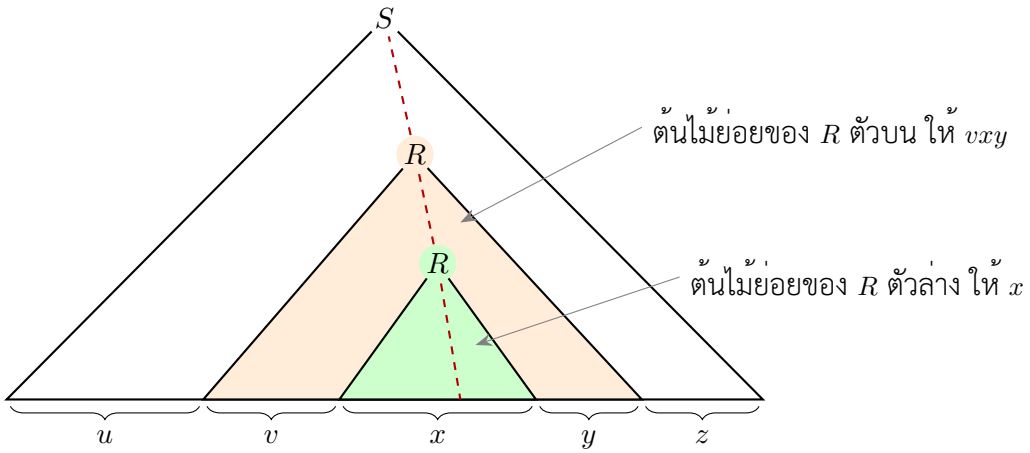
เปรียบเทียบกับบทตั้งการปั๊มของภาษาปรกติ ซึ่งปั๊มชิ้นเดียว ($xy^i z$) ในที่นี้เราปั๊มสองชิ้น v และ y *พร้อมกัน* ด้วยจำนวนครั้งเท่ากัน เงื่อนไข (2) บอกว่าอย่างน้อยหนึ่งในสองชิ้นไม่ว่าง และเงื่อนไข (3) บอกว่าทั้งสองชิ้นรวมทั้งส่วนตรงกลางอยู่ใกล้กัน

แนวคิดการพิสูจน์. ให้ $G = (V, \Sigma, R, S)$ เป็นไวยากรณ์ที่สร้าง A และให้ $b \geq 2$ เป็นความยาวมากที่สุดของฝั่งขวาของกฎใน G ในต้นไม้แจงส่วน แต่ละโหนดมีลูกไม่เกิน b ลูก ดังนั้นต้นไม้ที่สูง h (จำนวนเส้นเชื่อมบนเส้นทางที่ยาวที่สุดจากรากถึงใบ) มีใบไม่เกิน b^h ใบ จึงให้สตริงยาวไม่เกิน b^h

เลือก $p = b^{|V|+1}$ ให้ $s \in A$ ที่ $|s| \geq p$ และเลือกต้นไม้แจงส่วน τ ของ s ที่มีจำนวนโหนดน้อยที่สุด เพราะ $|s| \geq b^{|V|+1} > b^{|V|}$ ต้นไม้ τ ต้องสูงอย่างน้อย $|V|+1$ ดังนั้นเส้นทางที่ยาวที่สุดจากรากถึงใบมีโหนดอย่างน้อย $|V|+2$ โหนด ซึ่งเป็นตัวแปรอย่างน้อย $|V|+1$ โหนด (ยกเว้นใบ) ตามหลักรังนกพิราบ (pigeonhole principle) ต้องมีตัวแปร R ปรากฏซ้ำอย่างน้อยสองครั้งในเส้นทางนี้ เลือก R ให้เป็นตัวแปรที่ซ้ำกันภายใน $|V|+1$ ตัวแปรล่างสุดของเส้นทาง

ให้ต้นไม้ย่อยของ R ตัวบนสร้างสตริง vxy และต้นไม้ย่อยของ R ตัวล่างสร้าง x ดังรูปที่ 1.7 ส่วนที่เหลือของ s ทางซ้ายและขวาคือ u และ z

- **เงื่อนไข (1)**: เนื่องจากทั้งสองโหนดเป็นตัวแปร R เหมือนกัน เราสามารถนำต้นไม้ย่อยตัวบนไปแทนที่ตัวล่างซ้ำ ๆ ได้ ได้ uv^ixy^iz สำหรับ $i \geq 2$ หรือนำต้นไม้ย่อยตัวล่างขึ้นมาแทนตัวบน ได้ uxz ซึ่งคือกรณี $i = 0$ (ดูรูปที่ 1.8)
- **เงื่อนไข (2)**: ถ้า v และ y เป็นสตริงว่างทั้งคู่ การแทนตัวบนด้วยตัวล่างจะได้ต้นไม้ของ s ที่มีโหนดน้อยกว่า τ ซึ่งขัดกับการเลือก τ ให้เล็กที่สุด
- **เงื่อนไข (3)**: เพราะเลือก R จากส่วนล่างของเส้นทาง ต้นไม้ย่อยของ R ตัวบนสูงไม่เกิน $|V|+1$ จึงสร้างสตริงยาวไม่เกิน $b^{|V|+1} = p$ นั่นคือ $|vxy| \leq p$ □

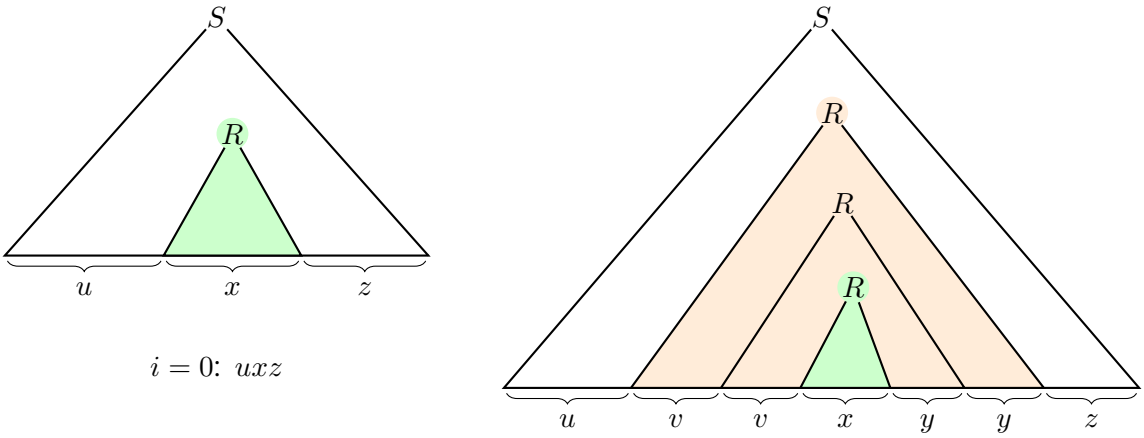


รูปที่ 1.7: ต้นไม้แจงส่วนของ s ที่มีตัวแปร R ซ้ำบนเส้นทางยาวที่สุด (เส้นประ) และการแบ่ง $s = uvxyz$

1.4.2 การใช้บทตั้งการปั๊ม

รูปแบบการใช้เหมือนกับบทตั้งการปั๊มของภาษาปรกติ คือพิสูจน์โดยข้อขัดแย้ง

1. สมมุติว่า A เป็นภาษาไม่พึ่งบริบท ให้ p เป็นความยาวปั๊มจากบทตั้ง
2. เลือกสตริง $s \in A$ ที่ $|s| \geq p$ อย่างชาญฉลาด (ปกติ s จะขึ้นกับ p)
3. แสดงว่า*ทุก*วิธีแบ่ง $s = uvxyz$ ที่สอดคล้องกับเงื่อนไข (2) และ (3) มี i บางค่าที่ทำให้ $uv^ixy^iz \notin A$
4. สรุปว่าเกิดข้อขัดแย้ง ดังนั้น A ไม่ใช่ภาษาไม่พึ่งบริบท



รูปที่ 1.8: การปั๊ม: ซ้าย นำต้นไม้ย่อยตัวล่างขึ้นแทนตัวบน ($i = 0$); ขวา นำต้นไม้ย่อยตัวบนลงไปแทนตัวล่าง ($i = 2$)

ขั้นที่ 3 มักแบ่งเป็นกรณีตามตำแหน่งที่ v และ y ตกอยู่ เงื่อนไข $|vxy| \le p$ มักเป็นตัวช่วยสำคัญในการจำกัดจำนวนกรณี

ตัวอย่าง 1.17. ภาษา $B = \{\, a^n b^n c^n : n \ge 0 \,\}$ ไม่ใช่ภาษาไม่พึ่งบริบท

พิสูจน์. สมมุติว่า B เป็นภาษาไม่พึ่งบริบท ให้ p เป็นความยาวปั๊ม เลือก $s = a^p b^p c^p \in B$ ให้ $s = uvxyz$ เป็นการแบ่งใด ๆ ที่ $|vy| > 0$ พิจารณาสองกรณี

- v **หรือ** y **มีตัวอักษรมากกว่าหนึ่งชนิด** เช่น $v = a^j b^k$ ที่ $j, k > 0$ เมื่อปั๊มขึ้น ($i = 2$) สตริง uv^2xy^2z จะมี b ปรากฏก่อน a บางตัว ซึ่งผิดรูปแบบ $a^*b^*c^*$ จึงไม่อยู่ใน B
- **ทั้ง** v **และ** y **มีตัวอักษรเพียงชนิดเดียว (หรือว่าง)** ดังนั้นตัวอักษรใน vy มีได้อย่างมากสองชนิดจากสามชนิด เมื่อปั๊มขึ้น จำนวนของชนิดที่ปรากฏใน vy (ซึ่งมีอย่างน้อยหนึ่งชนิดเพราะ $|vy| > 0$) จะเพิ่มขึ้น แต่ชนิดที่ไม่ปรากฏยังคงมี p ตัว จำนวนทั้งสามจึงไม่เท่ากัน $uv^2xy^2z \notin B$

ทุกกรณีขัดแย้งกับเงื่อนไข (1) ดังนั้น B ไม่ใช่ภาษาไม่พึ่งบริบท □

ตัวอย่าง 1.18. ภาษา $C = \{\, a^i b^j c^k : 0 \le i < j < k \,\}$ ไม่ใช่ภาษาไม่พึ่งบริบท

พิสูจน์. สมมุติว่า C เป็นภาษาไม่พึ่งบริบท ให้ p เป็นความยาวปั๊ม เลือก

$$s = a^p\, b^{p+1}\, c^{p+2} \in C$$

ให้ $s = uvxyz$ เป็นการแบ่งใด ๆ ที่ $|vy| > 0$ และ $|vxy| \le p$ ถ้า v หรือ y มีตัวอักษรต่างชนิดปนกัน การปั๊มขึ้นทำให้ลำดับตัวอักษรเสีย จึงไม่อยู่ใน C เหมือนตัวอย่างก่อน สมมุติต่อไปว่า v และ y แต่ละตัวมีตัวอักษรชนิดเดียว เนื่องจากบล็อก b ยาว $p + 1 > p \ge |vxy|$ สตริง vxy จึงไม่สามารถมีทั้ง a และ c พร้อมกัน

- vy **ไม่มี** c ดังนั้น vy มี a หรือ b อย่างน้อยหนึ่งตัว ปั๊มขึ้นด้วย $i = 2$
  - ถ้า vy มี b จำนวน b จะกลายเป็นอย่างน้อย $p + 2$ ซึ่งไม่น้อยกว่าจำนวน c ที่ยังคงเป็น $p + 2$ จึงผิดเงื่อนไข $j < k$

  - ถ้า vy มีแต่ a จำนวน a จะกลายเป็นอย่างน้อย $p+1$ ซึ่งไม่น้อยกว่าจำนวน b ที่ยังคงเป็น $p+1$ จึงผิดเงื่อนไข $i < j$

- vy **มี** c ดังนั้น vy ไม่มี a ปั๊มลงด้วย $i = 0$

  - ถ้า vy มี b ด้วย จำนวน b จะเหลือไม่เกิน p ซึ่งไม่มากกว่าจำนวน a ที่ยังคงเป็น p จึงผิดเงื่อนไข $i < j$
  - ถ้า vy มีแต่ c จำนวน c จะเหลือไม่เกิน $p+1$ ซึ่งไม่มากกว่าจำนวน b ที่ยังคงเป็น $p+1$ จึงผิดเงื่อนไข $j < k$

ทุกกรณีมี i ที่ทำให้ $uv^ixy^iz \notin C$ เกิดข้อขัดแย้ง ดังนั้น C ไม่ใช่ภาษาไม่พึ่งบริบท □

สังเกตว่าในตัวอย่างนี้ บางกรณีต้อง*ปั๊มขึ้น* และบางกรณีต้อง*ปั๊มลง* การเลือก i ให้เหมาะกับแต่ละกรณีเป็นทักษะสำคัญ

ตัวอย่าง 1.19. ภาษา $D = \{\, a^{n^2} : n \geq 0 \,\}$ ซึ่งประกอบด้วยสตริงของ a ที่มีความยาวเป็นกำลังสองสมบูรณ์ ไม่ใช่ภาษาไม่พึ่งบริบท

พิสูจน์. สมมุติว่า D เป็นภาษาไม่พึ่งบริบท ให้ p เป็นความยาวปั๊ม เลือก $s = a^{p^2} \in D$ ซึ่งยาว $p^2 \geq p$ ให้ $s = uvxyz$ เป็นการแบ่งใด ๆ ที่ $1 \leq |vy|$ และ $|vxy| \leq p$ จะได้ $1 \leq |vy| \leq p$ เมื่อปั๊มขึ้นด้วย $i = 2$ ความยาวของ uv^2xy^2z คือ $p^2 + |vy|$ ซึ่งสอดคล้องกับ

$$p^2 \;<\; p^2 + |vy| \;\leq\; p^2 + p \;<\; p^2 + 2p + 1 = (p+1)^2$$

ความยาวนี้อยู่ระหว่างกำลังสองสมบูรณ์สองตัวที่ติดกันอย่างแท้จริง จึงไม่ใช่กำลังสองสมบูรณ์ ดังนั้น $uv^2xy^2z \notin D$ เกิดข้อขัดแย้ง □

หมายเหตุ. ควรระวังว่าบทตั้งการปั๊มเป็นเพียง*เงื่อนไขจำเป็น* ภาษาที่ผ่านเงื่อนไขของบทตั้งไม่จำเป็นต้องเป็นภาษาไม่พึ่งบริบท บทตั้งจึงใช้ได้เฉพาะเพื่อแสดงว่าภาษา*ไม่ใช่* CFL เท่านั้น

1.4.3 สมบัติปิดของภาษาไม่พึ่งบริบท

จากการสร้างไวยากรณ์โดยตรง จะเห็นว่าภาษาไม่พึ่งบริบท**ปิดภายใต้** (closed under) การดำเนินการต่อไปนี้ ถ้า G_A และ G_B มีตัวแปรเริ่มต้น S_A และ S_B (ตั้งชื่อตัวแปรไม่ให้ซ้ำกัน) และ S เป็นตัวแปรเริ่มต้นใหม่

- **ยูเนียน** (union): เพิ่มกฎ $S \to S_A \mid S_B$
- **การต่อ** (concatenation): เพิ่มกฎ $S \to S_A S_B$
- **ดาวคลีน** (Kleene star): เพิ่มกฎ $S \to SS_A \mid \varepsilon$

แต่ภาษาไม่พึ่งบริบท*ไม่*ปิดภายใต้อินเตอร์เซกชัน (intersection) และคอมพลีเมนต์ (complement)

ทฤษฎีบท 1.20. *มีภาษาไม่พึ่งบริบท L และ L' ที่ $L \cap L'$ ไม่ใช่ภาษาไม่พึ่งบริบท และมีภาษาไม่พึ่งบริบทที่คอมพลีเมนต์ไม่ใช่ภาษาไม่พึ่งบริบท*

พิสูจน์. ให้

$$L = \{\, a^n b^n c^m : n, m \geq 0 \,\}, \qquad L' = \{\, a^m b^n c^n : n, m \geq 0 \,\}$$

ทั้งสองเป็นภาษาไม่พึ่งบริบท เช่น L สร้างได้ด้วย $S \to XY$, $X \to aXb \mid \varepsilon$, $Y \to cY \mid \varepsilon$ และ L' สร้างได้ในทำนองเดียวกัน แต่ $L \cap L' = \{a^n b^n c^n : n \geq 0\}$ ซึ่งไม่ใช่ CFL ตามตัวอย่างที่ 1.17

สำหรับคอมพลีเมนต์ ถ้า CFL ปิดภายใต้คอมพลีเมนต์ จากกฎของเดอมอร์แกน

$$L \cap L' = \overline{\overline{L} \cup \overline{L'}}$$

และการที่ CFL ปิดภายใต้ยูเนียน จะได้ว่า $L \cap L'$ เป็น CFL ซึ่งขัดแย้งกับข้างต้น □

หมายเหตุ. แม้จะไม่ปิดภายใต้อินเตอร์เซกชันทั่วไป แต่อินเตอร์เซกชันของภาษาไม่พึ่งบริบทกับ*ภาษาปรกติ*เป็นภาษาไม่พึ่งบริบทเสมอ (จำลอง PDA และ DFA ควบคู่กันโดยใช้สถานะแบบคู่อันดับ ส่วนกองซ้อนใช้ของ PDA ตามเดิม) นอกจากนี้ ภาษาไม่พึ่งบริบทเชิงกำหนดปิดภายใต้คอมพลีเมนต์ ซึ่งเป็นอีกเหตุผลหนึ่งที่ทำให้ทราบว่า DPDA อ่อนกว่า PDA ทั่วไป

สรุปท้ายบท

สรุปสาระสำคัญของบทที่ 1

- **รูปแบบปกติของชอมสกี**: ทุกกฎอยู่ในรูป $A \to BC$ หรือ $A \to a$ (และ $S \to \varepsilon$ ได้) ทุก CFL มีไวยากรณ์ในรูปนี้ ได้จากการเพิ่มตัวแปรเริ่มต้นใหม่ กำจัดกฎว่าง กำจัดกฎหน่วย และแตกกฎยาว สตริงยาว $n \geq 1$ ใช้การสืบทอด $2n - 1$ ขั้นพอดี
- **ออโตมาตาแบบกดลง** $(Q, \Sigma, \Gamma, \delta, q_0, F)$: ออโตมาตาจำกัดไม่เชิงกำหนดที่มีกองซ้อน ป้าย $a, b \to c$ หมายถึง อ่าน a ดึง b กด c ใช้ \$ เป็นเครื่องหมายก้นกองซ้อน
- PDA ไม่เชิงกำหนดมีพลังมากกว่า DPDA อย่างแท้จริง (เช่น $\{ww^{\mathcal{R}}\}$)
- **ความสมมูล**: ภาษาเป็น CFL ก็ต่อเมื่อมี PDA รู้จำ
  - CFG $\to$ PDA: PDA สามสถานะจำลองการสืบทอดซ้ายสุดบนกองซ้อน
  - PDA $\to$ CFG: ตัวแปร A_{pq} สร้างสตริงที่พา p ไป q โดยกองซ้อนว่างทั้งต้นและปลาย

  ภาษาปรกติทุกภาษาเป็น CFL
- **บทตั้งการปั๊ม**: สตริงยาวใน CFL แบ่งเป็น $uvxyz$ ที่ $|vy| > 0$, $|vxy| \leq p$ และ $uv^i xy^i z$ อยู่ในภาษาสำหรับทุก $i \geq 0$ ใช้แสดงว่า $\{a^n b^n c^n\}$, $\{a^i b^j c^k : i < j < k\}$, $\{a^{n^2}\}$ ไม่ใช่ CFL
- **สมบัติปิด**: CFL ปิดภายใต้ยูเนียน การต่อ และดาวคลีน แต่ไม่ปิดภายใต้อินเตอร์เซกชันและคอมพลีเมนต์ (อินเตอร์เซกชันกับภาษาปรกติยังเป็น CFL)

บทที่ 2

เครื่องจักรทัวริง และวิทยานิพนธ์เชิร์ช–ทัวริง

ในบทก่อน ๆ เราได้รู้จักแบบจำลองการคำนวณที่มีหน่วยความจำจำกัด เช่น ออโตมาตาจำกัด (finite automaton) ซึ่งจำได้เพียงสถานะปัจจุบัน และออโตมาตากดลง (pushdown automaton) ซึ่งมีหน่วยความจำแบบกองซ้อน (stack) ที่เข้าถึงได้เฉพาะด้านบนสุด แบบจำลองทั้งสองมีข้อจำกัดชัดเจน เช่น ไม่มีออโตมาตากดลงใดรู้จำภาษา $\{0^n1^n2^n \mid n \geq 0\}$ ได้ บทนี้จะแนะนำแบบจำลองที่ทรงพลังกว่ามาก คือ **เครื่องจักรทัวริง** (Turing machine) ซึ่งเสนอโดย Alan Turing ในปี ค.ศ. 1936 แม้จะมีโครงสร้างเรียบง่าย แต่เครื่องจักรทัวริงสามารถทำทุกอย่างที่คอมพิวเตอร์จริงทำได้ และเป็นรากฐานของคำถามว่า “อะไรคำนวณได้และอะไรคำนวณไม่ได้” ซึ่งเป็นหัวใจของบทต่อ ๆ ไป

2.1 เครื่องจักรทัวริง

2.1.1 ภาพรวมเชิงสัญชาตญาณ

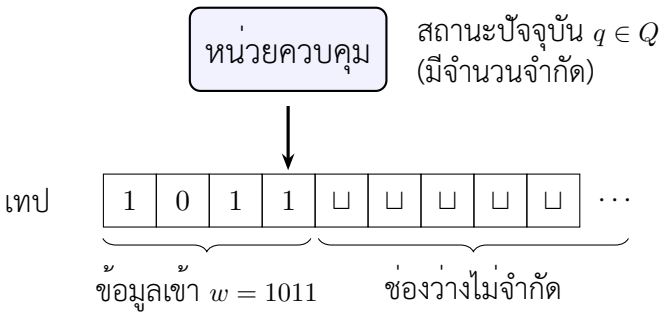
ลองนึกภาพคนที่กำลังคิดเลขด้วยกระดาษทดยาวไม่จำกัด เขามองเห็นได้ทีละช่อง เขียนหรือลบสัญลักษณ์ในช่องนั้นได้ แล้วเลื่อนสายตาไปช่องข้างเคียง ส่วน “สภาวะจิตใจ” ของเขามีได้เพียงจำนวนจำกัด เครื่องจักรทัวริงคือการจับภาพกระบวนการนี้ให้เป็นคณิตศาสตร์ โดยมีองค์ประกอบดังนี้

- **เทป** (tape) ยาวไม่จำกัดไปทางขวา แบ่งเป็นช่อง (cell) ช่องละหนึ่งสัญลักษณ์
- **หัวอ่าน/เขียน** (read/write head) ชี้ที่ช่องหนึ่งช่องในแต่ละขณะ อ่านสัญลักษณ์ เขียนทับ และเลื่อนซ้ายหรือขวาได้ทีละหนึ่งช่อง
- **หน่วยควบคุม** (control) ซึ่งอยู่ในสถานะใดสถานะหนึ่งจากเซตจำกัดของสถานะ

ในตอนเริ่มต้น สตริงข้อมูลเข้า (input) จะถูกวางไว้ที่ช่องซ้ายสุดของเทป ช่องที่เหลือทั้งหมดเป็น **ช่องว่าง** (blank) แทนด้วย $\sqcup$ และหัวอ่านอยู่ที่ช่องซ้ายสุด

ความแตกต่างสำคัญระหว่างเครื่องจักรทัวริงกับออโตมาตาจำกัดมีสี่ประการ

1. เครื่องจักรทัวริง*เขียน*ลงเทปได้ ไม่ได้อ่านอย่างเดียว
2. หัวอ่านเลื่อนได้ทั้งซ้ายและขวา จึงย้อนกลับไปอ่านข้อมูลซ้ำกี่ครั้งก็ได้
3. เทปยาวไม่จำกัด จึงมีที่ทดเหลือเฟือ
4. สถานะยอมรับ (accept) และปฏิเสธ (reject) มีผล*ทันที* เมื่อเครื่องเข้าสถานะใดสถานะหนึ่งในสองนี้ เครื่องจะหยุด (halt) ไม่จำเป็นต้องอ่านข้อมูลเข้าให้หมดก่อน



รูปที่ 2.1: เครื่องจักรทัวริงประกอบด้วยหน่วยควบคุมที่มีสถานะจำกัด หัวอ่าน/เขียน และเทปยาวไม่จำกัดทางขวา (ในรูปหัวอ่านอยู่ที่ช่องที่สี่)

ข้อที่สี่นำไปสู่ความเป็นไปได้ใหม่ที่ไม่เคยมีในออโตมาตาจำกัด นั่นคือเครื่องอาจ*ไม่หยุดเลย* หรือที่เรียกว่า **วนไม่รู้จบ** (loop)

2.1.2 นิยามเชิงรูปนัย

นิยาม 2.1 (เครื่องจักรทัวริง)**. เครื่องจักรทัวริง** คือทูเพิลเจ็ดสิ่ง

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$$

โดยที่ Q, Σ, Γ เป็นเซตจำกัดทั้งหมด และ

1. Q คือเซตของสถานะ (states)
2. Σ คือ**อักษรข้อมูลเข้า** (input alphabet) ซึ่งไม่มีสัญลักษณ์ช่องว่าง $\sqcup$
3. Γ คือ**อักษรเทป** (tape alphabet) โดย $\sqcup \in \Gamma$ และ $\Sigma \subseteq \Gamma$
4. $\delta : Q \times \Gamma \to Q \times \Gamma \times \{\mathrm{L}, \mathrm{R}\}$ คือ**ฟังก์ชันการเปลี่ยนสถานะ** (transition function)
5. $q_0 \in Q$ คือสถานะเริ่มต้น
6. $q_{\text{accept}} \in Q$ คือสถานะยอมรับ
7. $q_{\text{reject}} \in Q$ คือสถานะปฏิเสธ โดย $q_{\text{reject}} \neq q_{\text{accept}}$

ความหมายของ $\delta(q, a) = (r, b, \mathrm{R})$ คือ “ถ้าอยู่ในสถานะ q และหัวอ่านอ่านได้ a ให้เขียน b ทับ a เปลี่ยนไปสถานะ r แล้วเลื่อนหัวอ่านไปทางขวาหนึ่งช่อง” ส่วน L หมายถึงเลื่อนไปทางซ้าย หากหัวอ่านอยู่ที่ช่องซ้ายสุดแล้วถูกสั่งให้เลื่อนซ้าย หัวอ่านจะอยู่ที่เดิม (เทปไม่มีส่วนที่ยื่นออกไปทางซ้าย)

หมายเหตุ. การที่ $\sqcup \notin \Sigma$ ทำให้เครื่องรู้ได้ว่าข้อมูลเข้าสิ้นสุดตรงไหน คือที่ช่องว่างช่องแรก นอกจากนี้ Γ มักมีสัญลักษณ์เพิ่มเติม (เช่น เครื่องหมายกากบาท x) ไว้ใช้ทำเครื่องหมายบนเทประหว่างการคำนวณ

2.1.3 โครงแบบและความสัมพันธ์ “ให้ผลเป็น”

เพื่ออธิบายการทำงานทีละขั้นอย่างแม่นยำ เราใช้แนวคิดของ **โครงแบบ** (configuration) ซึ่งเป็นภาพถ่าย ณ ขณะใดขณะหนึ่ง ประกอบด้วยสามสิ่ง: สถานะปัจจุบัน เนื้อหาของเทป และตำแหน่งหัวอ่าน

นิยาม 2.2 (สัญกรณ์โครงแบบ)**.** ให้ $q \in Q$ และ $u, v \in \Gamma^*$ เราเขียนโครงแบบเป็นสตริง $u\,q\,v$ ซึ่งหมายความว่า สถานะปัจจุบันคือ q เนื้อหาของเทปคือ uv (ตามด้วยช่องว่างไม่จำกัด) และหัวอ่านอยู่ที่สัญลักษณ์ตัวแรกของ v (ถ้า v เป็นสตริงว่าง หัวอ่านอยู่ที่ช่องว่างถัดจาก u)

ตัวอย่างเช่น โครงแบบ $10\,q_3\,11$ แทนสถานการณ์ที่เทปมี 1011 เครื่องอยู่ในสถานะ q_3 และหัวอ่านชี้ที่เลข 1 ตัวที่สาม ซึ่งตรงกับรูปที่ 2.1 ถ้าหน่วยควบคุมอยู่ในสถานะ q_3

นิยาม 2.3 (ให้ผลเป็น)**.** ให้ $a, b, c \in \Gamma$, $u, v \in \Gamma^*$ และ $q_i, q_j \in Q$ เรากล่าวว่าโครงแบบ C_1 **ให้ผลเป็น** (yields) โครงแบบ C_2 เขียนว่า $C_1 \vdash C_2$ ถ้าเครื่องเปลี่ยนจาก C_1 เป็น C_2 ได้ในหนึ่งก้าว อย่างเป็นทางการคือ

- ถ้า $\delta(q_i, b) = (q_j, c, \mathsf{L})$ แล้ว $u\,a\,q_i\,b\,v \vdash u\,q_j\,a\,c\,v$
- ถ้า $\delta(q_i, b) = (q_j, c, \mathsf{R})$ แล้ว $u\,a\,q_i\,b\,v \vdash u\,a\,c\,q_j\,v$

กรณีพิเศษที่ขอบซ้าย: ถ้า $\delta(q_i, b) = (q_j, c, \mathsf{L})$ แล้ว $q_i\,b\,v \vdash q_j\,c\,v$ (หัวอ่านไม่ขยับ) และถ้าหัวอ่านอยู่ท้ายสุด ($u\,a\,q_i$) ให้ถือว่าเหมือน $u\,a\,q_i\,\sqcup$

โครงแบบชนิดพิเศษที่ควรรู้จักมีดังนี้

- **โครงแบบเริ่มต้น** (start configuration) บนข้อมูลเข้า w คือ $q_0\,w$
- **โครงแบบยอมรับ** (accepting configuration) คือโครงแบบที่มีสถานะ q_{accept}
- **โครงแบบปฏิเสธ** (rejecting configuration) คือโครงแบบที่มีสถานะ q_{reject}
- โครงแบบยอมรับและโครงแบบปฏิเสธรวมกันเรียกว่า **โครงแบบหยุด** (halting configuration) ซึ่งไม่ให้ผลเป็นโครงแบบใดต่อ

นิยาม 2.4 (การยอมรับ)**.** เครื่องจักรทัวริง M **ยอมรับ** ข้อมูลเข้า w ถ้ามีลำดับของโครงแบบ $C_1, C_2, \ldots, C_k$ ซึ่ง C_1 เป็นโครงแบบเริ่มต้นของ M บน w, $C_i \vdash C_{i+1}$ สำหรับทุก $i < k$ และ C_k เป็นโครงแบบยอมรับ เซตของสตริงทั้งหมดที่ M ยอมรับเรียกว่า **ภาษาของ** M เขียนว่า $L(M)$ และกล่าวว่า M **รู้จำ** (recognizes) ภาษานี้

เมื่อรันเครื่อง M บนข้อมูลเข้า w จะมีผลลัพธ์ได้สามแบบเท่านั้น

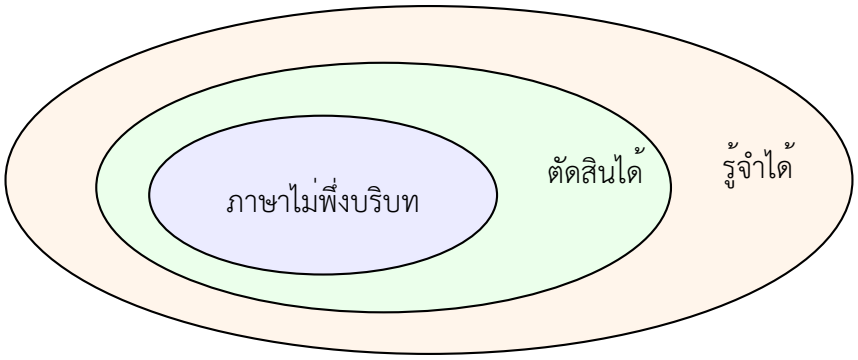
ผลลัพธ์	ความหมาย
ยอมรับ (accept)	ไปถึง q_{accept} ในจำนวนก้าวจำกัด
ปฏิเสธ (reject)	ไปถึง q_{reject} ในจำนวนก้าวจำกัด
วนไม่รู้จบ (loop)	ไม่เคยไปถึงสถานะหยุดเลย

สังเกตว่า “วนไม่รู้จบ” ไม่จำเป็นต้องเป็นการวนซ้ำแบบตายตัว เครื่องอาจทำงานซับซ้อนไปเรื่อย ๆ โดยไม่ซ้ำเดิมก็ได้ ปัญหาในทางปฏิบัติคือ ผู้สังเกตที่เฝ้าดูเครื่องทำงานอยู่ไม่มีทางแยกได้ว่าเครื่อง “กำลังจะยอมรับอีกไม่นาน” หรือ “จะไม่มีวันหยุด” นี่คือเหตุผลที่เราต้องแยกนิยามสองแบบต่อไปนี้

นิยาม 2.5. 1. ภาษา L เป็น**ภาษาที่ทัวริงรู้จำได้** (Turing-recognizable) ถ้ามีเครื่องจักรทัวริง M ที่ $L(M) = L$ (ภาษาประเภทนี้บางตำราเรียกว่า ภาษาที่แจกแจงได้แบบเวียนเกิด หรือ recursively enumerable)

2. เครื่องจักรทัวริงที่หยุดเสมอบนทุกข้อมูลเข้าเรียกว่า **ตัวตัดสิน** (decider)
3. *ภาษา* L เป็น**ภาษาที่ทัวริงตัดสินได้** (Turing-decidable) หรือเรียกสั้น ๆ ว่า **ตัดสินได้** (decidable) ถ้ามีตัวตัดสิน M ที่ $L(M) = L$ ในกรณีนี้กล่าวว่า M **ตัดสิน** (decides) L

ความแตกต่างคือ เครื่องที่เพียง *รู้จำ* L ต้องยอมรับทุกสตริงใน L แต่สำหรับสตริงที่ไม่อยู่ใน L เครื่องจะปฏิเสธหรือวนไม่รู้จบก็ได้ ส่วนเครื่องที่ *ตัดสิน* L ต้องให้คำตอบที่ชัดเจนเสมอ ทุกภาษาที่ตัดสินได้จึงรู้จำได้ แต่ในบทต่อไปเราจะเห็นว่าบทกลับไม่จริง



(ภาษาที่ทัวริงรู้จำได้ทั้งหมดอยู่ในวงรีนอกสุด)

รูปที่ 2.2: ความสัมพันธ์ระหว่างคลาสของภาษา: ภาษาไม่พึ่งบริบททุกภาษาตัดสินได้ และภาษาที่ตัดสินได้ทุกภาษารู้จำได้ (การบรรจุทั้งสองเป็นการบรรจุแท้ ดังจะแสดงในบทต่อไป)

2.1.4 ตัวอย่างระดับสูง: ภาษา $\{0^n1^n2^n\}$

ก่อนจะลงรายละเอียดของฟังก์ชันการเปลี่ยนสถานะ เราลองอธิบายเครื่องด้วยภาษาธรรมชาติก่อน ซึ่งเป็นวิธีที่นักวิทยาการคอมพิวเตอร์ใช้กันโดยทั่วไป

ตัวอย่าง 2.6. ให้ $T = \{0^n1^n2^n \mid n \geq 0\}$ ภาษานี้ไม่ใช่ภาษาไม่พึ่งบริบท แต่เครื่องจักรทัวริงตัดสินได้ไม่ยาก แนวคิดคือ “ขีดฆ่า” สัญลักษณ์ 0, 1, 2 ทีละหนึ่งตัวต่อรอบ จนกว่าตัวใดตัวหนึ่งหมดก่อนตัวอื่น

$M_T =$ “บนข้อมูลเข้า w:

1. กวาดจากซ้ายไปขวาหนึ่งรอบ ถ้า w ไม่อยู่ในรูป $0^*1^*2^*$ ให้*ปฏิเสธ*
2. เลื่อนหัวอ่านกลับไปที่ช่องซ้ายสุด
3. ถ้าบนเทปไม่เหลือ 0, 1, 2 ที่ยังไม่ถูกขีดฆ่าเลย ให้*ยอมรับ*
4. หา 0 ตัวแรกที่ยังไม่ถูกขีดฆ่าแล้วขีดฆ่า ถ้าหาไม่พบให้*ปฏิเสธ*
5. เลื่อนไปทางขวาเพื่อหา 1 ตัวแรกที่ยังไม่ถูกขีดฆ่าแล้วขีดฆ่า ถ้าหาไม่พบให้*ปฏิเสธ*
6. เลื่อนไปทางขวาต่อเพื่อหา 2 ตัวแรกที่ยังไม่ถูกขีดฆ่าแล้วขีดฆ่า ถ้าหาไม่พบให้*ปฏิเสธ*
7. กลับไปทำขั้นที่ 2”

ทุกรอบของขั้นที่ 2–7 ขีดฆ่าสัญลักษณ์อย่างน้อยหนึ่งตัว และมีสัญลักษณ์จำกัด เครื่องจึงหยุดเสมอ เครื่องยอมรับเมื่อทั้งสามชนิดหมดพร้อมกันเท่านั้น ซึ่งเกิดขึ้นก็ต่อเมื่อจำนวนของแต่ละชนิดเท่ากัน ดังนั้น M_T เป็นตัวตัดสินของ T และ T เป็นภาษาที่ตัดสินได้

ภาพเทปหลังแต่ละรอบบนข้อมูลเข้า 001122 (ใช้ x แทนการขีดฆ่า) เป็นดังนี้

$$001122 \longrightarrow x0x1x2 \longrightarrow xxxxxx \longrightarrow \text{ยอมรับ}$$

ส่วนบนข้อมูลเข้า 00112 หลังรอบแรกเทปเป็น $x0x1x$ และในรอบที่สองเครื่องขีดฆ่า 0 และ 1 ได้ แต่หา 2 ไม่พบ จึงปฏิเสธ

2.1.5 ตัวอย่างละเอียด: ภาษาพาลินโดรม

ต่อไปเราจะสร้างเครื่องจักรทัวริงอย่างครบถ้วนพร้อมแผนภาพสถานะ สำหรับภาษา

$$PAL = \{w \in \{a,b\}^* \mid w = w^{\mathcal{R}}\}$$

คือเซตของ**พาลินโดรม** (palindrome) สตริงที่อ่านจากซ้ายไปขวาหรือขวาไปซ้ายได้ผลเหมือนกัน เช่น ε, a, $abba$, $babab$

แนวคิด เครื่องจะจับคู่สัญลักษณ์ซ้ายสุดที่ยังไม่ถูกขีดฆ่ากับสัญลักษณ์ขวาสุดที่ยังไม่ถูกขีดฆ่า โดยจำสัญลักษณ์ซ้ายสุดไว้ใน*สถานะ* (ใช้สถานะ q_a หรือ q_b) แล้ววิ่งไปทางขวาจนสุด ถอยหนึ่งช่องแล้วตรวจว่าตรงกันหรือไม่ ถ้าตรงกันก็ขีดฆ่าแล้ววิ่งกลับไปทางซ้าย ทำซ้ำจนกว่าจะไม่เหลือสัญลักษณ์ที่ยังไม่ถูกขีดฆ่า

นิยามเชิงรูปนัย ให้ $M_{PAL} = (Q, \Sigma, \Gamma, \delta, q_1, q_{\text{accept}}, q_{\text{reject}})$ โดย

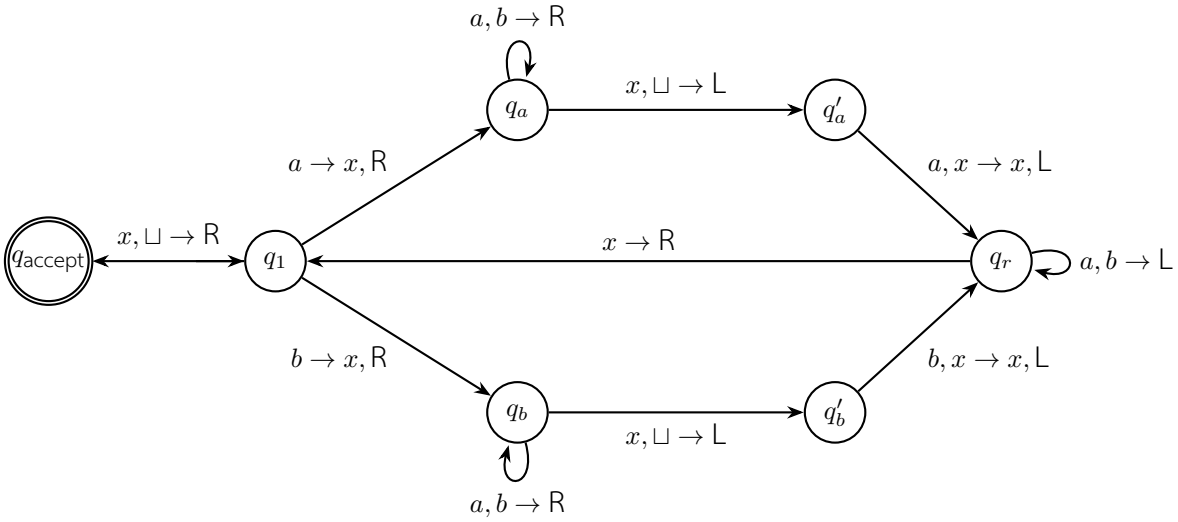
$$Q = \{q_1, q_a, q'_a, q_b, q'_b, q_r, q_{\text{accept}}, q_{\text{reject}}\}, \qquad \Sigma = \{a,b\}, \qquad \Gamma = \{a, b, x, \sqcup\}$$

ความหมายของแต่ละสถานะคือ

- q_1: หัวอ่านอยู่ที่สัญลักษณ์ซ้ายสุดที่ยังไม่ถูกขีดฆ่า (ถ้ามี)
- q_a / q_b: เพิ่งขีดฆ่า a / b ทางซ้าย กำลังวิ่งไปทางขวาหาขอบ
- q'_a / q'_b: อยู่ที่สัญลักษณ์ขวาสุดที่ยังไม่ถูกขีดฆ่า และต้องการพบ a / b
- q_r: วิ่งกลับไปทางซ้ายเพื่อเริ่มรอบใหม่

ฟังก์ชัน δ แสดงในรูปที่ 2.3 โดยใช้ข้อตกลงดังนี้: ลูกศรที่มีป้าย $a \to x, \mathsf{R}$ หมายถึงอ่าน a เขียน x แล้วเลื่อนขวา ป้าย $a, b \to \mathsf{R}$ หมายถึงอ่าน a หรือ b แล้วเขียนสัญลักษณ์เดิม (ไม่เปลี่ยนเทป) และเลื่อนขวา และเพื่อให้แผนภาพอ่านง่าย *การเปลี่ยนสถานะที่ไม่ได้วาดไว้ทั้งหมดจะไปยัง* q_{reject}

ข้อสังเกตเกี่ยวกับกรณีความยาวคี่ เมื่อ w มีความยาวคี่ สัญลักษณ์ตรงกลางจะถูกขีดฆ่าในสถานะ q_1 แล้วเครื่องเข้า q_a (หรือ q_b) ซึ่งจะพบ x ทันทีในช่องถัดไป จึงถอยกลับมาที่ช่องที่เพิ่งขีดฆ่า ในสถานะ q'_a เครื่องจะอ่านได้ x ซึ่งเราอนุญาตไว้ (ป้าย $a, x \to x, \mathsf{L}$) เพราะสัญลักษณ์ตรงกลางไม่ต้องจับคู่กับใคร จากนั้น q_r อ่าน x แล้วเลื่อนขวา ทำให้ q_1 อ่านได้ x และยอมรับ



รูปที่ 2.3: แผนภาพสถานะของ M_{PAL} ซึ่งตัดสินภาษาพาลินโดรมบน $\{a, b\}$ (การเปลี่ยนสถานะที่ไม่ได้วาดไว้ไปยัง q_{reject} ซึ่งไม่ได้แสดงในรูป)

การติดตามโครงแบบ บนข้อมูลเข้า $abba$ ลำดับโครงแบบของ M_{PAL} คือ

$$\begin{aligned} & q_1\, abba \;\vdash\; x\, q_a\, bba \;\vdash\; xb\, q_a\, ba \;\vdash\; xbb\, q_a\, a \;\vdash\; xbba\, q_a\, \sqcup \;\vdash\; xbb\, q'_a\, a \\ \vdash\; & xb\, q_r\, bx \;\vdash\; x\, q_r\, bbx \;\vdash\; q_r\, xbbx \;\vdash\; x\, q_1\, bbx \;\vdash\; xx\, q_b\, bx \;\vdash\; xxb\, q_b\, x \\ \vdash\; & xx\, q'_b\, bx \;\vdash\; x\, q_r\, xxx \;\vdash\; xx\, q_1\, xx \;\vdash\; xxx\, q_{\text{accept}}\, x \end{aligned}$$

เครื่องจึงยอมรับ $abba$ ส่วนบนข้อมูลเข้า ab จะได้

$$q_1\, ab \;\vdash\; x\, q_a\, b \;\vdash\; xb\, q_a\, \sqcup \;\vdash\; x\, q'_a\, b \;\vdash\; x\, q_{\text{reject}}\, x$$

เพราะ $\delta(q'_a, b)$ ไม่ได้วาดไว้จึงไปยัง q_{reject} (สัญลักษณ์ที่เขียนและทิศทางในก้าวนี้ไม่มีความสำคัญ)

เนื่องจากทุกรอบขีดฆ่าสัญลักษณ์อย่างน้อยหนึ่งตัว และแต่ละรอบใช้จำนวนก้าวจำกัด M_{PAL} จึงหยุดบนทุกข้อมูลเข้า ทำให้ *PAL* เป็นภาษาที่ตัดสินได้ (แม้ว่า *PAL* จะเป็นภาษาไม่พึ่งบริบทอยู่แล้ว ตัวอย่างนี้แสดงเทคนิค “จำไว้ในสถานะ” และ “ขีดฆ่าบนเทป” ที่ใช้กันบ่อยมาก)

2.2 รูปแบบต่าง ๆ ของเครื่องจักรทัวริง

มีการดัดแปลงนิยามของเครื่องจักรทัวริงได้หลายแบบ เช่น เพิ่มจำนวนเทป หรืออนุญาตให้มีการเลือกแบบไม่เชิงกำหนด ข้อเท็จจริงที่น่าทึ่งคือ รูปแบบที่ “สมเหตุสมผล” ทั้งหมดมีพลังเท่ากับแบบดั้งเดิม คือรู้จำคลาสของภาษาเดียวกัน คุณสมบัตินี้เรียกว่า **ความทนทาน** (robustness) ของแบบจำลอง ซึ่งเป็นหลักฐานสำคัญว่าเครื่องจักรทัวริงจับแนวคิดของ “การคำนวณ” ได้ถูกต้อง

การแสดงว่าสองแบบจำลองมีพลังเท่ากัน ทำได้โดยแสดงว่าแต่ละแบบ**จำลอง** (simulate) อีกแบบหนึ่งได้

2.2.1 เครื่องที่หัวอ่านอยู่กับที่ได้

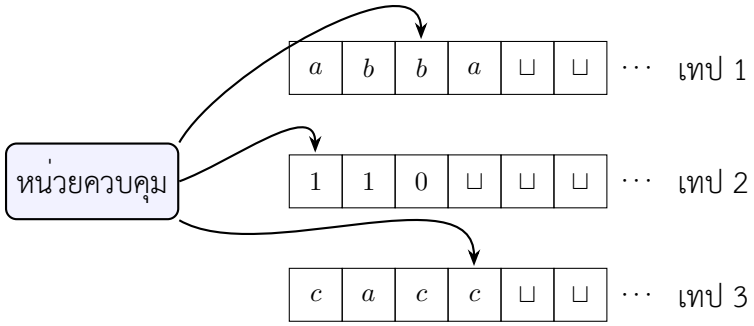
สมมติเราขยายฟังก์ชันการเปลี่ยนสถานะเป็น $\delta : Q \times \Gamma \to Q \times \Gamma \times \{\mathsf{L}, \mathsf{R}, \mathsf{S}\}$ โดย S (stay put) หมายถึงหัวอ่านไม่เลื่อน รูปแบบนี้ไม่ได้เพิ่มพลัง เพราะเราแทนการเปลี่ยนสถานะแบบ S ได้ด้วยสองก้าว: ก้าวแรกเขียนสัญลักษณ์และเลื่อนขวาไปยังสถานะช่วยคั่นกลาง ก้าวที่สองเลื่อนซ้ายกลับโดยไม่เปลี่ยนสัญลักษณ์ ในทางกลับกัน เครื่องแบบดั้งเดิมเป็นกรณีพิเศษของเครื่องแบบนี้อยู่แล้ว

2.2.2 เครื่องจักรทัวริงหลายเทป

เครื่องจักรทัวริงหลายเทป (multitape Turing machine) มีเทป k เทป แต่ละเทปมีหัวอ่าน/เขียนของตนเองที่เคลื่อนที่อย่างอิสระ ในตอนเริ่มต้น ข้อมูลเข้าอยู่บนเทปที่ 1 และเทปอื่นว่างทั้งหมด ฟังก์ชันการเปลี่ยนสถานะมีรูปแบบ

$$\delta : Q \times \Gamma^k \to Q \times \Gamma^k \times \{\mathsf{L}, \mathsf{R}, \mathsf{S}\}^k$$

เช่น $\delta(q, a_1, \ldots, a_k) = (r, b_1, \ldots, b_k, \mathsf{L}, \mathsf{R}, \ldots, \mathsf{S})$ หมายถึงเมื่ออยู่ในสถานะ q และหัวอ่านทั้ง k อ่านได้ $a_1, \ldots, a_k$ ให้ไปสถานะ r เขียน $b_1, \ldots, b_k$ แล้วเลื่อนหัวอ่านแต่ละหัวตามที่ระบุ



รูปที่ 2.4: เครื่องจักรทัวริงสามเทป หัวอ่านแต่ละหัวเคลื่อนที่อิสระจากกัน แต่ทุกหัวถูกควบคุมโดยหน่วยควบคุมเดียว

เครื่องหลายเทปสะดวกกว่าในการออกแบบมาก ตัวอย่างเช่น ภาษา $\{w\#w \mid w \in \{0,1\}^*\}$ ตัดสินได้ด้วยเครื่องสองเทปโดยคัดลอกส่วนก่อน # ไปไว้บนเทปที่ 2 แล้วเปรียบเทียบทีละตัว โดยไม่ต้องวิ่งไปกลับหลายรอบ แต่ความสะดวกนี้ไม่ได้เพิ่มพลังในการรู้จำ

ทฤษฎีบท 2.7. *สำหรับเครื่องจักรทัวริงหลายเทป M ใด ๆ จะมีเครื่องจักรทัวริงเทปเดียว S ที่ $L(S) = L(M)$ และถ้า M เป็นตัวตัดสินแล้ว S ก็เป็นตัวตัดสินด้วย*

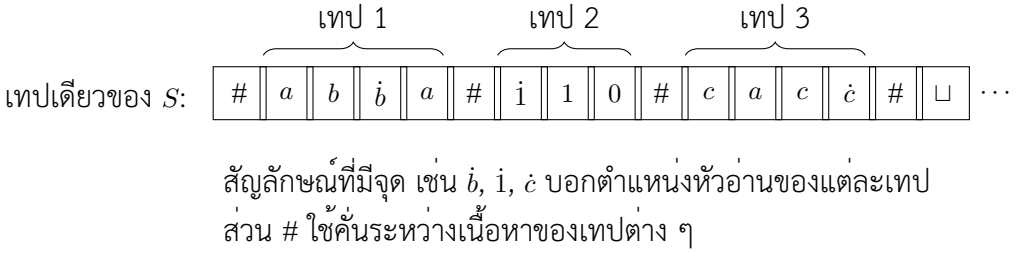
แนวคิดการพิสูจน์. ให้ M มี k เทป เราเก็บเนื้อหาของทั้ง k เทปไว้บนเทปเดียวของ S โดยคั่นด้วยสัญลักษณ์ใหม่ # และระบุตำแหน่งหัวอ่านของแต่ละเทปด้วยการ “ใส่จุด” บนสัญลักษณ์ที่หัวอ่านชี้อยู่ นั่นคือ สำหรับทุก $c \in \Gamma$ เราเพิ่มสัญลักษณ์ $\dot{c}$ เข้าไปในอักษรเทปของ S ดังรูปที่ 2.5 หัวอ่านเหล่านี้จึงเป็น *หัวอ่านเสมือน* (virtual heads)

บนข้อมูลเข้า $w = w_1 \cdots w_n$ เครื่อง S ทำดังนี้

1. จัดเทปให้อยู่ในรูป $\#\,\dot{w}_1 w_2 \cdots w_n \,\#\, \dot{\sqcup} \,\#\, \dot{\sqcup} \,\#\cdots\#$ ซึ่งแทนโครงแบบเริ่มต้นของ M

2. เพื่อจำลองหนึ่งก้าวของ M: กวาดจาก # ซ้ายสุดไปถึง # ตัวที่ $k+1$ เพื่ออ่านสัญลักษณ์ที่มีจุดทั้ง k ตัว (จำไว้ในสถานะ ซึ่งทำได้เพราะ k และ Γ จำกัด) จากนั้นกวาดรอบที่สองเพื่อเขียนสัญลักษณ์ใหม่และย้ายจุดตามที่ δ ของ M กำหนด
3. ถ้าหัวอ่านเสมือนเลื่อนไปทับ # ทางขวา แสดงว่าเทปนั้นต้องการพื้นที่เพิ่ม: S เขียน $\dot{\sqcup}$ ลงที่ช่องนั้น แล้วเลื่อนเนื้อหาทั้งหมดทางขวาไปหนึ่งช่องก่อนทำงานต่อ
4. ถ้า M เข้าสถานะยอมรับหรือปฏิเสธ S ก็ทำเช่นเดียวกัน

S ยอมรับ ปฏิเสธ หรือวนไม่รู้จบ ตรงกับ M ทุกประการ จึงได้ผลตามต้องการ □



รูปที่ 2.5: การเก็บเนื้อหาของสามเทปในรูปที่ 2.4 ไว้บนเทปเดียว (เทป 1 มี $abba$ หัวอ่านอยู่ที่ตัวที่สาม, เทป 2 มี 110 หัวอ่านอยู่ที่ตัวแรก, เทป 3 มี $cacc$ หัวอ่านอยู่ที่ตัวสุดท้าย)

บทแทรก 2.8. *ภาษาหนึ่งเป็นภาษาที่ทัวริงรู้จำได้ ก็ต่อเมื่อ มีเครื่องจักรทัวริงหลายเทปที่รู้จำภาษานั้น*

พิสูจน์. ทิศทางหนึ่งได้จากทฤษฎีบท 5.6 อีกทิศทางหนึ่งชัดเจน เพราะเครื่องเทปเดียวคือเครื่องหลายเทปที่ $k = 1$ □

2.2.3 เครื่องจักรทัวริงไม่เชิงกำหนด

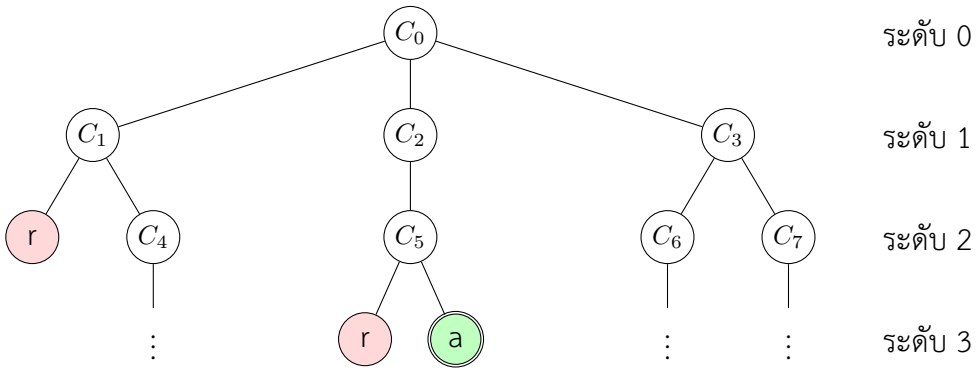
เช่นเดียวกับออโตมาตาจำกัด เราอาจยอมให้เครื่องจักรทัวริงมีทางเลือกหลายทางในแต่ละก้าว **เครื่องจักรทัวริงไม่เชิงกำหนด** (nondeterministic Turing machine) มีฟังก์ชันการเปลี่ยนสถานะ

$$\delta : Q \times \Gamma \to \mathcal{P}(Q \times \Gamma \times \{\mathrm{L}, \mathrm{R}\})$$

โดย $\mathcal{P}$ คือเซตกำลัง การคำนวณบนข้อมูลเข้า w จึงไม่ใช่ลำดับเส้นตรง แต่เป็น**ต้นไม้การคำนวณ** (computation tree) ซึ่งแต่ละโหนดเป็นโครงแบบ และลูกของโหนดคือโครงแบบทั้งหมดที่ได้ในหนึ่งก้าว เครื่องไม่เชิงกำหนด*ยอมรับ* w ถ้า*มีอย่างน้อยหนึ่ง*กิ่งของต้นไม้ที่ไปถึงโครงแบบยอมรับ

ทฤษฎีบท 2.9. *สำหรับเครื่องจักรทัวริงไม่เชิงกำหนด N ใด ๆ จะมีเครื่องจักรทัวริงเชิงกำหนด D ที่ $L(D) = L(N)$*

แนวคิดการพิสูจน์. D จะค้นหาโครงแบบยอมรับในต้นไม้การคำนวณของ N อย่างเป็นระบบ คำถามสำคัญคือ *ค้นหาลำดับใด* ถ้าใช้การค้นหาแนวลึก (depth-first search) D อาจหลงลงไปในกิ่งที่ยาวไม่สิ้นสุด (เช่นกิ่งใต้ C_4 ในรูปที่ 2.6) และไม่เคยไปพบกิ่งที่ยอมรับเลย เราจึงใช้การค้นหา**แนวกว้าง** (breadth-first search) คือสำรวจทุกโหนดในระดับ d ให้ครบก่อนไประดับ $d+1$ ซึ่งรับประกันว่าถ้ามีโครงแบบยอมรับที่ระดับใด D จะพบมันในเวลาจำกัด



a = โครงแบบยอมรับ, r = โครงแบบปฏิเสธ, ⋮ = กิ่งที่อาจยาวไม่สิ้นสุด

รูปที่ 2.6: ต้นไม้การคำนวณของเครื่องไม่เชิงกำหนด กิ่งผ่าน C_2, C_5 ไปถึงโครงแบบยอมรับ เครื่องจึงยอมรับข้อมูลเข้านี้ แม้จะมีกิ่งอื่นที่ปฏิเสธหรือไม่สิ้นสุด

ให้ b เป็นจำนวนทางเลือกมากที่สุดของ δ ในแต่ละก้าว ทุกโหนดในต้นไม้จึงระบุได้ด้วย**ที่อยู่** (address) ซึ่งเป็นสตริงบนอักษร $\Gamma_b = \{1, 2, \ldots, b\}$ เช่น ที่อยู่ 21 หมายถึง “จากราก เลือกทางที่ 2 แล้วเลือกทางที่ 1” (ในรูปคือ C_5) บางสตริงอาจไม่ใช่ที่อยู่ที่ถูกต้อง หากทางเลือกที่ระบุไม่มีอยู่จริง

D ใช้สามเทป (ซึ่งจำลองด้วยเทปเดียวได้ตามทฤษฎีบท 5.6)

เทป	ชื่อ	หน้าที่
1	เทปข้อมูลเข้า	เก็บ w ไว้ตลอด ไม่มีการแก้ไข
2	เทปจำลอง	เก็บเทปของ N ตามกิ่งที่กำลังทดลอง
3	เทปที่อยู่	เก็บที่อยู่ปัจจุบันในต้นไม้ของ N

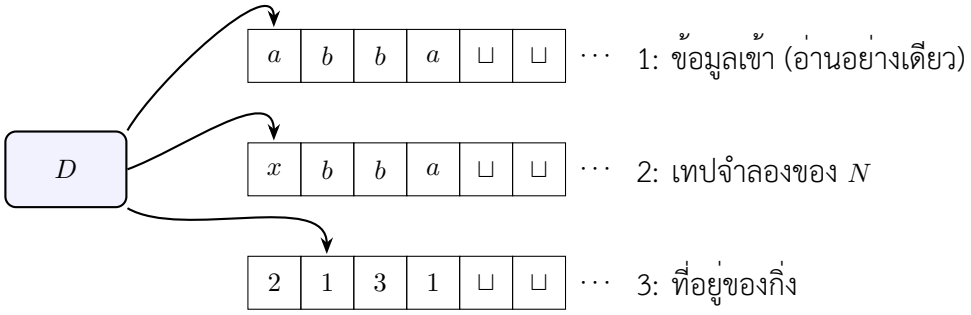
การทำงานของ D คือ

1. เริ่มด้วยเทป 3 ว่าง (แทนรากของต้นไม้)
2. คัดลอกเทป 1 ไปยังเทป 2 (ล้างของเดิมทิ้ง)
3. จำลอง N บนเทป 2 ตามกิ่งที่ระบุบนเทป 3 โดยแต่ละก้าวเลือกทางตามสัญลักษณ์ถัดไปบนเทป 3 ถ้าพบโครงแบบยอมรับให้*ยอมรับ* ถ้าสัญลักษณ์บนเทป 3 หมด หรือทางเลือกไม่มีอยู่จริง หรือเจอโครงแบบปฏิเสธ ให้ไปขั้นที่ 4
4. เปลี่ยนเทป 3 เป็นสตริงถัดไปตาม**ลำดับสั้นก่อน** (shortlex order: เรียงตามความยาวก่อน แล้วตามพจนานุกรม) แล้วกลับไปขั้นที่ 2

การไล่ที่อยู่ตามลำดับสั้นก่อนคือการค้นหาแนวกว้างนั่นเอง ถ้า N ยอมรับ w ด้วยกิ่งที่มีที่อยู่ α แล้ว D จะไปถึง α หลังจากลองสตริงจำนวนจำกัดก่อนหน้า และยอมรับ ส่วนถ้า N ไม่ยอมรับ w เลย D ก็ไม่มีทางยอมรับ □

บทแทรก 2.10. *ภาษาหนึ่งเป็นภาษาที่ทัวริงรู้จำได้ ก็ต่อเมื่อ มีเครื่องจักรทัวริงไม่เชิงกำหนดที่รู้จำภาษานั้น*

เราเรียกเครื่องไม่เชิงกำหนดว่าเป็น**ตัวตัดสิน** ถ้า*ทุก*กิ่งของต้นไม้การคำนวณบน*ทุก*ข้อมูลเข้าหยุด เราปรับ D ได้ว่า ถ้าทุกกิ่งที่ระดับหนึ่งปฏิเสธหมดแล้ว (ไม่มีกิ่งใดยาวต่อได้) ให้ D ปฏิเสธ เนื่องจาก



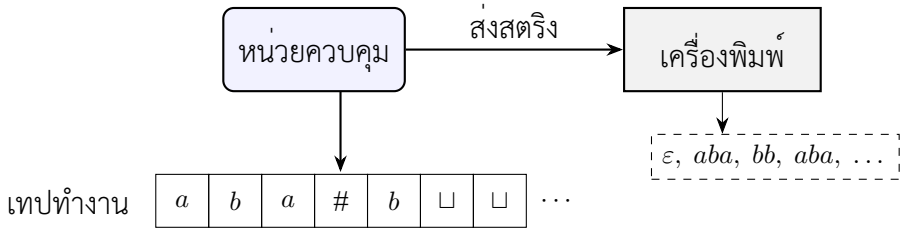
รูปที่ 2.7: เครื่องเชิงกำหนด D ที่จำลองเครื่องไม่เชิงกำหนด N บนเทปที่อยู่ ค่า 2131 หมายถึงกิ่งที่เลือกทางที่ 2, 1, 3, 1 ตามลำดับ

ต้นไม้ของตัวตัดสินมีกิ่งจำกัดและแต่ละกิ่งจำกัด (โดยบทตั้งของ König) ต้นไม้ทั้งต้นจึงจำกัด และ D จะหยุดเสมอ จึงได้

บทแทรก 2.11. *ภาษาหนึ่งตัดสินได้ ก็ต่อเมื่อ มีเครื่องจักรทัวริงไม่เชิงกำหนดที่ตัดสินภาษานั้น*

2.2.4 ตัวแจกแจง

ตัวแจกแจง (enumerator) คือเครื่องจักรทัวริงที่ต่อเข้ากับ**เครื่องพิมพ์** (printer) ตัวแจกแจงไม่รับข้อมูลเข้า มันเริ่มทำงานบนเทปทำงานที่ว่างเปล่า และระหว่างทำงานสามารถส่งสตริงออกไปพิมพ์ได้เป็นระยะ ภาษาที่ตัวแจกแจง E **แจกแจง** (enumerates) คือเซตของสตริงทั้งหมดที่ E พิมพ์ออกมาในที่สุด E อาจพิมพ์สตริงซ้ำกันหรือไม่เรียงลำดับก็ได้ และอาจไม่หยุดเลย (จำเป็นอย่างยิ่งเมื่อภาษามีสมาชิกไม่จำกัด)



รูปที่ 2.8: ตัวแจกแจงประกอบด้วยเครื่องจักรทัวริงกับเครื่องพิมพ์ เทปเริ่มต้นว่างเปล่า และสตริงที่พิมพ์ออกมาอาจซ้ำหรือไม่เรียงลำดับ

ทฤษฎีบท 2.12. *ภาษาหนึ่งเป็นภาษาที่ทัวริงรู้จำได้ ก็ต่อเมื่อ มีตัวแจกแจงที่แจกแจงภาษานั้น*

พิสูจน์. ($\Leftarrow$) สมมติ E แจกแจง L สร้างเครื่อง $M =$ "บนข้อมูลเข้า w: รัน E ทุกครั้งที่ E พิมพ์สตริงออกมา ให้เปรียบเทียบกับ w ถ้าตรงกันให*้ยอมรับ*" ถ้า $w \in L$ แล้ว E จะพิมพ์ w ในที่สุด M จึงยอมรับ ถ้า $w \notin L$ แล้ว M ไม่มีทางยอมรับ (อาจวนไม่รู้จบ) ดังนั้น $L(M) = L$

($\Rightarrow$) สมมติ M รู้จำ L ให้ $s_1, s_2, s_3, \ldots$ เป็นสตริงทั้งหมดบน Σ เรียงตามลำดับสั้นก่อน แนวคิดที่ไร้เดียงสาคือรัน M บน s_1 แล้วบน s_2 และต่อไปเรื่อย ๆ แต่ถ้า M วนไม่รู้จบบน s_1 เราจะไม่มีวันได้ทดลอง s_2 เราจึงใช้เทคนิค**การสลับขั้นตอน** (dovetailing) ดังนี้

E = "ไม่สนใจข้อมูลเข้า: สำหรับ $i = 1, 2, 3, \ldots$ ให้รัน M บนแต่ละสตริง $s_1, \ldots, s_i$ เป็นเวลา i ก้าว ถ้ารันใดยอมรับ ให้พิมพ์สตริงนั้นออกมา"

ถ้า M ยอมรับ s_j ภายใน t ก้าว แล้วเมื่อ $i \geq \max(j, t)$ เครื่อง E จะพิมพ์ s_j ออกมา (และพิมพ์ซ้ำในทุกรอบหลังจากนั้น ซึ่งไม่ผิดกติกา) ในทางกลับกัน E พิมพ์เฉพาะสตริงที่ M ยอมรับ ดังนั้น E แจกแจง L พอดี □

ชื่อ "แจกแจงได้แบบเวียนเกิด" (recursively enumerable) ของภาษาที่ทัวริงรู้จำได้ ก็มาจากทฤษฎีบทนี้นั่นเอง

2.2.5 ความทนทานของแบบจำลอง

แบบจำลองการคำนวณมีมากมาย นอกจากที่กล่าวมาแล้วยังมีเครื่องที่มีเทปยาวไม่จำกัดทั้งสองทาง เครื่องที่มีเทปสองมิติ เครื่องเข้าถึงสุ่ม (random-access machine) แคลคูลัสแลมบ์ดา (lambda calculus) ฟังก์ชันเวียนเกิด (recursive functions) และภาษาโปรแกรมทั่วไปที่สมมติว่ามีหน่วยความจำไม่จำกัด ทุกแบบจำลองที่สมเหตุสมผลเหล่านี้ล้วนมีคุณสมบัติร่วมกันสองข้อ

1. ทำงานได้เพียง*จำนวนจำกัด*ในแต่ละก้าว
2. เข้าถึงหน่วยความจำได้ไม่จำกัด

และได้รับการพิสูจน์แล้วว่าทั้งหมดมีพลังเท่ากับเครื่องจักรทัวริง เปรียบได้กับภาษาโปรแกรมต่าง ๆ เช่น Python หรือ C ที่ต่างกันมากในรูปแบบ แต่โปรแกรมที่เขียนในภาษาหนึ่งย่อมแปลงเป็นอีกภาษาหนึ่งได้เสมอ ดังนั้นคลาสของ*อัลกอริทึม*ที่ได้จึงไม่ขึ้นกับแบบจำลองที่เลือก

หมายเหตุ. ความทนทานนี้ใช้กับ "ความสามารถ" ในการคำนวณเท่านั้น ส่วน "ประสิทธิภาพ" อาจต่างกันได้ เช่น การจำลองเครื่อง k เทป ด้วยเครื่องเทปเดียวทำให้เวลาเพิ่มขึ้นเป็นกำลังสอง และการจำลองเครื่องไม่เชิงกำหนดในช่างต้นใช้เวลาเพิ่มขึ้นแบบเลขชี้กำลัง ประเด็นเรื่องเวลาเหล่านี้จะกลับมามีความสำคัญมากในเรื่องทฤษฎีความซับซ้อน

2.3 วิทยานิพนธ์เชิร์ช–ทัวริง

2.3.1 อัลกอริทึมคืออะไร

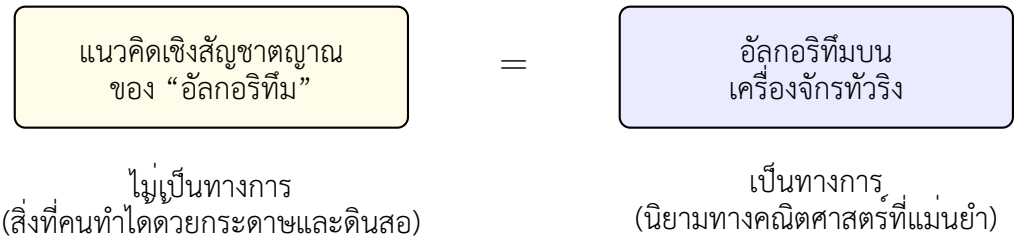
อัลกอริทึม (algorithm) คือชุดคำสั่งที่ชัดเจนและทำตามได้อย่างเป็นกลไก เพื่อแก้ปัญหาบางอย่าง มนุษย์ใช้อัลกอริทึมมาหลายพันปี เช่น อัลกอริทึมของยุคลิดสำหรับหา ห.ร.ม. แต่จนถึงต้นศตวรรษที่ 20 ยังไม่มีใครนิยามคำว่า "อัลกอริทึม" ได้อย่างแม่นยำทางคณิตศาสตร์ เพราะไม่มีความจำเป็น ตราบใดที่เราต้องการเพียง*แสดง*อัลกอริทึม ก็แค่เขียนมันออกมาให้ผู้อื่นเห็นว่าทำงานได้ แต่ถ้าต้องการพิสูจน์ว่า*ไม่มี*อัลกอริทึมสำหรับปัญหาใด เราต้องมีนิยามที่แน่นอน

ปัญหาข้อที่สิบของฮิลแบร์ท ในปี ค.ศ. 1900 David Hilbert ได้นำเสนอปัญหาท้าทาย 23 ข้อต่อที่ประชุมนักคณิตศาสตร์นานาชาติ ข้อที่สิบขอให้หา "กระบวนการ" ที่ทดสอบได้ในจำนวนขั้นตอนจำกัดว่า พหุนามหลายตัวแปรที่มีสัมประสิทธิ์เป็นจำนวนเต็มที่กำหนดให้ มี**รากเป็นจำนวนเต็ม** (integral root) หรือไม่ ตัวอย่างเช่น พหุนาม

$$p(x, y, z) = x^2 + y^2 - z^2$$

มีรากเป็นจำนวนเต็ม (เช่น $x = 3, y = 4, z = 5$) ส่วนพหุนาม $q(x, y) = x^2 - 2y^2 - 3$ ไม่มี (ลองพิจารณามอดุโล 8 จะพบว่า $x^2 - 2y^2$ ไม่มีวันเหลือเศษ 3) ฮิลแบร์ทไม่ได้ใช้คำว่าอัลกอริทึม แต่เห็นได้ชัดว่าเขาตั้งสมมติฐานว่ากระบวนการเช่นนั้นมีอยู่และรอให้คนค้นพบ

ปัจจุบันเรารู้แล้วว่า*ไม่มี*อัลกอริทึมดังกล่าว ผลนี้พิสูจน์สำเร็จในปี ค.ศ. 1970 โดย Yuri Matiyasevich ซึ่งต่อยอดจากงานของ Martin Davis, Hilary Putnam และ Julia Robinson การพิสูจน์เช่นนี้เป็นไปได้ก็เพราะในช่วงทศวรรษ 1930 Alonzo Church (ผ่านแคลคูลัสแลมบ์ดา) และ Alan Turing (ผ่านเครื่องจักรของเขา) ได้ให้นิยามที่แม่นยำของอัลกอริทึม และนิยามทั้งสองได้รับการพิสูจน์ว่าเท่ากัน



รูปที่ 2.9: วิทยานิพนธ์เชิร์ช–ทัวริงเชื่อมแนวคิดที่ไม่เป็นทางการเข้ากับนิยามที่เป็นทางการ

วิทยานิพนธ์เชิร์ช–ทัวริง (Church–Turing thesis)

แนวคิดเชิงสัญชาตญาณของ “อัลกอริทึม” ตรงกันพอดีกับอัลกอริทึมบนเครื่องจักรทัวริง กล่าวคือ ปัญหาใดแก้ได้ด้วยกระบวนการเชิงกลไกใด ๆ ปัญหานั้นก็แก้ได้ด้วยเครื่องจักรทัวริง

วิทยานิพนธ์นี้ไม่ใช่ทฤษฎีบทที่พิสูจน์ได้ เพราะด้านหนึ่งของมันเป็นแนวคิดที่ไม่เป็นทางการ แต่หลักฐานสนับสนุนมีน้ำหนักมาก ได้แก่ ความทนทานของแบบจำลองที่กล่าวไว้ในหัวข้อที่แล้ว และการที่ยังไม่เคยมีใครพบกระบวนการที่ทุกคนยอมรับว่าเป็นอัลกอริทึม แต่เครื่องจักรทัวริงทำไม่ได้

เมื่อแปลปัญหาข้อที่สิบของฮิลแบร์ทเป็นภาษาของเรา ให้

$$D = \{\, p \mid p \text{ เป็นพหุนามที่มีสัมประสิทธิ์เป็นจำนวนเต็มและมีรากเป็นจำนวนเต็ม} \,\}$$

ปัญหาของฮิลแบร์ทก็คือคำถามว่า D ตัดสินได้หรือไม่ คำตอบคือไม่ได้ แต่ที่น่าสนใจคือ D *รู้จำได้* เครื่องที่รู้จำ D เพียงไล่ลองค่าจำนวนเต็มทุกชุด เรียงตามค่าสัมบูรณ์มากสุดจากน้อยไปมาก (0, จากนั้นชุดที่ใช้ค่าใน $\{-1, 0, 1\}$, และต่อไปเรื่อย ๆ) แล้วยอมรับเมื่อพบชุดที่ทำให้ p มีค่าเป็นศูนย์ ถ้า p ไม่มีราก เครื่องนี้จะวนไม่รู้จบ สำหรับพหุนามตัวแปรเดียวมีขอบเขตของรากที่คำนวณได้จากสัมประสิทธิ์ จึงตัดสินได้ แต่ในกรณีหลายตัวแปรไม่มีขอบเขตเช่นนั้น

2.3.2 ระดับของการบรรยายเครื่องจักรทัวริง

จากนี้ไปเราจะใช้เครื่องจักรทัวริงเป็นแบบจำลองของอัลกอริทึม และเพื่อไม่ให้จมอยู่กับรายละเอียด เราอาจบรรยายเครื่องได้สามระดับ

1. **การบรรยายเชิงรูปนัย** (formal description) ระบุทูเพิลเจ็ดสิ่งครบถ้วน รวมทั้ง δ ทุกค่า เช่น แผนภาพสถานะของ M_{PAL} ในรูปที่ 2.3 ระดับนี้ละเอียดที่สุดแต่ยาวและอ่านยาก

2. **การบรรยายระดับการสร้าง** (implementation description) ใช้ภาษาธรรมชาติอธิบายว่าหัวอ่านเคลื่อนที่อย่างไร และเก็บข้อมูลบนเทปอย่างไร โดยไม่ระบุสถานะทีละตัว เช่น “วิ่งไปทางขวาจนพบช่องว่าง แล้วถอยหนึ่งช่อง” การพิสูจน์ทฤษฎีบท 5.6 และ 2.9 อยู่ในระดับนี้

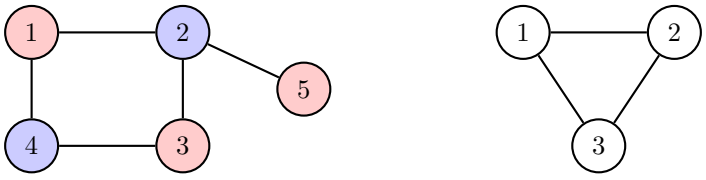
3. **การบรรยายระดับสูง** (high-level description) ใช้ภาษาธรรมชาติอธิบายอัลกอริทึมโดยไม่พูดถึงเทปหรือหัวอ่านเลย เหมือนการเขียนรหัสเทียม (pseudocode) ทั่วไป

ระดับที่สามเป็นระดับที่เราจะใช้เป็นหลักต่อจากนี้ วิทยานิพนธ์เชิร์ช–ทัวริงรับรองว่าทุกขั้นตอนที่ชัดเจนและทำได้เชิงกลไก แปลงลงสู่ระดับล่างได้เสมอ ผู้เรียนควรฝึกแปลงระหว่างระดับจนมั่นใจว่าทำได้ ก่อนจะใช้ระดับสูงอย่างคล่องแคล่ว

2.3.3 การเข้ารหัสวัตถุเป็นสตริง

ข้อมูลเข้าของเครื่องจักรทัวริงเป็นสตริงเสมอ แต่เรามักต้องการให้อัลกอริทึมทำงานกับวัตถุอื่น เช่น จำนวน พหุนาม กราฟ ไวยากรณ์ หรือแม้แต่เครื่องจักรทัวริงด้วยกันเอง วิธีแก้คือ**เข้ารหัส** (encode) วัตถุให้เป็นสตริง เราเขียน $\langle O \rangle$ แทนการเข้ารหัสของวัตถุ O และ $\langle O_1, O_2, \ldots, O_k \rangle$ แทนการเข้ารหัสของวัตถุหลายชิ้นรวมกันเป็นสตริงเดียว

วิธีเข้ารหัสที่เลือกไม่สำคัญนัก ตราบใดที่เครื่องจักรทัวริงแปลงระหว่างการเข้ารหัสที่สมเหตุสมผลสองแบบได้ เช่น จำนวนธรรมชาติ n อาจเข้ารหัสเป็นเลขฐานสองหรือฐานสิบก็ได้ เพราะแปลงไปมาได้ด้วยอัลกอริทึม เมื่อบรรยายในระดับสูง เราถือว่าเครื่องตรวจสอบก่อนเสมอว่าข้อมูลเข้าเป็นการเข้ารหัสที่ถูกต้องของวัตถุที่ต้องการ และปฏิเสธทันทีหากไม่ใช่ จึงมักไม่เขียนขั้นตอนนี้ออกมา



กราฟ G_1 กราฟ G_2

รูปที่ 2.10: กราฟ G_1 แบ่งสองส่วนได้ (ระบายสีแดง/น้ำเงินโดยไม่มีเส้นเชื่อมสีเดียวกัน) ส่วน G_2 เป็นวัฏจักรความยาวคี่ จึงแบ่งสองส่วนไม่ได้

ตัวอย่างเช่น กราฟไม่มีทิศทาง G_1 ในรูปที่ 2.10 อาจเข้ารหัสเป็นรายการจุดยอดตามด้วยรายการเส้นเชื่อม

$$\langle G_1 \rangle = (1,2,3,4,5)((1,2),(2,3),(3,4),(4,1),(2,5))$$

ซึ่งเป็นสตริงบนอักษรที่มีตัวเลข วงเล็บ และจุลภาค

2.3.4 ตัวอย่างการบรรยายระดับสูง: กราฟแบ่งสองส่วน

กราฟไม่มีทิศทาง $G = (V, E)$ เป็น**กราฟแบ่งสองส่วน** (bipartite graph) ถ้าแบ่งจุดยอดออกเป็นสองกลุ่ม โดยทุกเส้นเชื่อมมีปลายหนึ่งอยู่ในแต่ละกลุ่ม หรือเทียบเท่ากับการระบายสีจุดยอดด้วยสองสีได้

โดยไม่มีเส้นเชื่อมใดที่ปลายทั้งสองมีสีเดียวกัน พิจารณาภาษา

$$\mathit{BIP} = \{\langle G\rangle \mid G \text{ เป็นกราฟไม่มีทิศทางที่แบ่งสองส่วนได้}\}$$

ตัวอย่าง 2.13. เครื่องจักรทัวริงต่อไปนี้ตัดสิน *BIP* โดยระบายสีแบบไล่ทีละชั้น เริ่มจากจุดยอดที่ยังไม่มีสี แล้วบังคับให้เพื่อนบ้านได้สีตรงข้าม

$B =$ "บนข้อมูลเข้า $\langle G\rangle$ ซึ่งเป็นการเข้ารหัสของกราฟไม่มีทิศทาง $G = (V, E)$:

1. ถ้ายังมีจุดยอดที่ไม่มีสี ให้เลือกมาหนึ่งจุด v และระบาย v เป็นสีแดง ถ้าไม่มีแล้วให้ไปขั้นที่ 5
2. ทำซ้ำจนกว่าจะไม่มีการระบายสีใหม่ในรอบนั้น:
3. สำหรับแต่ละเส้นเชื่อม $(u, u') \in E$ ที่ปลายหนึ่งมีสีแล้วแต่อีกปลายยังไม่มีสี ให้ระบายปลายที่ยังไม่มีสีเป็นสีตรงข้ามกับปลายที่มีสี
4. กลับไปขั้นที่ 1
5. ตรวจทุกเส้นเชื่อมใน E ถ้ามีเส้นเชื่อมที่ปลายทั้งสองมีสีเดียวกันให้*ปฏิเสธ* ถ้าไม่มีให้*ยอมรับ*"

การหยุดของเครื่อง ขั้นที่ 1 ทำซ้ำได้ไม่เกิน $|V|$ ครั้ง และแต่ละรอบของขั้นที่ 3 (ยกเว้นรอบสุดท้าย) ระบายสีจุดยอดใหม่อย่างน้อยหนึ่งจุด จำนวนรอบทั้งหมดจึงไม่เกิน $2|V|$ และแต่ละรอบตรวจเส้นเชื่อม $|E|$ เส้น B จึงหยุดเสมอ

ความถูกต้อง ถ้า B ยอมรับ แสดงว่าสีที่ได้เป็นการระบายสองสีที่ถูกต้อง G จึงแบ่งสองส่วนได้ ในทางกลับกัน สมมติ G แบ่งสองส่วนได้ ในแต่ละส่วนประกอบเชื่อมโยง (connected component) สีของทุกจุดยอดถูกบังคับโดยสีของจุดเริ่มต้น เพราะเพื่อนบ้านต้องมีสีตรงข้ามเสมอ การระบายสองสีที่ถูกต้องของส่วนประกอบนั้น (ซึ่งมีอยู่ตามสมมติฐาน หลังจากสลับสีถ้าจำเป็น) จึงตรงกับสีที่ B ให้ ทำให้ขั้นที่ 5 ไม่พบความขัดแย้ง และ B ยอมรับ

ตัวอย่างการทำงาน บน $\langle G_1\rangle$ ถ้าเริ่มที่จุดยอด 1 (แดง) จะได้ 2 และ 4 เป็นน้ำเงิน จากนั้น 3 และ 5 เป็นแดง ไม่มีเส้นเชื่อมใดมีสีเดียวกันที่ปลายทั้งสอง B จึงยอมรับ บน $\langle G_2\rangle$ ถ้าเริ่มที่ 1 (แดง) ในรอบแรก 2 และ 3 ต่างได้สีน้ำเงินเพราะต่อกับ 1 แต่เส้นเชื่อม $(2, 3)$ มีปลายทั้งสองเป็นสีน้ำเงิน B จึงปฏิเสธ

สรุปท้ายบท

สรุปบทที่ 2

- เครื่องจักรทัวริง $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ มีเทปไม่จำกัด เขียนได้ และเลื่อนหัวอ่านได้ทั้งสองทิศ
- โครงแบบ $u\,q\,v$ บอกสถานะ เนื้อหาเทป และตำแหน่งหัวอ่าน; $C_1 \vdash C_2$ คือการเปลี่ยนในหนึ่งก้าว
- บนข้อมูลเข้าหนึ่ง ๆ เครื่องจะยอมรับ ปฏิเสธ หรือวนไม่รู้จบ
- *รู้จำได้* : มีเครื่องที่ยอมรับพอดีสตริงในภาษา; *ตัดสินได้* : มีเครื่องเช่นนั้นที่หยุดเสมอ
- เครื่องหลายเทป เครื่องไม่เชิงกำหนด และตัวแจกแจง มีพลังเท่ากับเครื่องเทปเดียว (การจำลองเครื่องไม่เชิงกำหนดต้องค้นหาแนวกว้าง ส่วนตัวแจกแจงใช้การสลับขั้นตอน)
- วิทยานิพนธ์เชิร์ช–ทัวริง: อัลกอริทึมในความหมายเชิงสัญชาตญาณ $=$ อัลกอริทึมบนเครื่องจักรทัวริง ทำให้พิสูจน์ได้ว่าปัญหาบางข้อ เช่น ปัญหาข้อที่สิบของฮิลแบร์ท ไม่มีอัลกอริทึม
- เราบรรยายเครื่องได้สามระดับ (เชิงรูปนัย ระดับการสร้าง ระดับสูง) และเขียน $\langle O \rangle$ แทนการเข้ารหัสวัตถุ O เป็นสตริง

บทที่ 3

ภาษาที่ตัดสินได้ ภาษาที่รู้จำได้ และความตัดสินไม่ได้

ในบทก่อนเราได้รู้จักเครื่องจักรทัวริง (Turing machine) ในฐานะแบบจำลองที่ทรงพลังที่สุดของการคำนวณ และยอมรับวิทยานิพนธ์เชิร์ช–ทัวริง (Church–Turing thesis) ว่า "ขั้นตอนวิธี" ใด ๆ ก็ตาม สามารถเขียนเป็นเครื่องจักรทัวริงได้ บทนี้จะใช้เครื่องมือดังกล่าวตอบคำถามที่ลึกกว่าเดิม คือ *ปัญหาใดบ้างที่คอมพิวเตอร์แก้ได้ และปัญหาใดบ้างที่ไม่มีวันแก้ได้ ไม่ว่าเราจะมีเวลาและหน่วยความจำมากเพียงใด*

เนื้อหาแบ่งออกเป็นสามส่วน ส่วนแรกรวบรวมตัวอย่างปัญหาที่ **ตัดสินได้** (decidable) โดยเฉพาะปัญหาเกี่ยวกับออโตมาตาและไวยากรณ์ที่เรารู้จักดีแล้ว ส่วนที่สองศึกษาความสัมพันธ์ระหว่างภาษาที่ตัดสินได้กับภาษาที่ **รู้จำได้แบบทัวริง** (Turing-recognizable) และส่วนสุดท้ายพิสูจน์ผลลัพธ์สำคัญที่สุดผลหนึ่งของวิทยาการคอมพิวเตอร์ นั่นคือ มีปัญหาที่กำหนดได้ชัดเจนแต่ **ตัดสินไม่ได้** (undecidable)

3.1 ภาษาที่ตัดสินได้

ทบทวนนิยามสั้น ๆ ก่อน เครื่องจักรทัวริง M เป็น **ตัวตัดสิน** (decider) ถ้า M หยุดทำงาน (halt) บนข้อมูลเข้า *ทุก* สตริง โดยจบที่สถานะ q_{accept} หรือ q_{reject} อย่างใดอย่างหนึ่ง ภาษา L เป็น **ภาษาที่ตัดสินได้** ถ้ามีตัวตัดสิน M ซึ่ง $L(M) = L$

เมื่อพูดถึง "ปัญหา" เราจะแปลงปัญหาให้เป็นภาษาเสมอ วัตถุใด ๆ เช่น ออโตมาตา ไวยากรณ์ กราฟ หรือเครื่องจักรทัวริงเอง จะถูกเข้ารหัสเป็นสตริงด้วยสัญกรณ์ $\langle O \rangle$ และถ้ามีวัตถุหลายชิ้นจะเขียน $\langle O_1, O_2, \ldots, O_k \rangle$ รายละเอียดของการเข้ารหัสไม่สำคัญนัก ขอเพียงให้เครื่องจักรทัวริงถอดรหัสกลับได้ เมื่อเราอธิบายเครื่องจักรในระดับสูง จะถือโดยปริยายว่าขั้นแรกของเครื่องคือการตรวจรูปแบบข้อมูลเข้า และถ้าข้อมูลเข้าไม่ใช่รหัสที่ถูกต้อง เครื่องจะปฏิเสธทันที

3.1.1 ปัญหาที่ตัดสินได้เกี่ยวกับภาษาปรกติ

ปัญหาการยอมรับของ DFA

ปัญหาพื้นฐานที่สุดคือ "ออโตมาตาจำกัดเชิงกำหนด (DFA) ที่กำหนดให้ ยอมรับสตริงที่กำหนดให้หรือไม่" เขียนเป็นภาษาได้ว่า

$$A_{\text{DFA}} = \{\langle B, w\rangle \mid B \text{ เป็น DFA ที่ยอมรับสตริง } w\}.$$

สังเกตว่าคำถาม "B ยอมรับ w หรือไม่" กับคำถาม "$\langle B, w\rangle \in A_{\text{DFA}}$ หรือไม่" เป็นคำถามเดียวกัน การแสดงว่าภาษานี้ตัดสินได้จึงเท่ากับการแสดงว่าปัญหานี้มีขั้นตอนวิธีที่จบเสมอ

ทฤษฎีบท 3.1. A_{DFA} *เป็นภาษาที่ตัดสินได้*

พิสูจน์. สร้างเครื่องจักรทัวริง M_1 ดังนี้

M_1 = "เมื่อได้รับข้อมูลเข้า $\langle B, w\rangle$ โดยที่ B เป็น DFA และ w เป็นสตริง

1. จำลองการทำงานของ B บน w ทีละสัญลักษณ์ โดยบันทึกสถานะปัจจุบันของ B และตำแหน่งปัจจุบันใน w ไว้บนเทป
2. เมื่ออ่าน w หมดแล้ว ถ้าสถานะสุดท้ายเป็นสถานะยอมรับของ B ให้ *ยอมรับ* มิฉะนั้นให้ *ปฏิเสธ*"

ในแต่ละก้าว M_1 เพียงค้นหาฟังก์ชันเปลี่ยนสถานะของ B จากตารางที่เข้ารหัสไว้ ซึ่งใช้เวลาจำกัด และมีทั้งหมด $|w|$ ก้าว ดังนั้น M_1 หยุดเสมอ และยอมรับก็ต่อเมื่อ B ยอมรับ w □

ปัญหาการยอมรับของ NFA และนิพจน์ปรกติ

ในทำนองเดียวกันกำหนด

$$A_{\text{NFA}} = \{\langle B, w\rangle \mid B \text{ เป็น NFA ที่ยอมรับ } w\},$$
$$A_{\text{REX}} = \{\langle R, w\rangle \mid R \text{ เป็นนิพจน์ปรกติที่ก่อกำเนิด } w\}.$$

ทฤษฎีบท 3.2. A_{NFA} *เป็นภาษาที่ตัดสินได้*

พิสูจน์. แทนที่จะจำลอง NFA โดยตรง (ซึ่งต้องติดตามหลายกิ่งพร้อมกัน) เราใช้ผลที่รู้แล้วว่า NFA ทุกตัวแปลงเป็น DFA ที่สมมูลกันได้ ด้วยการสร้างแบบเซตย่อย (subset construction) ซึ่งเป็นขั้นตอนวิธีที่จบเสมอ

N_1 = "เมื่อได้รับ $\langle B, w\rangle$ โดยที่ B เป็น NFA

1. แปลง B เป็น DFA C ที่สมมูลกันด้วยการสร้างแบบเซตย่อย
2. รัน M_1 จากทฤษฎีบท 3.1 บนข้อมูลเข้า $\langle C, w\rangle$
3. ถ้า M_1 ยอมรับให้ *ยอมรับ* มิฉะนั้นให้ *ปฏิเสธ*"

ขั้นที่ 1 ใช้เวลาจำกัด (อาจมากถึง 2^k สถานะเมื่อ B มี k สถานะ แต่ก็ยังจำกัด) และ M_1 เป็นตัวตัดสิน ดังนั้น N_1 เป็นตัวตัดสินของ A_{NFA} □

ทฤษฎีบท 3.3. A_{REX} *เป็นภาษาที่ตัดสินได้*

พิสูจน์. นิพจน์ปรกติ R ทุกตัวแปลงเป็น NFA ที่สมมูลกันได้ด้วยการสร้างแบบอุปนัยตามโครงสร้างของ R (ใช้การประกอบสำหรับยูเนียน การต่อกัน และดาวคลีน) ตัวตัดสินจึงทำดังนี้

$P_1 =$ "เมื่อได้รับ $\langle R, w\rangle$: แปลง R เป็น NFA A แล้วรัน N_1 บน $\langle A, w\rangle$ และตอบตาม N_1" □

ปัญหาความว่างของ DFA

คำถามต่อไปคือ "DFA ที่กำหนดให้ยอมรับสตริงใดบ้างหรือไม่"

$$E_{\text{DFA}} = \{\langle A\rangle \mid A \text{ เป็น DFA และ } L(A) = \emptyset\}.$$

ปัญหานี้ต่างจากปัญหาการยอมรับตรงที่ไม่มีสตริงให้จำลอง และเราไม่สามารถลองทุกสตริงได้เพราะมีอนันต์ตัว แต่ $L(A) \neq \emptyset$ ก็ต่อเมื่อมีสถานะยอมรับบางสถานะที่ *เข้าถึงได้* จากสถานะเริ่มต้นตามลูกศรในแผนภาพ ปัญหาจึงกลายเป็นปัญหาการเข้าถึงในกราฟจำกัด

ทฤษฎีบท 3.4. E_{DFA} *เป็นภาษาที่ตัดสินได้*

พิสูจน์. $T =$ "เมื่อได้รับ $\langle A\rangle$ โดยที่ A เป็น DFA

1. ทำเครื่องหมายที่สถานะเริ่มต้นของ A
2. ทำซ้ำจนกว่าจะไม่มีสถานะใหม่ถูกทำเครื่องหมาย: ทำเครื่องหมายที่สถานะ s ใด ๆ ที่มีลูกศรเข้าจากสถานะที่ถูกทำเครื่องหมายแล้ว
3. ถ้าไม่มีสถานะยอมรับใดถูกทำเครื่องหมาย ให้ *ยอมรับ* มิฉะนั้นให้ *ปฏิเสธ*"

ทุกรอบของขั้นที่ 2 (ยกเว้นรอบสุดท้าย) ทำเครื่องหมายสถานะใหม่อย่างน้อยหนึ่งสถานะ และ A มีสถานะจำกัด ดังนั้นขั้นที่ 2 วนไม่เกิน $|Q|$ รอบแล้วจบ สถานะที่ถูกทำเครื่องหมายคือสถานะที่เข้าถึงได้ทั้งหมดพอดี ซึ่งพิสูจน์ได้ด้วยอุปนัยบนความยาวของเส้นทาง □

ปัญหาความสมมูลของ DFA

สุดท้ายสำหรับภาษาปรกติ พิจารณาคำถาม "DFA สองตัวรู้จำภาษาเดียวกันหรือไม่"

$$EQ_{\text{DFA}} = \{\langle A, B\rangle \mid A, B \text{ เป็น DFA และ } L(A) = L(B)\}.$$

แนวคิดคือสร้าง DFA ตัวใหม่ C ที่ยอมรับเฉพาะสตริงที่ A กับ B "ไม่เห็นตรงกัน" นั่นคือสตริงที่อยู่ในภาษาหนึ่งแต่ไม่อยู่ในอีกภาษาหนึ่ง เซตนี้เรียกว่า **ผลต่างสมมาตร** (symmetric difference)

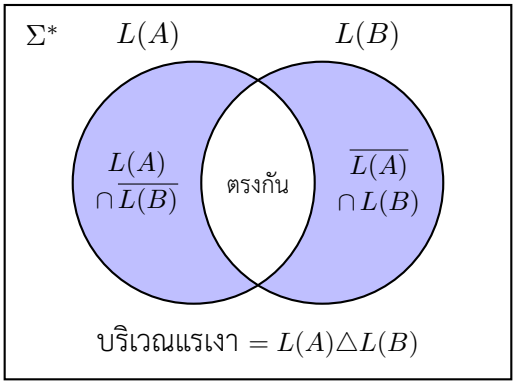
$$L(C) = \big(L(A) \cap \overline{L(B)}\big) \cup \big(\overline{L(A)} \cap L(B)\big) = L(A) \triangle L(B).$$

ดังรูปที่ 3.1 ส่วนที่แรเงาคือ $L(C)$ และ $L(C) = \emptyset$ ก็ต่อเมื่อ $L(A) = L(B)$

ทฤษฎีบท 3.5. EQ_{DFA} *เป็นภาษาที่ตัดสินได้*

พิสูจน์. ภาษาปรกติปิดภายใต้คอมพลีเมนต์ อินเทอร์เซกชัน และยูเนียน และการสร้างแต่ละแบบ (สลับสถานะยอมรับ หรือสร้างออโตมาตาผลคูณ) เป็นขั้นตอนวิธีที่จบเสมอ

$F =$ "เมื่อได้รับ $\langle A, B\rangle$ โดยที่ A, B เป็น DFA



รูปที่ 3.1: ผลต่างสมมาตรของสองภาษา บริเวณแรเงาว่างก็ต่อเมื่อวงกลมทั้งสองทับกันสนิท

1. สร้าง DFA C ที่ $L(C) = L(A) \triangle L(B)$ ตามสูตรข้างต้น
2. รันตัวตัดสิน T ของ E_{DFA} บน $\langle C \rangle$
3. ถ้า T ยอมรับให้ *ยอมรับ* ถ้า T ปฏิเสธให้ *ปฏิเสธ"*

ทุกขั้นจบในเวลาจำกัด และ F ยอมรับก็ต่อเมื่อ $L(A) \triangle L(B) = \emptyset$ ซึ่งสมมูลกับ $L(A) = L(B)$ □

3.1.2 ปัญหาที่ตัดสินได้เกี่ยวกับภาษาไม่พึ่งบริบท

ปัญหาการยอมรับของ CFG

กำหนด

$$A_{\mathsf{CFG}} = \{\langle G, w \rangle \mid G \text{ เป็น CFG ที่ก่อกำเนิดสตริง } w\}.$$

ความคิดแรกที่อาจผุดขึ้นคือ "ลองไล่การแปลง (derivation) ทุกแบบของ G แล้วดูว่าได้ w หรือไม่" ปัญหาคือการแปลงมีอนันต์แบบ ถ้า $w \notin L(G)$ การค้นหาแบบนี้จะไม่มีวันจบ ผลที่ได้จึงเป็นเพียงเครื่องรู้จำ (recognizer) ไม่ใช่ตัวตัดสิน ทางแก้คือจำกัดความยาวของการแปลงที่ต้องตรวจ โดยอาศัย **รูปแบบปรกติของชอมสกี** (Chomsky normal form, CNF) ซึ่งมีกฎเพียงสองแบบคือ $A \to BC$ และ $A \to a$ (และอาจมี $S \to \varepsilon$)

บทตั้ง 3.6. *ถ้า G อยู่ในรูป CNF และ $w \in L(G)$ โดย $|w| = n \geq 1$ แล้ว การแปลงใด ๆ ของ w ใน G ใช้ $2n - 1$ ขั้นพอดี*

พิสูจน์. กฎแบบ $A \to BC$ เพิ่มจำนวนตัวแปรในสายอักขระขึ้นหนึ่งตัว เริ่มจากตัวแปรเดียว S ต้องใช้กฎแบบนี้ $n - 1$ ครั้งเพื่อให้ได้ตัวแปร n ตัว และกฎแบบ $A \to a$ เปลี่ยนตัวแปรหนึ่งตัวเป็นเทอร์มินัลหนึ่งตัว ต้องใช้ n ครั้งเพื่อให้ได้เทอร์มินัล n ตัว รวม $(n-1) + n = 2n - 1$ ขั้น □

ตัวอย่าง 3.7. ให้ G ใน CNF มีกฎ $S \to XY,\ X \to XX \mid a,\ Y \to b$ ซึ่งก่อกำเนิด $a^k b$ เมื่อ $k \geq 1$ สำหรับ $w = aab$ ($n = 3$) การแปลง

$$S \Rightarrow XY \Rightarrow XXY \Rightarrow aXY \Rightarrow aaY \Rightarrow aab$$

ใช้ $5 = 2 \cdot 3 - 1$ ขั้นตรงตามบทตั้ง

ทฤษฎีบท 3.8. *A_{CFG} เป็นภาษาที่ตัดสินได้*

พิสูจน์. $S =$ "เมื่อได้รับ $\langle G, w\rangle$ โดยที่ G เป็น CFG

1. แปลง G เป็นไวยากรณ์ G' ที่สมมูลกันในรูป CNF
2. ถ้า $w = \varepsilon$ ให้ตรวจว่า G' มีกฎ $S_0 \to \varepsilon$ หรือไม่ ถ้ามีให้ *ยอมรับ* ถ้าไม่มีให้ *ปฏิเสธ*
3. ถ้า $|w| = n \geq 1$ ให้ไล่รายการการแปลงทุกแบบที่ยาว $2n - 1$ ขั้น (ซึ่งมีจำนวนจำกัด)
4. ถ้ามีการแปลงใดได้ผลเป็น w ให้ *ยอมรับ* มิฉะนั้นให้ *ปฏิเสธ*"

การแปลงเป็น CNF เป็นขั้นตอนวิธีที่จบเสมอ และในขั้นที่ 3 แต่ละขั้นของการแปลงมีทางเลือกจำกัด (เลือกตัวแปรและกฎจากเซตจำกัด) จำนวนการแปลงที่ต้องตรวจจึงจำกัด ความถูกต้องตามมาจากบทตั้ง 3.6 □

ปัญหาความว่างของ CFG

$$E_{\text{CFG}} = \{\langle G\rangle \mid G \text{ เป็น CFG และ } L(G) = \emptyset\}.$$

เราไม่สามารถใช้ S ตรวจทุกสตริงได้เพราะมีอนันต์ตัว แนวคิดคล้ายกับ E_{DFA} แต่ทำ "ย้อนจากล่างขึ้นบน": เราจะหาว่าตัวแปรใดบ้าง *ก่อกำเนิดสตริงของเทอร์มินัลได้* เริ่มจากเทอร์มินัลเองซึ่งก่อกำเนิดตัวมันเองได้ชัดเจน แล้วขยายออกไปทีละชั้น

ทฤษฎีบท 3.9. E_{CFG} *เป็นภาษาที่ตัดสินได้*

พิสูจน์. $R =$ "เมื่อได้รับ $\langle G\rangle$ โดยที่ G เป็น CFG

1. ทำเครื่องหมายที่สัญลักษณ์เทอร์มินัลทุกตัวใน G
2. ทำซ้ำจนกว่าจะไม่มีตัวแปรใหม่ถูกทำเครื่องหมาย: ถ้ามีกฎ $A \to U_1U_2\cdots U_k$ ที่ $U_1, \ldots, U_k$ ทุกตัวถูกทำเครื่องหมายแล้ว (รวมกรณี $k = 0$ คือ $A \to \varepsilon$) ให้ทำเครื่องหมายที่ A
3. ถ้าตัวแปรเริ่มต้นไม่ถูกทำเครื่องหมาย ให้ *ยอมรับ* มิฉะนั้นให้ *ปฏิเสธ*"

จำนวนตัวแปรจำกัด ขั้นที่ 2 จึงวนไม่เกินจำนวนตัวแปรรอบ โดยอุปนัยจะได้ว่าตัวแปรถูกทำเครื่องหมาย ก็ต่อเมื่อมันก่อกำเนิดสตริงเทอร์มินัลบางสตริงได้ □

ปัญหาความสมมูลของ CFG

ถ้าจะลองใช้วิธีเดียวกับ EQ_{DFA} กับภาษา

$$EQ_{\text{CFG}} = \{\langle G, H\rangle \mid G, H \text{ เป็น CFG และ } L(G) = L(H)\}$$

จะติดขัดทันที เพราะภาษาไม่พึ่งบริบท *ไม่ปิด* ภายใต้คอมพลีเมนต์และอินเทอร์เซกชัน เราจึงสร้างไวยากรณ์ของผลต่างสมมาตรไม่ได้ ที่จริงแล้วข้อจำกัดนี้ไม่ใช่แค่เรื่องเทคนิค: EQ_{CFG} *เป็นภาษาที่ตัดสินไม่ได้* ซึ่งพิสูจน์ได้ด้วยเทคนิคการลดทอน (reduction) ที่จะศึกษาในบทถัดไป

ภาษาไม่พึ่งบริบททุกภาษาตัดสินได้

ทฤษฎีบท 3.10. *ภาษาไม่พึ่งบริบททุกภาษาเป็นภาษาที่ตัดสินได้*

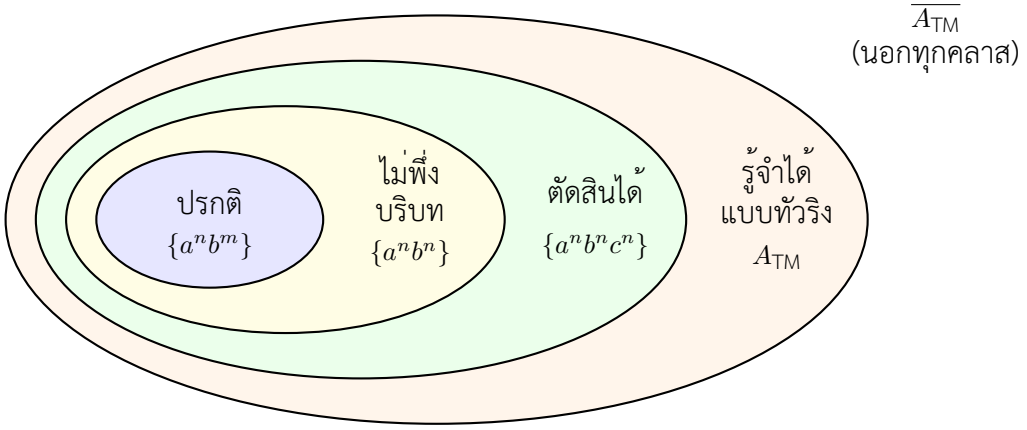
พิสูจน์. ให้ L เป็นภาษาไม่พึ่งบริบท และ G เป็น CFG ที่ $L(G) = L$ สร้างเครื่องจักรทัวริงที่ "ฝัง" G ไว้ในตัว

M_G = "เมื่อได้รับข้อมูลเข้า w: รันตัวตัดสิน S จากทฤษฎีบท 3.8 บน $\langle G, w\rangle$ ถ้า S ยอมรับให้*ยอมรับ* มิฉะนั้นให้ *ปฏิเสธ*"

เนื่องจาก S หยุดเสมอ M_G ก็หยุดเสมอ และ M_G ยอมรับ w ก็ต่อเมื่อ $w \in L(G) = L$ □

หมายเหตุ. ความต่างระหว่าง M_G กับ S อยู่ที่ว่า G เป็นส่วนหนึ่งของ *ตัวเครื่อง* ไม่ใช่ส่วนของข้อมูลเข้า สังเกตด้วยว่าเราไม่ได้พิสูจน์โดยจำลองออโตมาตาแบบกดลง (PDA) ตรง ๆ เพราะกิ่งการคำนวณบางกิ่งของ PDA แบบไม่กำหนดอาจผลักและดึงสแตกไปเรื่อย ๆ โดยไม่จบ การจำลองแบบตรงไปตรงมาจึงไม่รับประกันว่าจะหยุด

เมื่อรวมกับผลจากบทก่อน ๆ เราได้ลำดับชั้นของคลาสภาษาดังรูปที่ 3.2 ภาษาปรกติทุกภาษาเป็นภาษาไม่พึ่งบริบท ภาษาไม่พึ่งบริบททุกภาษาตัดสินได้ (ทฤษฎีบท 3.10) และภาษาที่ตัดสินได้ทุกภาษารู้จำได้แบบทัวริง (ตัวตัดสินก็เป็นเครื่องรู้จำชนิดหนึ่ง) ทุกการเป็นสับเซตเป็นแบบแท้ ตัวอย่างภาษาที่ใช้แยกคลาสแสดงไว้ในรูป โดยภาษา A_{TM} และ $\overline{A_{\mathsf{TM}}}$ จะเป็นหัวข้อหลักของส่วนที่ 3.3



รูปที่ 3.2: ความสัมพันธ์แบบซ้อนกันของคลาสภาษา พร้อมตัวอย่างภาษาที่อยู่ในแต่ละชั้นแต่ไม่อยู่ในชั้นที่เล็กกว่า

3.2 ภาษาที่รู้จำได้แบบทัวริง

3.2.1 ทบทวนนิยามและความแตกต่างจากความตัดสินได้

เครื่องจักรทัวริง M บนข้อมูลเข้า w มีผลลัพธ์ได้สามแบบ: ยอมรับ ปฏิเสธ หรือ *วนไม่รู้จบ* (loop) ภาษา L **รู้จำได้แบบทัวริง** ถ้ามีเครื่องจักรทัวริง M ที่ $L(M) = L$ นั่นคือ M ยอมรับทุกสตริงใน L ส่วนสตริงที่ไม่อยู่ใน L นั้น M อาจปฏิเสธหรือวนไม่รู้จบก็ได้ เราเรียก M ว่า **เครื่องรู้จำ** (recognizer) ของ L

ความแตกต่างนี้มีผลในทางปฏิบัติอย่างมาก สมมติเรารันเครื่องรู้จำมาแล้วล้านก้าวแต่ยังไม่หยุด เราไม่มีทางรู้ได้ว่ามัน "กำลังจะยอมรับในอีกไม่กี่ก้าว" หรือ "จะวนไปตลอดกาล" ตัวตัดสินจึงมีค่ามากกว่าเพราะรับประกันคำตอบในเวลาจำกัดเสมอ

นิยาม 3.11. ภาษา L เป็น **ภาษาที่รู้จำได้แบบร่วมทัวริง** (co-Turing-recognizable) ถ้าคอมพลีเมนต์ $\overline{L} = \Sigma^* \setminus L$ เป็นภาษาที่รู้จำได้แบบทัวริง

ในทางสัญชาตญาณ ภาษารู้จำได้คือภาษาที่ “คำตอบใช่” มีหลักฐานที่ตรวจพบได้ในเวลาจำกัด ส่วนภาษารู้จำได้แบบร่วมคือภาษาที่ “คำตอบไม่ใช่” มีหลักฐานที่ตรวจพบได้ในเวลาจำกัด ทฤษฎีบทต่อไปนี้บอกว่า ถ้าทั้งสองด้านมีหลักฐานแบบนี้ ปัญหาก็ตัดสินได้

3.2.2 ความตัดสินได้ = รู้จำได้ทั้งสองด้าน

ทฤษฎีบท 3.12. *ภาษา L ตัดสินได้ ก็ต่อเมื่อ L รู้จำได้แบบทัวริง และ L รู้จำได้แบบร่วมทัวริง (นั่นคือ ทั้ง L และ $\overline{L}$ รู้จำได้แบบทัวริง)*

พิสูจน์. ($\Rightarrow$) ถ้า L ตัดสินได้ด้วยตัวตัดสิน M แล้ว M เป็นเครื่องรู้จำของ L ทันที สำหรับ $\overline{L}$ ให้สร้าง $\overline{M}$ ที่ได้จาก M โดยสลับบทบาทของ q_{accept} กับ q_{reject} เพราะ M หยุดเสมอ $\overline{M}$ ก็หยุดเสมอ และยอมรับ w ก็ต่อเมื่อ M ปฏิเสธ w คือ $w \in \overline{L}$ ดังนั้น $\overline{L}$ ตัดสินได้และย่อมรู้จำได้

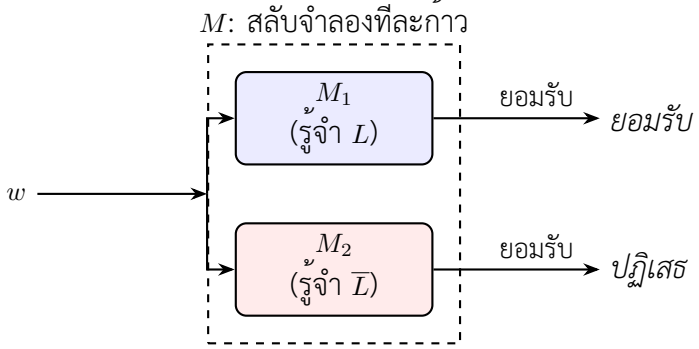
($\Leftarrow$) ให้ M_1 เป็นเครื่องรู้จำของ L และ M_2 เป็นเครื่องรู้จำของ $\overline{L}$ สร้าง

$M =$ “เมื่อได้รับข้อมูลเข้า w

1. รัน M_1 และ M_2 บน w *แบบขนาน* คือสลับกันจำลองทีละก้าว: ก้าวที่ 1 ของ M_1, ก้าวที่ 1 ของ M_2, ก้าวที่ 2 ของ M_1, ก้าวที่ 2 ของ M_2, ...
2. ถ้า M_1 ยอมรับ ให้ *ยอมรับ* ถ้า M_2 ยอมรับ ให้ *ปฏิเสธ*”

(ในทางเทคนิค M อาจใช้เทปสองเส้น เส้นละเครื่อง ซึ่งจำลองด้วยเทปเดียวได้ตามผลในบทก่อน)

สตริง w ทุกตัวอยู่ใน L หรือใน $\overline{L}$ อย่างใดอย่างหนึ่งพอดี ถ้า $w \in L$ แล้ว M_1 ยอมรับ w หลังผ่านไปจำนวนก้าวจำกัด ถ้า $w \in \overline{L}$ แล้ว M_2 ยอมรับ w หลังผ่านไปจำนวนก้าวจำกัด ดังนั้นการจำลองในขั้นที่ 1 ต้องสิ้นสุดเสมอ M จึงเป็นตัวตัดสิน และเนื่องจาก M_1, M_2 ไม่มีทางยอมรับ w ตัวเดียวกันทั้งคู่ M ยอมรับ w ก็ต่อเมื่อ $w \in L$ □



รูปที่ 3.3: ตัวตัดสิน M ที่สร้างจากเครื่องรู้จำของ L และของ $\overline{L}$ เครื่องใดเครื่องหนึ่งต้องยอมรับในที่สุด

หมายเหตุ. ห้ามรัน M_1 ให้จบก่อนแล้วค่อยรัน M_2 เพราะถ้า $w \notin L$ เครื่อง M_1 อาจวนไม่รู้จบ และเราจะไม่มีโอกาสได้รัน M_2 เลย การสลับทีละก้าวคือหัวใจของบทพิสูจน์

บทแทรก 3.13. *ภาษาที่ตัดสินได้ปิดภายใต้คอมพลีเมนต์ แต่ถ้า L รู้จำได้แบบทัวริงแต่ตัดสินไม่ได้ แล้ว $\overline{L}$ จะไม่รู้จำได้แบบทัวริง*

พิสูจน์. ส่วนแรกมาจากการสลับ q_{accept} กับ q_{reject} ในบทพิสูจน์ข้างต้น ส่วนที่สอง ถ้า $\overline{L}$ รู้จำได้ ทฤษฎีบท 3.12 จะบังคับให้ L ตัดสินได้ ซึ่งขัดกับสมมติฐาน □

3.2.3 การจำลองแบบสลับซ้อน (dovetailing)

เทคนิคการรันสองเครื่องสลับกันขยายไปสู่การรันการคำนวณ *จำนวนอนันต์* "พร้อมกัน" ได้ โดยไม่ติดอยู่กับการคำนวณใดที่ไม่จบ เทคนิคนี้เรียกว่า **การสลับซ้อน** (dovetailing) ตามภาพการต่อไม้แบบหางเหยี่ยวที่สอดประสานกันเป็นชั้น ๆ

สมมติว่าเราต้องการรันเครื่อง M บนสตริง $s_1, s_2, s_3, \ldots$ (เรียงสตริงทั้งหมดใน Σ^* ตามลำดับความยาวแล้วตามพจนานุกรม) และยอมรับทันทีที่พบว่า M ยอมรับสตริงใดสตริงหนึ่ง ถ้ารัน M บน s_1 ให้จบก่อน อาจติดอยู่ที่ s_1 ตลอดไป วิธีสลับซ้อนทำดังนี้

$$\text{สำหรับ } i = 1, 2, 3, \ldots \quad \text{รัน } M \text{ บน } s_1, s_2, \ldots, s_i \text{ คนละ } i \text{ ก้าว}$$

ถ้า M ยอมรับ s_j ภายใน t ก้าว เหตุการณ์นี้จะถูกตรวจพบที่รอบ $i = \max(j, t)$ ซึ่งเป็นรอบจำกัด ทั้งที่แต่ละรอบใช้เวลาจำกัดเสมอ หากวาดตารางอนันต์ที่แถวคือสตริงและคอลัมน์คือจำนวนก้าว รอบที่ i จะครอบคลุมสี่เหลี่ยมจัตุรัสขนาด $i \times i$ ที่มุมบนซ้าย ซึ่งขยายจนครอบคลุมทุกช่องในที่สุด

ตัวอย่าง 3.14. ภาษา

$$NE_{\text{TM}} = \{\langle M \rangle \mid M \text{ เป็นเครื่องจักรทัวริงที่ยอมรับสตริงอย่างน้อยหนึ่งสตริง}\}$$

รู้จำได้แบบทัวริง ด้วยเครื่องที่ทำการสลับซ้อนตามที่อธิบายข้างต้นบนข้อมูลเข้า $\langle M \rangle$ ถ้า $L(M) \neq \emptyset$ เครื่องนี้จะพบสตริงที่ M ยอมรับในที่สุดและยอมรับ ถ้า $L(M) = \emptyset$ เครื่องนี้วนไม่รู้จบ ซึ่งเครื่องรู้จำทำได้โดยไม่ผิดนิยาม (ภายหลังจะเห็นว่า NE_{TM} ตัดสินไม่ได้ จึงหลีกเลี่ยงการวนไม่รู้จบนี้ไม่ได้)

หมายเหตุ. ด้วยการสลับซ้อนแบบเดียวกัน เราพิสูจน์ได้ว่าภาษารู้จำได้แบบทัวริงปิดภายใต้ยูเนียน และพิสูจน์ได้ว่าภาษารู้จำได้ก็ต่อเมื่อมีตัวแจกแจง (enumerator) ที่พิมพ์ภาษานั้นออกมา

3.3 ความตัดสินไม่ได้

ถึงตรงนี้ทุกปัญหาที่เราพบล้วนตัดสินได้ อาจชวนให้คิดว่าปัญหาใด ๆ ที่นิยามได้ชัดเจนย่อมมีขั้นตอนวิธีแก้ ส่วนนี้จะแสดงว่าความคิดนั้นผิด ปัญหาที่ตัดสินไม่ได้ปัญหาแรกที่เราจะพิสูจน์คือ ปัญหาว่าเครื่องจักรทัวริงยอมรับข้อมูลเข้าที่กำหนดหรือไม่ ซึ่งเป็นคำถามที่เป็นธรรมชาติมาก เทียบได้กับคำถาม "โปรแกรมนี้ให้ผลลัพธ์ตามที่ต้องการบนข้อมูลนี้หรือไม่" การตัดสินไม่ได้ของปัญหานี้จึงบอกว่า การตรวจสอบความถูกต้องของซอฟต์แวร์แบบอัตโนมัติทั้งหมด เป็นไปไม่ได้ในกรณีทั่วไป

3.3.1 ภาษา A_{TM} และเครื่องจักรทัวริงเอกภพ

$$A_{\text{TM}} = \{\langle M, w \rangle \mid M \text{ เป็นเครื่องจักรทัวริงที่ยอมรับ } w\}.$$

ทฤษฎีบท 3.15. *A_{TM} รู้จำได้แบบทัวริง*

พิสูจน์. สร้างเครื่อง

$U =$ "เมื่อได้รับ $\langle M, w\rangle$ โดยที่ M เป็นเครื่องจักรทัวริงและ w เป็นสตริง

1. จำลอง M บนข้อมูลเข้า w
2. ถ้า M เข้าสู่สถานะยอมรับ ให้ *ยอมรับ* ถ้า M เข้าสู่สถานะปฏิเสธ ให้ *ปฏิเสธ*"

การจำลองทำได้โดยเก็บรูปแบบการจัดเรียง (configuration) ปัจจุบันของ M ไว้บนเทป แล้วในแต่ละก้าวค้นตารางฟังก์ชันเปลี่ยนสถานะใน $\langle M\rangle$ เพื่อปรับรูปแบบการจัดเรียง เห็นได้ว่า U ยอมรับ $\langle M, w\rangle$ ก็ต่อเมื่อ M ยอมรับ w □

เครื่อง U เรียกว่า **เครื่องจักรทัวริงเอกภพ** (universal Turing machine) เพราะจำลองเครื่องจักรทัวริงใด ๆ ได้จากคำบรรยายของเครื่องนั้น ความคิดนี้เป็นต้นแบบของ คอมพิวเตอร์แบบเก็บโปรแกรมไว้ในหน่วยความจำ (stored-program computer): ฮาร์ดแวร์ชุดเดียวรันโปรแกรมใดก็ได้ที่ป้อนเข้ามาเป็นข้อมูล

หมายเหตุ. U เป็นเพียงเครื่องรู้จำ ไม่ใช่ตัวตัดสิน เพราะถ้า M วนไม่รู้จบบน w แล้ว U ก็วนไม่รู้จบไปด้วย ถ้า U ตรวจได้ว่า "M จะไม่มีวันหยุด" เมื่อใด U ก็จะปฏิเสธได้ทันที แต่ทฤษฎีบทที่ 3.22 ข้างหน้าจะแสดงว่าไม่มีเครื่องใดทำเช่นนั้นได้ในทุกกรณี

3.3.2 วิธีแนวทแยง

เครื่องมือหลักในการพิสูจน์ว่า A_{TM} ตัดสินไม่ได้คือ **วิธีแนวทแยง** (diagonalization method) ซึ่งเกออร์ก คันทอร์ (Georg Cantor) คิดค้นขึ้นในปี ค.ศ. 1873 เพื่อเปรียบเทียบ "ขนาด" ของเซตอนันต์ สำหรับเซตจำกัด เราเปรียบเทียบขนาดด้วยการนับ แต่สำหรับเซตอนันต์ การนับไม่มีวันจบ คันทอร์จึงเสนอให้เปรียบเทียบด้วยการ *จับคู่*

นิยาม 3.16. ให้ $f\colon A \to B$ เป็นฟังก์ชัน

- f เป็น **ฟังก์ชันหนึ่งต่อหนึ่ง** (one-to-one, injective) ถ้า $a \neq a'$ แล้ว $f(a) \neq f(a')$
- f เป็น **ฟังก์ชันทั่วถึง** (onto, surjective) ถ้าทุก $b \in B$ มี $a \in A$ ที่ $f(a) = b$
- f เป็น **การสมนัยหนึ่งต่อหนึ่ง** (correspondence, bijection) ถ้า f เป็นทั้งหนึ่งต่อหนึ่งและทั่วถึง

เซต A และ B **มีขนาดเท่ากัน** ถ้ามีการสมนัยหนึ่งต่อหนึ่งจาก A ไป B

การสมนัยหนึ่งต่อหนึ่งจับคู่สมาชิกของ A กับสมาชิกของ B โดยไม่มีใครเหลือและไม่มีใครถูกจับซ้ำ นิยามนี้ให้ผลที่ขัดกับสัญชาตญาณบางอย่าง

ตัวอย่าง 3.17. ให้ $\mathbb{N} = \{1, 2, 3, \dots\}$ และ $\mathcal{T} = \{3, 6, 9, \dots\}$ คือเซตของพหุคูณบวกของ 3 ฟังก์ชัน $f(n) = 3n$ เป็นการสมนัยหนึ่งต่อหนึ่งจาก $\mathbb{N}$ ไป $\mathcal{T}$ ดังนั้น $\mathcal{T}$ มีขนาดเท่ากับ $\mathbb{N}$ ทั้งที่ $\mathcal{T}$ เป็นสับเซตแท้ของ $\mathbb{N}$ และ "ดูเหมือน" จะมีสมาชิกเพียงหนึ่งในสาม

นิยาม 3.18. เซต A เป็น **เซตนับได้** (countable) ถ้า A เป็นเซตจำกัด หรือ A มีขนาดเท่ากับ $\mathbb{N}$ มิฉะนั้นเรียกว่า **เซตนับไม่ได้** (uncountable)

การแสดงว่าเซตอนันต์ A นับได้ เทียบเท่ากับการเรียงสมาชิกทั้งหมดของ A เป็นรายการ $a_1, a_2, a_3, \dots$ ที่สมาชิกทุกตัวปรากฏในตำแหน่งจำกัดบางตำแหน่ง

ตัวอย่าง 3.19. สำหรับอักษรจำกัด Σ เซต Σ^* นับได้ เพราะเรียงสตริงตามความยาวก่อน แล้วเรียงสตริงที่ยาวเท่ากันตามพจนานุกรม เช่น เมื่อ $\Sigma = \{0,1\}$ ได้รายการ $\varepsilon, 0, 1, 00, 01, 10, 11, 000, \ldots$ สตริงแต่ละตัวมีความยาวจำกัด จึงปรากฏในตำแหน่งจำกัดเสมอ

เซตอนันต์ทุกเซตนับได้หรือไม่ คันทอร์ตอบว่าไม่

ทฤษฎีบท 3.20. *เซตจำนวนจริง $\mathbb{R}$ นับไม่ได้*

พิสูจน์. พิสูจน์โดยข้อขัดแย้ง สมมติว่ามีการสมนัยหนึ่งต่อหนึ่ง $f\colon \mathbb{N} \to \mathbb{R}$ เราจะสร้างจำนวนจริง x ที่ไม่เท่ากับ $f(n)$ สำหรับทุก n ซึ่งขัดกับการที่ f ทั่วถึง

เขียนส่วนทศนิยมของ $f(n)$ แต่ละตัวเรียงเป็นแถวของตาราง (ดูรูปที่ 3.4) ให้ d_n เป็นทศนิยมตำแหน่งที่ n ของ $f(n)$ ซึ่งคือตัวเลขบน *แนวทแยง* ของตาราง กำหนดให้ $x = 0.x_1x_2x_3\ldots$ โดย

$$x_n = \begin{cases} d_n + 1 & \text{ถ้า } 1 \le d_n \le 7, \\ 5 & \text{ถ้า } d_n \in \{0, 8, 9\}. \end{cases}$$

จะได้ $x_n \neq d_n$ เสมอ ดังนั้น x ต่างจาก $f(n)$ ที่ทศนิยมตำแหน่งที่ n ทุก n และเนื่องจาก x ไม่มีเลข 0 หรือ 9 ปัญหาการเขียนทศนิยมได้สองแบบ (เช่น $0.1999\ldots = 0.2000\ldots$) จึงไม่เกิดขึ้น สรุปได้ว่า $x \neq f(n)$ ทุก n ซึ่งขัดแย้งกับการที่ f เป็นฟังก์ชันทั่วถึง □

n	$f(n)$	1	2	3	4	5	6	
1	7.	2	7	1	8	2	8	$\cdots$
2	0.	5	0	0	0	0	0	$\cdots$
3	1.	3	3	3	3	3	3	$\cdots$
4	2.	1	4	1	4	2	1	$\cdots$
5	0.	9	0	9	0	9	0	$\cdots$
6	4.	6	2	5	1	7	7	$\cdots$
$\vdots$				$\vdots$				
	$x = 0.$	3	5	4	5	5	8	$\cdots$

รูปที่ 3.4: การสร้างแนวทแยงของคันทอร์ ตัวเลขในกรอบคือ $d_1, \ldots, d_6 = 2, 0, 3, 4, 9, 7$ และ x ถูกสร้างให้ต่างจากแถวที่ n ที่ตำแหน่งที่ n เสมอ

หมายเหตุ. หัวใจของวิธีแนวทแยงคือ: สมมติว่ามีรายการที่ครบถ้วน แล้วสร้างวัตถุใหม่ที่ *ต่างจากวัตถุตัวที่ n ในรายการ ณ จุดที่ n* วัตถุใหม่นี้จึงไม่อยู่ในรายการ ข้อสมมติว่ารายการครบถ้วนจึงผิด แบบแผนเดียวกันนี้จะใช้ซ้ำในการพิสูจน์ว่า A_{TM} ตัดสินไม่ได้

มีภาษาที่ไม่รู้จำได้แบบทัวริง

วิธีแนวทแยงให้ผลทันทีว่ามีภาษาที่ไม่มีเครื่องจักรทัวริงใดรู้จำได้ โดยอาศัยการนับเพียงอย่างเดียว

บทแทรก 3.21. *มีภาษาบางภาษาที่ไม่รู้จำได้แบบทัวริง*

พิสูจน์. ขั้นที่ 1: เซตของเครื่องจักรทัวริงทั้งหมดนับได้ เครื่องจักรทัวริงแต่ละเครื่อง M มีรหัส $\langle M\rangle$ ซึ่งเป็นสตริงจำกัดบนอักษรจำกัด และ Σ^* นับได้ เราจึงไล่สตริงใน Σ^* ตามลำดับและข้ามสตริงที่ไม่ใช่รหัสของเครื่องจักรทัวริง ได้รายการของเครื่องจักรทัวริงทั้งหมด

ขั้นที่ 2: เซตของภาษาทั้งหมดนับไม่ได้ ให้ $\mathcal{B}$ เป็นเซตของลำดับทวิภาคอนันต์ทั้งหมด อาร์กิวเมนต์แนวทแยงแบบเดียวกับทฤษฎีบท 3.20 (พลิกบิตบนแนวทแยง) แสดงว่า $\mathcal{B}$ นับไม่ได้ ต่อไปเรียงสตริงใน Σ^* เป็น $s_1, s_2, s_3, \ldots$ แล้วจับคู่ภาษา $L \subseteq \Sigma^*$ แต่ละภาษากับ **ลำดับลักษณะเฉพาะ** (characteristic sequence) $\chi_L \in \mathcal{B}$ ซึ่งบิตที่ i เป็น 1 ก็ต่อเมื่อ $s_i \in L$ การจับคู่นี้เป็นการสมนัยหนึ่งต่อหนึ่ง เซตของภาษาทั้งหมดจึงมีขนาดเท่ากับ $\mathcal{B}$ และนับไม่ได้

ขั้นที่ 3: เครื่องจักรทัวริงแต่ละเครื่องรู้จำภาษาได้เพียงภาษาเดียว ถ้าทุกภาษารู้จำได้ เราจะได้ฟังก์ชันทั่วถึงจากเซตนับได้ (เครื่องจักร) ไปยังเซตนับไม่ได้ (ภาษา) ซึ่งเป็นไปไม่ได้ ดังนั้นต้องมีภาษาที่ไม่มีเครื่องใดรู้จำ □

บทแทรกนี้บอกว่าภาษาที่ไม่รู้จำได้มี “มากกว่า” ภาษาที่รู้จำได้อย่างมหาศาล แต่ไม่ได้ชี้ให้เห็นภาษาเจาะจงสักภาษาเดียว ในหัวข้อถัดไปเราจะได้ตัวอย่างที่เป็นรูปธรรม

3.3.3 A_{TM} ตัดสินไม่ได้

ทฤษฎีบท 3.22. A_{TM} *เป็นภาษาที่ตัดสินไม่ได้*

แนวคิดการพิสูจน์. สมมติว่ามีตัวตัดสิน H ของ A_{TM} แล้วใช้ H สร้างเครื่อง D ที่ “ทำตรงข้าม” กับสิ่งที่ เครื่องแต่ละเครื่องทำเมื่อได้รับรหัสของตัวเองเป็นข้อมูลเข้า เมื่อป้อนรหัสของ D ให้ D เอง จะเกิดข้อขัดแย้ง □

พิสูจน์. สมมติว่า A_{TM} ตัดสินได้ด้วยตัวตัดสิน H นั่นคือสำหรับทุกเครื่องจักรทัวริง M และสตริง w

$$H(\langle M, w\rangle) = \begin{cases} \text{ยอมรับ} & \text{ถ้า } M \text{ ยอมรับ } w, \\ \text{ปฏิเสธ} & \text{ถ้า } M \text{ ไม่ยอมรับ } w \text{ (ปฏิเสธหรือวนไม่รู้จบ).} \end{cases}$$

ใช้ H เป็นซับรูทีนสร้างเครื่องใหม่

$D =$ “เมื่อได้รับ $\langle M\rangle$ โดยที่ M เป็นเครื่องจักรทัวริง

1. รัน H บนข้อมูลเข้า $\langle M, \langle M\rangle\rangle$
2. ให้ผลตรงข้ามกับ H: ถ้า H ยอมรับ ให้ *ปฏิเสธ* ถ้า H ปฏิเสธ ให้ *ยอมรับ*”

การป้อนรหัสของเครื่องให้เครื่องนั้นเองไม่มีอะไรผิดปกติ เช่นเดียวกับที่คอมไพเลอร์ภาษาหนึ่ง อาจเขียนด้วยภาษานั้นและคอมไพล์ตัวเองได้ เนื่องจาก H หยุดเสมอ D ก็หยุดเสมอ และ

$$D(\langle M\rangle) = \begin{cases} \text{ยอมรับ} & \text{ถ้า } M \text{ ไม่ยอมรับ } \langle M\rangle, \\ \text{ปฏิเสธ} & \text{ถ้า } M \text{ ยอมรับ } \langle M\rangle. \end{cases}$$

ทีนี้ป้อน $\langle D\rangle$ ให้ D โดยแทน M ด้วย D ในสมการข้างบน

$$D(\langle D\rangle) = \begin{cases} \text{ยอมรับ} & \text{ถ้า } D \text{ ไม่ยอมรับ } \langle D\rangle, \\ \text{ปฏิเสธ} & \text{ถ้า } D \text{ ยอมรับ } \langle D\rangle. \end{cases}$$

ไม่ว่า D จะทำอะไรบน $\langle D \rangle$ มันถูกบังคับให้ทำสิ่งตรงข้ามกับที่มันทำ ซึ่งเป็นไปไม่ได้ ดังนั้นทั้ง D และ H ไม่มีอยู่จริง สรุปว่า A_{TM} ตัดสินไม่ได้ □

มองบทพิสูจน์เป็นการสร้างแนวทแยง

เรียงเครื่องจักรทัวริงทั้งหมดเป็นรายการ $M_1, M_2, M_3, \ldots$ (ทำได้เพราะนับได้) แล้วสร้างตารางที่แถว i คอลัมน์ j บอกผลของ H บน $\langle M_i, \langle M_j \rangle \rangle$ ซึ่งเป็น "ยอมรับ" (acc) หรือ "ปฏิเสธ" (rej) เสมอ เพราะ H เป็นตัวตัดสิน เครื่อง D อ่านค่าบนแนวทแยงแล้วกลับค่า ดังนั้นแถวของ D ต่างจากแถวของ M_i ที่คอลัมน์ i ทุก i แต่ D เองก็เป็นเครื่องจักรทัวริง จึงต้องปรากฏในรายการเป็นแถวใดแถวหนึ่ง ช่องที่แถว D ตัดกับคอลัมน์ $\langle D \rangle$ จะต้องมีค่าตรงข้ามกับตัวเอง ดังรูปที่ 3.5

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$	$\langle M_5 \rangle$	$\cdots$	$\langle D \rangle$	$\cdots$
M_1	acc	rej	acc	rej	rej		acc	
M_2	rej	rej	acc	acc	rej		rej	
M_3	acc	acc	acc	rej	acc		rej	
M_4	rej	acc	rej	rej	acc		acc	
M_5	acc	rej	rej	acc	acc		rej	
$\vdots$						$\ddots$		
D	rej	acc	rej	acc	rej		?	
$\vdots$								$\ddots$

รูปที่ 3.5: ผลของ H บน $\langle M_i, \langle M_j \rangle \rangle$ (ค่าในตารางเป็นตัวอย่างสมมติ) แถวของ D (แรเงา) ได้จากการกลับค่าบนแนวทแยง (กรอบแดง) ช่องที่ D พบรหัสของตัวเองจึงไม่มีค่าใดที่สอดคล้องได้

หมายเหตุ. ข้อสังเกตเชิงเทคนิค: ถ้าเราใช้ U แทน H ในการสร้าง D จะไม่เกิดข้อขัดแย้ง เพราะ U อาจวนไม่รู้จบ ช่อง $(D, \langle D \rangle)$ จึงอาจเป็น "วนไม่รู้จบ" ได้โดยไม่ขัดแย้งกับอะไร ข้อขัดแย้งเกิดขึ้นได้ก็เพราะสมมติว่า H *หยุดเสมอ* นี่คือเหตุผลที่ A_{TM} รู้จำได้แต่ตัดสินไม่ได้

3.3.4 ภาษาที่ไม่รู้จำได้แบบทัวริงที่เป็นรูปธรรม

บทแทรก 3.23. *$\overline{A_{\mathsf{TM}}}$ ไม่รู้จำได้แบบทัวริง*

พิสูจน์. จากทฤษฎีบท 3.15 A_{TM} รู้จำได้แบบทัวริง ถ้า $\overline{A_{\mathsf{TM}}}$ รู้จำได้ด้วย ทฤษฎีบท 3.12 จะให้ว่า A_{TM} ตัดสินได้ ซึ่งขัดกับทฤษฎีบท 3.22 □

ในทางสัญชาตญาณ สมาชิกของ $\overline{A_{\mathsf{TM}}}$ (ไม่นับสตริงที่ไม่ใช่รหัสที่ถูกต้อง) คือคู่ $\langle M, w \rangle$ ที่ M ไม่ยอมรับ w ซึ่งรวมถึงกรณีที่ M วนไม่รู้จบ การยืนยันว่า "เครื่องนี้จะไม่มีวันยอมรับ" ต้องอาศัยการมองไปข้างหน้าไม่รู้จบ ไม่มีหลักฐานจำกัดใดที่ตรวจได้เสมอ $\overline{A_{\mathsf{TM}}}$ จึงเป็นตัวอย่างรูปธรรมของภาษาที่บทแทรก 3.21 รับประกันว่ามีอยู่ และเติมเต็มรูปที่ 3.2 ในชั้นนอกสุด

หมายเหตุ. ผลในบทนี้เป็นจุดเริ่มต้นของการพิสูจน์ความตัดสินไม่ได้ของปัญหาอื่นอีกมาก เช่น ปัญหาการหยุด $HALT_{\mathsf{TM}}$ ปัญหาความว่าง E_{TM} และปัญหาความสมมูล EQ_{TM} โดยแทนที่จะสร้างแนวทแยงใหม่ทุกครั้ง เราจะ *ลดทอน* A_{TM} ไปยังปัญหาเหล่านั้น ซึ่งเป็นหัวข้อของบทถัดไป

สรุปบทที่ 3

- **ตัดสินได้:** $A_{\mathsf{DFA}}, A_{\mathsf{NFA}}, A_{\mathsf{REX}}, E_{\mathsf{DFA}}, EQ_{\mathsf{DFA}}$ (จำลอง แปลงรูป หาสถานะที่เข้าถึงได้ ผลต่างสมมาตร) และ A_{CFG} (CNF ใช้ $2n-1$ ขั้น), E_{CFG} (ทำเครื่องหมายจากล่างขึ้นบน)
- EQ_{CFG} **ตัดสินไม่ได้** เพราะ CFL ไม่ปิดภายใต้คอมพลีเมนต์และอินเทอร์เซกชัน วิธีผลต่างสมมาตรจึงใช้ไม่ได้
- ภาษาไม่พึ่งบริบททุกภาษาตัดสินได้: ปรกติ $\subsetneq$ ไม่พึ่งบริบท $\subsetneq$ ตัดสินได้ $\subsetneq$ รู้จำได้แบบทัวริง
- L ตัดสินได้ $\iff$ L และ $\overline{L}$ รู้จำได้ทั้งคู่ (รันสองเครื่องสลับทีละก้าว)
- การสลับซ้อน (dovetailing) ใช้รันการคำนวณจำนวนอนันต์โดยไม่ติดอยู่กับการคำนวณที่ไม่จบ
- Σ^* และเซตของเครื่องจักรทัวริงนับได้ แต่ $\mathbb{R}$ และเซตของภาษาทั้งหมดนับไม่ได้ จึงมีภาษาที่ไม่รู้จำได้
- A_{TM} รู้จำได้ (เครื่องเอกภพ U) แต่ตัดสินไม่ได้ (เครื่อง D ทำตรงข้ามบนแนวทแยง)
- $\overline{A_{\mathsf{TM}}}$ ไม่รู้จำได้แบบทัวริง

บทที่ 4

การลดรูปปัญหา (Reducibility)

ในบทก่อนเราได้พิสูจน์ว่า A_{TM} เป็นปัญหาที่ตัดสินไม่ได้ (undecidable) ด้วยวิธีแนวทแยง (diagonalization) ซึ่งเป็นวิธีที่ทรงพลังแต่ต้องออกแบบใหม่ทุกครั้ง ในบทนี้เราจะได้เครื่องมือที่ใช้ซ้ำได้สะดวกกว่ามาก นั่นคือ **การลดรูป** (reduction) ซึ่งเป็นการแปลงปัญหาหนึ่งให้กลายเป็นอีกปัญหาหนึ่ง เมื่อรู้ว่าปัญหาตั้งต้นยาก ก็จะสรุปได้ทันทีว่าปัญหาปลายทางยากด้วย แนวคิดเดียวกันนี้จะกลับมามีบทบาทสำคัญอีกครั้งในบทถัดไปเรื่องความซับซ้อน

4.1 ปัญหาที่ตัดสินไม่ได้ผ่านการลดรูป

4.1.1 แนวคิดของการลดรูป

กล่าวอย่างไม่เป็นทางการ ปัญหา A **ลดรูปได้** (reducible) ไปยังปัญหา B ถ้าเราสามารถใช้ "วิธีแก้ปัญหา B" เป็นชิ้นส่วนสำเร็จรูปในการสร้าง "วิธีแก้ปัญหา A" ได้ ตัวอย่างในชีวิตประจำวัน: ปัญหา "หาทางจากหอพักไปสนามบิน" ลดรูปได้ไปยังปัญหา "อ่านแผนที่เส้นทางรถไฟฟ้า" เพราะถ้าอ่านแผนที่เป็น ก็หาทางไปสนามบินได้

สิ่งที่ต้องจำให้แม่นคือ *ทิศทาง* ของข้อสรุป ถ้า A ลดรูปไปยัง B แล้ว

- ถ้า B ตัดสินได้ แล้ว A ตัดสินได้ (ใช้ตัวตัดสินของ B ประกอบเป็นตัวตัดสินของ A)
- ดังนั้นโดยการแย้งสลับที่ (contrapositive) ถ้า A ตัดสินไม่ได้ แล้ว B ก็ตัดสินไม่ได้

ในบทนี้เราจะใช้ข้อที่สองเป็นหลัก รูปแบบการพิสูจน์มาตรฐานคือ *สมมติว่า B มีตัวตัดสิน R แล้วใช้ R สร้างตัวตัดสินของ A_{TM} ซึ่งขัดแย้งกับสิ่งที่รู้แล้ว*

หมายเหตุ. ความผิดพลาดที่พบบ่อยที่สุดของผู้เริ่มเรียนคือลดรูปผิดทิศ เช่น แปลงปัญหาใหม่ B ให้เป็น A_{TM} ซึ่งไม่ได้พิสูจน์อะไรเลย เพราะการที่ B "ง่ายกว่าหรือเท่ากับ" ปัญหาที่ยาก ไม่ได้บอกว่า B ยาก ต้องแปลงจาก *ปัญหาที่รู้ว่ายาก* ไปเป็น *ปัญหาที่ต้องการพิสูจน์ว่ายาก* เสมอ

4.1.2 ปัญหาการหยุดทำงาน

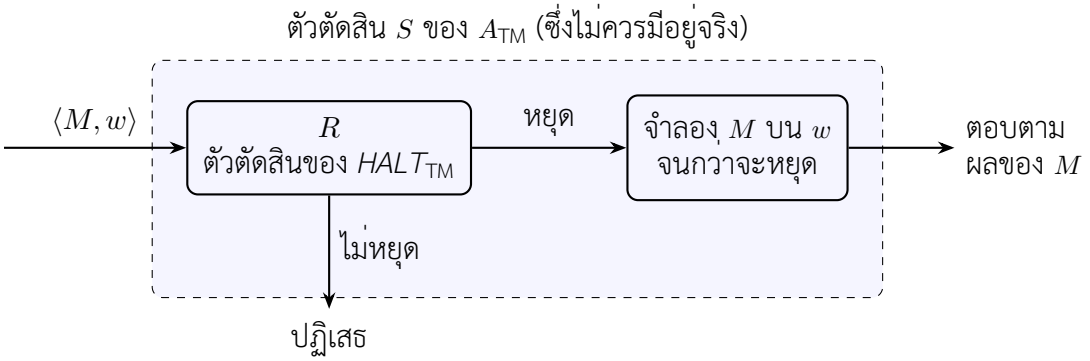
กำหนด

$$\mathit{HALT}_{\mathsf{TM}} = \{\langle M, w\rangle \mid M \text{ เป็นเครื่องจักรทัวริงที่หยุดทำงานบนอินพุต } w\}.$$

สังเกตว่าต่างจาก A_{TM} ตรงที่ไม่สนใจว่า M ยอมรับหรือปฏิเสธ ขอเพียงหยุด

ทฤษฎีบท 4.1. *$HALT_{TM}$ ตัดสินไม่ได้*

แนวคิดการพิสูจน์. ปัญหาเดียวที่ทำให้ A_{TM} ยากคือเครื่อง M อาจวนไม่รู้จบบน w ทำให้การจำลองตรง ๆ ไม่ให้คำตอบ ถ้ามีตัวตัดสินของ $HALT_{TM}$ เราก็ถามมันก่อนได้ว่า M จะหยุดบน w หรือไม่ ถ้าไม่หยุดก็ตอบปฏิเสธได้ทันที ถ้าหยุดก็จำลองได้อย่างปลอดภัย (ดูรูปที่ 4.1) □



รูปที่ 4.1: การใช้ตัวตัดสินสมมติ R ของ $HALT_{TM}$ สร้างตัวตัดสิน S ของ A_{TM}

พิสูจน์. สมมติเพื่อหาข้อขัดแย้งว่ามีเครื่องจักรทัวริง R ที่ตัดสิน $HALT_{TM}$ สร้างเครื่อง S ดังนี้

S = "เมื่อได้รับอินพุต $\langle M, w \rangle$:

1. รัน R บน $\langle M, w \rangle$
2. ถ้า R ปฏิเสธ (แปลว่า M ไม่หยุดบน w) ให้*ปฏิเสธ*
3. ถ้า R ยอมรับ ให้จำลอง M บน w จนกว่า M จะหยุด (รับประกันว่าหยุดแน่นอน)
4. ถ้า M ยอมรับ ให้*ยอมรับ* ถ้า M ปฏิเสธ ให้*ปฏิเสธ*"

ทุกขั้นตอนของ S ใช้เวลาจำกัด ดังนั้น S หยุดเสมอ และยอมรับ $\langle M, w \rangle$ ก็ต่อเมื่อ M ยอมรับ w นั่นคือ S ตัดสิน A_{TM} ได้ ขัดแย้งกับการที่ A_{TM} ตัดสินไม่ได้ ดังนั้น R ไม่มีอยู่จริง □

4.1.3 ปัญหาความว่างของภาษา

กำหนด $E_{TM} = \{\langle M \rangle \mid M \text{ เป็นเครื่องจักรทัวริงที่ } L(M) = \emptyset\}$

ทฤษฎีบท 4.2. *E_{TM} ตัดสินไม่ได้*

แนวคิดการพิสูจน์. ตัวตัดสินของ E_{TM} รับอินพุตเป็นเครื่องเพียงเครื่องเดียว ไม่มีสตริง w ให้ถาม เคล็ดลับคือ "ฝัง" w ลงไปในเครื่องใหม่ M_1 ที่ออกแบบให้ภาษาของมันว่างหรือไม่ว่าง ขึ้นอยู่กับว่า M ยอมรับ w หรือไม่ ตัวตัดสินของ A_{TM} ไม่ต้อง*รัน* M_1 เลย เพียงแค่*เขียนคำอธิบาย* $\langle M_1 \rangle$ ขึ้นมาแล้วส่งให้ตัวตัดสินของ E_{TM} □

พิสูจน์. สมมติว่า R ตัดสิน E_{TM} ได้ สำหรับเครื่อง M และสตริง w ใด ๆ นิยามเครื่อง

M_1 = "เมื่อได้รับอินพุต x:

1. ถ้า $x \neq w$ ให้*ปฏิเสธ*
2. ถ้า $x = w$ ให้รัน M บน w และ*ยอมรับ*ถ้า M ยอมรับ"

จะได้ว่า $L(M_1) = \{w\}$ ถ้า M ยอมรับ w และ $L(M_1) = \emptyset$ ในกรณีอื่น ต่อไปสร้าง

S = "เมื่อได้รับอินพุต $\langle M, w\rangle$:

1. เขียนคำอธิบาย $\langle M_1\rangle$ จาก M และ w ตามแบบข้างต้น
2. รัน R บน $\langle M_1\rangle$
3. ถ้า R ยอมรับ (ภาษาว่าง) ให้*ปฏิเสธ* ถ้า R ปฏิเสธ ให้*ยอมรับ*"

S ตัดสิน A_{TM} ได้ ซึ่งเป็นไปไม่ได้ ดังนั้น E_{TM} ตัดสินไม่ได้ □

ขั้นตอนที่ 1 ของ S ทำได้จริงเพราะการสร้าง $\langle M_1\rangle$ เป็นเพียงการ "ต่อโค้ด": เอาโค้ดเปรียบเทียบสตริงกับค่าคงที่ w มาต่อหน้าโค้ดของ M ซึ่งเครื่องจักรทัวริงทำได้เสมอ

4.1.4 ปัญหาความปรกติของภาษา

กำหนด $\mathit{REGULAR}_{\mathsf{TM}} = \{\langle M\rangle \mid L(M)$ เป็นภาษาปรกติ$\}$

ทฤษฎีบท 4.3. *$\mathit{REGULAR}_{\mathsf{TM}}$ ตัดสินไม่ได้*

พิสูจน์. สมมติว่า R ตัดสิน $\mathit{REGULAR}_{\mathsf{TM}}$ จาก $\langle M, w\rangle$ สร้างเครื่อง

M_2 = "เมื่อได้รับอินพุต x:

1. ถ้า x อยู่ในรูป 0^n1^n สำหรับบาง $n \geq 0$ ให้*ยอมรับ*
2. มิฉะนั้น รัน M บน w และ*ยอมรับ*ถ้า M ยอมรับ"

ถ้า M ยอมรับ w เครื่อง M_2 ยอมรับทุกสตริง จึงได้ $L(M_2) = \Sigma^*$ ซึ่งเป็นภาษาปรกติ ถ้า M ไม่ยอมรับ w จะได้ $L(M_2) = \{0^n1^n \mid n \geq 0\}$ ซึ่งไม่ใช่ภาษาปรกติ ดังนั้นตัวตัดสินของ A_{TM} คือ "สร้าง $\langle M_2\rangle$ แล้วตอบตาม R" ซึ่งเป็นไปไม่ได้ □

แนวคิดเดียวกันนี้ใช้ได้กับคุณสมบัติอื่นแทบทุกอย่าง เช่น "$L(M)$ ไม่พึ่งบริบท" หรือ "$L(M)$ จำกัด" เพียงเลือกภาษาสองภาษาที่ฝั่งหนึ่งมีคุณสมบัติและอีกฝั่งไม่มี แล้วให้ M_2 สลับระหว่างสองภาษานั้นตามพฤติกรรมของ M บน w

4.1.5 ปัญหาความเท่ากันของเครื่องจักรทัวริง

กำหนด $\mathit{EQ}_{\mathsf{TM}} = \{\langle M_1, M_2\rangle \mid M_1, M_2$ เป็นเครื่องจักรทัวริงที่ $L(M_1) = L(M_2)\}$

ทฤษฎีบท 4.4. *$\mathit{EQ}_{\mathsf{TM}}$ ตัดสินไม่ได้*

พิสูจน์. คราวนี้ลดรูปจาก E_{TM} แทน A_{TM} (ใช้ปัญหาที่ตัดสินไม่ได้ข้อใดเป็นจุดตั้งต้นก็ได้) สังเกตว่า E_{TM} เป็น "กรณีพิเศษ" ของ $\mathit{EQ}_{\mathsf{TM}}$ เมื่อเครื่องตัวที่สองมีภาษาว่าง สมมติว่า R ตัดสิน $\mathit{EQ}_{\mathsf{TM}}$ ให้ $M_\emptyset$ เป็นเครื่องที่ปฏิเสธทุกอินพุตทันที แล้วสร้าง S = "เมื่อได้รับ $\langle M\rangle$: รัน R บน $\langle M, M_\emptyset\rangle$ แล้วตอบตาม R" จะได้ว่า S ตัดสิน E_{TM} ซึ่งขัดแย้งกับทฤษฎีบทก่อนหน้า □

4.1.6 ทฤษฎีบทของไรซ์

ตัวอย่างทั้งหมดข้างต้นเป็นกรณีเฉพาะของผลลัพธ์ทั่วไปข้อหนึ่ง

ทฤษฎีบท 4.5 (ไรซ์ (Rice's theorem))**.** *ให้ P เป็นคุณสมบัติของภาษาที่เครื่องจักรทัวริงรู้จำ ซึ่ง (1) ไม่ใช่คุณสมบัติชัด (nontrivial) คือมีเครื่องบางเครื่องที่ภาษามีคุณสมบัติ P และบางเครื่องที่ไม่มี และ (2) ขึ้นกับภาษาเท่านั้น คือถ้า $L(M_1) = L(M_2)$ แล้ว $\langle M_1 \rangle$ และ $\langle M_2 \rangle$ มีคุณสมบัติตรงกัน แล้วภาษา $\{\langle M \rangle \mid \langle M \rangle$ มีคุณสมบัติ $P\}$ ตัดสินไม่ได้*

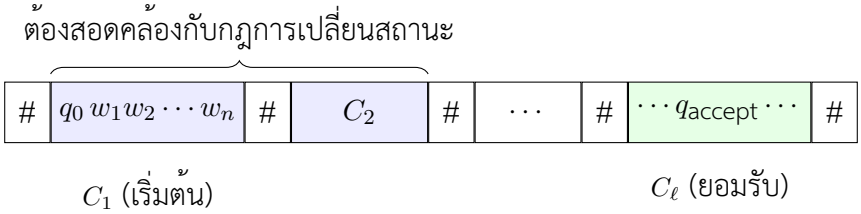
ตัวอย่างการใช้: ปัญหา "$L(M)$ มีสตริง 2026 หรือไม่" "$L(M)$ มีสมาชิกเป็นจำนวนจำกัดหรือไม่" หรือ "$L(M)$ ปิดภายใต้การกลับสตริงหรือไม่" ล้วนตัดสินไม่ได้ทั้งสิ้น แต่ต้องระวังว่าทฤษฎีบทนี้พูดถึง*ภาษา*ของเครื่อง ไม่ใช่ตัวเครื่อง เช่น "M มีสถานะไม่เกิน 10 สถานะ" ตัดสินได้ง่าย ๆ ด้วยการอ่านคำอธิบาย $\langle M \rangle$ และ "M เคยเลื่อนหัวอ่านไปทางซ้ายบนอินพุต w หรือไม่" ก็ไม่เข้าเงื่อนไขของทฤษฎีบทไรซ์

4.2 การลดรูปผ่านประวัติการคำนวณ

สำหรับปัญหาที่ไม่เกี่ยวกับเครื่องจักรทัวริงโดยตรง (เช่น ปัญหาเกี่ยวกับไวยากรณ์) เราต้องการเทคนิคที่ "แปลง" การทำงานของเครื่องจักรทัวริงให้กลายเป็นวัตถุอื่น เครื่องมือหลักคือ **ประวัติการคำนวณ** (computation history)

นิยาม 4.6. ให้ M เป็นเครื่องจักรทัวริงและ w เป็นสตริง **ประวัติการคำนวณที่ยอมรับ** (accepting computation history) ของ M บน w คือลำดับของการจัดรูปแบบ (configuration) $C_1, C_2, \ldots, C_\ell$ โดยที่ C_1 เป็นการจัดรูปแบบเริ่มต้นของ M บน w C_ℓ เป็นการจัดรูปแบบที่ยอมรับ และ C_{i+1} ได้จาก C_i ด้วยการเคลื่อนที่หนึ่งก้าวตามกฎของ M สำหรับทุก i

ประวัติการคำนวณเขียนเป็นสตริงเดียวได้ โดยคั่นแต่ละการจัดรูปแบบด้วย # ดังรูปที่ 4.2 ถ้า M เป็นเครื่องเชิงกำหนด (deterministic) และยอมรับ w ประวัตินี้มีเพียงหนึ่งเดียว ถ้า M ไม่ยอมรับ w ก็ไม่มีประวัติการคำนวณที่ยอมรับเลย



รูปที่ 4.2: ประวัติการคำนวณที่ยอมรับ เขียนเป็นสตริงเดียวคั่นด้วย #

4.2.1 ออโตมาตาขอบเขตเชิงเส้น

นิยาม 4.7. ออโตมาตาขอบเขตเชิงเส้น (linear bounded automaton, LBA) คือเครื่องจักรทัวริงที่หัวอ่านไม่สามารถเคลื่อนออกนอกส่วนของเทปที่อินพุตอยู่ได้ (ถ้าพยายามเลื่อนออกนอกขอบ หัวอ่านจะอยู่ที่เดิม) กล่าวคือหน่วยความจำมีขนาดเท่ากับความยาวอินพุต

แม้จะมีหน่วยความจำจำกัด แต่ LBA ก็มีพลังมาก ตัวตัดสินของภาษาไม่พึ่งบริบท เช่น อัลกอริทึมสำหรับ A_{CFG} สามารถทำงานเป็น LBA ได้ กำหนด $A_{\text{LBA}} = \{\langle M, w \rangle \mid M$ เป็น LBA ที่ยอมรับ $w\}$

บทตั้ง 4.8. *ถ้า LBA M มี q สถานะ และอักษรเทป g ตัว บนเทปยาว n ช่องจะมีการจัดรูปแบบที่ต่างกันไม่เกิน $q\,n\,g^n$ แบบ*

พิสูจน์. การจัดรูปแบบหนึ่งกำหนดด้วยสถานะ (q ทาง) ตำแหน่งหัวอ่าน (n ทาง) และเนื้อหาบนเทป (g^n ทาง) □

ทฤษฎีบท 4.9. *A_{LBA} ตัดสินได้*

แนวคิดการพิสูจน์. จำลอง M บน w ไม่เกิน $q\,n\,g^n$ ก้าว ถ้า M ยังไม่หยุดภายในจำนวนก้าวนี้ ตามหลักรังนกพิราบ (pigeonhole principle) ต้องมีการจัดรูปแบบซ้ำ ซึ่งหมายความว่า M วนซ้ำตลอดไปแน่นอน จึงปฏิเสธได้ □

น่าสนใจว่าปัญหาความว่างกลับตัดสินไม่ได้

ทฤษฎีบท 4.10. *$E_{\mathsf{LBA}} = \{\langle M\rangle \mid M$ เป็น LBA ที่ $L(M) = \emptyset\}$ ตัดสินไม่ได้*

แนวคิดการพิสูจน์. ลดรูปจาก A_{TM} จาก $\langle M, w\rangle$ สร้าง LBA B ที่ยอมรับสตริง x ก็ต่อเมื่อ x เป็นประวัติการคำนวณที่ยอมรับของ M บน w B ตรวจได้ภายในพื้นที่ของ x เอง: ตรวจว่า C_1 เป็นการจัดรูปแบบเริ่มต้นของ M บน w ตรวจว่า C_ℓ มี q_{accept} และตรวจว่า C_i กับ C_{i+1} แต่ละคู่สอดคล้องกับกฎของ M โดยเดินหัวอ่านกลับไปกลับมาระหว่างตำแหน่งที่ตรงกันในสองการจัดรูปแบบและทำเครื่องหมายไว้บนเทป จะได้ $L(B) \neq \emptyset$ ก็ต่อเมื่อ M ยอมรับ w ดังนั้นตัวตัดสินของ E_{LBA} จะให้ตัวตัดสินของ A_{TM} □

4.2.2 ปัญหาเกี่ยวกับไวยากรณ์ไม่พึ่งบริบท

ทฤษฎีบท 4.11. *$ALL_{\mathsf{CFG}} = \{\langle G\rangle \mid G$ เป็นไวยากรณ์ไม่พึ่งบริบทที่ $L(G) = \Sigma^*\}$ ตัดสินไม่ได้*

แนวคิดการพิสูจน์. ลดรูปจาก A_{TM} อีกครั้ง จาก $\langle M, w\rangle$ สร้างไวยากรณ์ G (หรือ PDA ที่เทียบเท่า) ที่สร้าง*ทุกสตริงที่ไม่ใช่* ประวัติการคำนวณที่ยอมรับของ M บน w สตริงจะ “ไม่ใช่ประวัติ” เมื่อผิดเงื่อนไขข้อใดข้อหนึ่ง: เริ่มต้นผิด จบผิด หรือมีคู่ C_i, C_{i+1} ที่ไม่สอดคล้องกัน PDA เลือกแบบไม่เชิงกำหนดว่าจะตรวจข้อผิดพลาดแบบใด สำหรับข้อสุดท้าย PDA ดัน C_i ลงสแตกแล้วเทียบกับ C_{i+1} ซึ่งจะทำได้เมื่อเขียนประวัติโดย*กลับลำดับอักษร*ของการจัดรูปแบบที่อยู่ตำแหน่งคู่ (เพราะสแตกให้ข้อมูลออกมากลับด้าน) ผลคือ $L(G) = \Sigma^*$ ก็ต่อเมื่อ M ไม่ยอมรับ w □

ผลที่ตามมาคือปัญหา “ไวยากรณ์ไม่พึ่งบริบทสองชุดสร้างภาษาเดียวกันหรือไม่” ก็ตัดสินไม่ได้เช่นกัน ต่างจากกรณีของออโตมาตาจำกัดที่ปัญหาเดียวกันตัดสินได้

4.3 การลดรูปแบบส่งแผนที่

การลดรูปที่ใช้มาจนถึงตอนนี้ยังไม่เป็นทางการนัก เพราะตัวตัดสินที่สร้างขึ้นสามารถใช้คำตอบของ R อย่างไรก็ได้ (เช่นกลับคำตอบ) ในส่วนนี้เราทำให้แนวคิดแม่นยำขึ้นด้วย **การลดรูปแบบส่งแผนที่** (mapping reducibility) ซึ่งยังช่วยให้พิสูจน์ได้ว่าบางภาษา*รู้จำไม่ได้* (not Turing-recognizable) ด้วย

4.3.1 ฟังก์ชันที่คำนวณได้

นิยาม 4.12. ฟังก์ชัน $f : \Sigma^* \to \Sigma^*$ เป็น **ฟังก์ชันที่คำนวณได้** (computable function) ถ้ามีเครื่องจักรทัวริง F ที่เมื่อเริ่มด้วยอินพุต w แล้วหยุดทำงานโดยเหลือเพียง $f(w)$ บนเทปเสมอ

ตัวอย่าง 4.13. ฟังก์ชันเลขคณิตทั่วไปคำนวณได้ เช่น $f(\langle m, n\rangle) = \langle m \cdot n\rangle$ ที่สำคัญกว่าสำหรับเราคือการแปลงคำอธิบายเครื่อง เช่น ฟังก์ชันที่รับ $\langle M, w\rangle$ แล้วคืน $\langle M_1\rangle$ ตามการสร้างในหัวข้อ E_{TM} ก็คำนวณได้ เพราะเป็นการต่อและแก้ไขข้อความอย่างเป็นระบบ

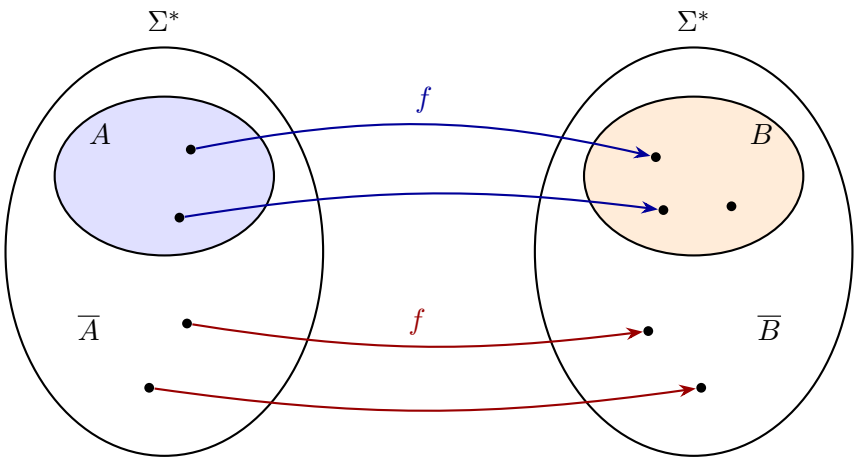
4.3.2 นิยามและสมบัติ

นิยาม 4.14. ภาษา A **ลดรูปแบบส่งแผนที่ได้** ไปยังภาษา B เขียนว่า $A \leq_{\mathsf{m}} B$ ถ้ามีฟังก์ชันที่คำนวณได้ $f : \Sigma^* \to \Sigma^*$ ซึ่งสำหรับทุก w

$$w \in A \iff f(w) \in B.$$

เรียก f ว่า **การลดรูป** (reduction) จาก A ไปยัง B

ภาพในใจคือ f ส่งสมาชิกของ A ทุกตัวเข้าไปใน B และส่งสิ่งที่อยู่นอก A ทุกตัวออกไปนอก B (รูปที่ 4.3) f ไม่จำเป็นต้องเป็นฟังก์ชันหนึ่งต่อหนึ่งหรือทั่วถึง



รูปที่ 4.3: การลดรูปแบบส่งแผนที่ f ส่ง A เข้าไปใน B และส่ง $\overline{A}$ เข้าไปใน $\overline{B}$ (จุดใน B บางจุดอาจไม่มีใครส่งมาถึงก็ได้)

ทฤษฎีบท 4.15. *ถ้า $A \leq_{\mathsf{m}} B$ และ B ตัดสินได้ แล้ว A ตัดสินได้*

พิสูจน์. ให้ M_B ตัดสิน B และ f เป็นการลดรูปจาก A ไปยัง B ตัวตัดสินของ A คือ “เมื่อได้รับ w: คำนวณ $f(w)$ แล้วรัน M_B บน $f(w)$ และตอบตาม M_B” ซึ่งหยุดเสมอและตอบถูกเพราะ $w \in A \iff f(w) \in B$ □

บทแทรก 4.16. *ถ้า $A \leq_{\mathsf{m}} B$ และ A ตัดสินไม่ได้ แล้ว B ตัดสินไม่ได้*

ทฤษฎีบท 4.17. *ถ้า $A \leq_{\mathsf{m}} B$ และ B รู้จำได้ (Turing-recognizable) แล้ว A รู้จำได้ ดังนั้นถ้า $A \leq_{\mathsf{m}} B$ และ A รู้จำไม่ได้ แล้ว B รู้จำไม่ได้*

พิสูจน์. เหมือนทฤษฎีบท 4.15 เพียงเปลี่ยน M_B เป็นตัวรู้จำ ซึ่งอาจไม่หยุดบนสตริงนอก B □

ข้อสังเกตที่มีประโยชน์: f เดียวกันเป็นการลดรูปจาก $\overline{A}$ ไปยัง $\overline{B}$ ด้วย ดังนั้น $A \leq_{\mathsf{m}} B$ ก็ต่อเมื่อ $\overline{A} \leq_{\mathsf{m}} \overline{B}$

ตัวอย่าง 4.18. การพิสูจน์ว่า $HALT_{\mathsf{TM}}$ ตัดสินไม่ได้เขียนเป็นการลดรูปแบบส่งแผนที่ $A_{\mathsf{TM}} \leq_{\mathsf{m}} HALT_{\mathsf{TM}}$ ได้ดังนี้ ให้ $f(\langle M, w\rangle) = \langle M', w\rangle$ เมื่อ

M' = "เมื่อได้รับ x: รัน M บน x ถ้า M ยอมรับ ให้*ยอมรับ* ถ้า M ปฏิเสธ ให้*วนซ้ำไม่รู้จบ*"

M' หยุดบน w ก็ต่อเมื่อ M ยอมรับ w จึงได้ $\langle M, w\rangle \in A_{\mathsf{TM}} \iff \langle M', w\rangle \in HALT_{\mathsf{TM}}$ (สำหรับสตริงที่ไม่ใช่รหัสที่ถูกรูปแบบ ให้ f ส่งไปยังสตริงคงที่ที่อยู่นอก $HALT_{\mathsf{TM}}$)

หมายเหตุ. การพิสูจน์ว่า E_{TM} ตัดสินไม่ได้ในหัวข้อก่อนต้อง*กลับคำตอบ* ในภาษาของการส่งแผนที่จึงเป็น $A_{\mathsf{TM}} \leq_{\mathsf{m}} \overline{E_{\mathsf{TM}}}$ ไม่ใช่ $A_{\mathsf{TM}} \leq_{\mathsf{m}} E_{\mathsf{TM}}$ และอันที่จริง $A_{\mathsf{TM}} \leq_{\mathsf{m}} E_{\mathsf{TM}}$ เป็นไปไม่ได้ เพราะ $\overline{E_{\mathsf{TM}}}$ รู้จำได้ ถ้า $A_{\mathsf{TM}} \leq_{\mathsf{m}} E_{\mathsf{TM}}$ จะได้ $\overline{A_{\mathsf{TM}}} \leq_{\mathsf{m}} \overline{E_{\mathsf{TM}}}$ ทำให้ $\overline{A_{\mathsf{TM}}}$ รู้จำได้ ซึ่งไม่จริง

4.3.3 ภาษาที่ทั้งรู้จำไม่ได้และส่วนเติมเต็มก็รู้จำไม่ได้

จากบทที่แล้ว A_{TM} รู้จำได้แต่ $\overline{A_{\mathsf{TM}}}$ รู้จำไม่ได้ คำถามคือมีภาษาที่ "ยากทั้งสองฝั่ง" หรือไม่ คำตอบคือมี และ EQ_{TM} เป็นตัวอย่าง

ทฤษฎีบท 4.19. *EQ_{TM} ไม่ใช่ภาษาที่รู้จำได้ และไม่ใช่ภาษาที่ส่วนเติมเต็มรู้จำได้ (co-Turing-recognizable)*

แนวคิดการพิสูจน์. เราจะแสดงสองการลดรูปจาก A_{TM}

1. $A_{\mathsf{TM}} \leq_{\mathsf{m}} \overline{EQ_{\mathsf{TM}}}$: ส่ง $\langle M, w\rangle$ ไปเป็น $\langle M_1, M_2\rangle$ โดย M_1 ปฏิเสธทุกอย่าง และ M_2 = "เมื่อได้รับ x ใด ๆ: รัน M บน w ถ้ายอมรับ ให้*ยอมรับ*" ถ้า M ยอมรับ w แล้ว $L(M_2) = \Sigma^* \neq \emptyset = L(M_1)$ ถ้าไม่ยอมรับ ทั้งสองภาษาว่างเท่ากัน จึงได้ $\overline{A_{\mathsf{TM}}} \leq_{\mathsf{m}} EQ_{\mathsf{TM}}$ ดังนั้น EQ_{TM} รู้จำไม่ได้
2. $A_{\mathsf{TM}} \leq_{\mathsf{m}} EQ_{\mathsf{TM}}$: ใช้ M_2 เดิม แต่ให้ M_1 *ยอมรับ*ทุกอย่าง ตอนนี้ $L(M_1) = L(M_2)$ ก็ต่อเมื่อ M ยอมรับ w จึงได้ $\overline{A_{\mathsf{TM}}} \leq_{\mathsf{m}} \overline{EQ_{\mathsf{TM}}}$ ดังนั้น $\overline{EQ_{\mathsf{TM}}}$ รู้จำไม่ได้ □

4.4 ปัญหาการจับคู่ของโพสต์

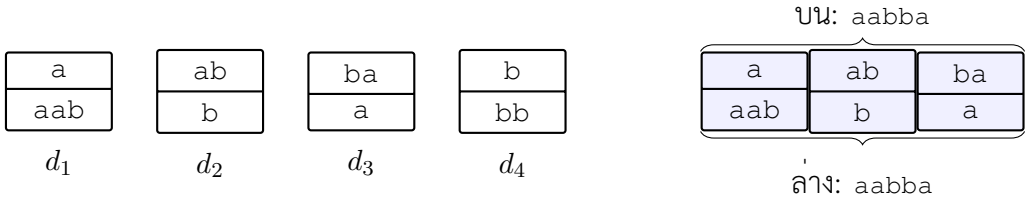
ปิดท้ายบทนี้ด้วยปัญหาที่ตัดสินไม่ได้ซึ่งดูเหมือนเกมปริศนาธรรมดา ไม่เกี่ยวกับเครื่องจักรเลย

นิยาม 4.20. อินสแตนซ์ของ **ปัญหาการจับคู่ของโพสต์** (Post Correspondence Problem, PCP) คือเซตจำกัดของ "แผ่นโดมิโน" แต่ละแผ่นมีสตริงด้านบน t_i และสตริงด้านล่าง b_i **การจับคู่** (match) คือลำดับดัชนี $i_1, i_2, \ldots, i_k$ ($k \geq 1$ ใช้แผ่นซ้ำได้) ซึ่ง

$$t_{i_1} t_{i_2} \cdots t_{i_k} = b_{i_1} b_{i_2} \cdots b_{i_k}.$$

ภาษา *PCP* คือเซตของอินสแตนซ์ที่มีการจับคู่

ตัวอย่าง 4.21. พิจารณาโดมิโนสี่แผ่นในรูปที่ 4.4 ลองเริ่มด้วยแผ่น d_1: ด้านบนได้ a ด้านล่างได้ aab ด้านบนต้องไล่ให้ทันด้วย ab จึงเลือก d_2 ได้บน aab ล่าง aabb แล้วเลือก d_3 ได้บน aabba ล่าง aabba ซึ่งตรงกัน ดังนั้น $d_1d_2d_3$ เป็นการจับคู่ สังเกตว่าแผ่นแรกของการจับคู่ใด ๆ ต้องมีด้านบนและด้านล่างขึ้นต้นเหมือนกัน ในตัวอย่างนี้จึงเริ่มได้เฉพาะจาก d_1 หรือ d_4



รูปที่ 4.4: อินสแตนซ์ PCP ขนาดเล็ก (ซ้าย) และการจับคู่ $d_1d_2d_3$ (ขวา)

ทฤษฎีบท 4.22. *PCP ตัดสินไม่ได้*

บทพิสูจน์เต็มยาวพอสมควร แนวคิดคือใช้ประวัติการคำนวณอีกครั้ง: จาก $\langle M, w\rangle$ สร้างชุดโดมิโนที่บังคับให้ การจับคู่ใด ๆ ต้องเขียนประวัติการคำนวณที่ยอมรับของ M บน w ออกมา โดยด้านล่าง “นำหน้า” ด้านบนไปหนึ่งการจัดรูปแบบเสมอ และด้านบนไล่ตามทันได้ก็ต่อเมื่อไปถึงสถานะยอมรับ PCP มีประโยชน์มากในฐานะจุดตั้งต้นของการลดรูป เช่น ใช้พิสูจน์ว่าปัญหาความกำกวมของไวยากรณ์ไม่พึ่งบริบทตัดสินไม่ได้

สรุปบทที่ 4

- ถ้า A ลดรูปไปยัง B: B ง่าย $\Rightarrow$ A ง่าย และ A ยาก $\Rightarrow$ B ยาก ต้องลดรูป*จาก*ปัญหาที่รู้ว่ายาก
- $HALT_{\mathsf{TM}}$, E_{TM}, $REGULAR_{\mathsf{TM}}$, EQ_{TM} ตัดสินไม่ได้ ทฤษฎีบทของไรซ์: คุณสมบัติไม่ชัดของภาษาของเครื่องจักรทัวริงตัดสินไม่ได้ทั้งหมด
- ประวัติการคำนวณใช้พิสูจน์ว่า E_{LBA}, ALL_{CFG} และ PCP ตัดสินไม่ได้ ขณะที่ A_{LBA} ตัดสินได้
- $A \leq_{\mathsf{m}} B$: มี f คำนวณได้ ที่ $w \in A \iff f(w) \in B$ รักษาทั้งความตัดสินได้และความรู้จำได้
- EQ_{TM} รู้จำไม่ได้ และส่วนเติมเต็มก็รู้จำไม่ได้

บทที่ 5

ทฤษฎีความซับซ้อนเบื้องต้น (Basic Complexity Theory)

ทฤษฎีความตัดสินได้ถามว่าปัญหา “แก้ได้หรือไม่” แต่ในทางปฏิบัติปัญหาที่แก้ได้ในหลักการอาจต้องใช้เวลานานกว่าอายุจักรวาล **ทฤษฎีความซับซ้อน** (complexity theory) จึงถามต่อว่าปัญหาที่ตัดสินได้นั้น “ต้องใช้ทรัพยากรมากเพียงใด” ในบทนี้เราสนใจทรัพยากรด้านเวลา และจะพบคำถามเปิดที่มีชื่อเสียงที่สุดในวิทยาการคอมพิวเตอร์ คือ P เทียบกับ NP

5.1 การวัดความซับซ้อน

5.1.1 เวลาทำงานและสัญกรณ์เชิงเส้นกำกับ

นิยาม 5.1. ให้ M เป็นเครื่องจักรทัวริงเชิงกำหนดที่หยุดบนทุกอินพุต **เวลาทำงาน** (running time) หรือ **ความซับซ้อนเชิงเวลา** (time complexity) ของ M คือฟังก์ชัน $f : \mathbb{N} \to \mathbb{N}$ โดย $f(n)$ คือจำนวนก้าว*มากที่สุด*ที่ M ใช้บนอินพุตความยาว n เราเรียก M ว่าเป็นเครื่องเวลา $f(n)$

นิยามนี้เป็นการวิเคราะห์ **กรณีเลวร้ายที่สุด** (worst case) และเนื่องจากค่าที่แม่นยำมักซับซ้อนและขึ้นกับรายละเอียดของเครื่อง เราจึงสนใจเฉพาะอัตราการเติบโตเมื่อ n ใหญ่

นิยาม 5.2. ให้ $f, g : \mathbb{N} \to \mathbb{R}^+$ เราเขียน $f(n) = O(g(n))$ (โอใหญ่, big-O) ถ้ามีจำนวนเต็มบวก c และ n_0 ที่ $f(n) \le c\,g(n)$ สำหรับทุก $n \ge n_0$ และเขียน $f(n) = o(g(n))$ (โอเล็ก, small-o) ถ้า $\lim_{n\to\infty} f(n)/g(n) = 0$

O เปรียบได้กับ “$\le$ ภายในตัวคูณคงที่” ส่วน o เปรียบได้กับ “$<$ อย่างเด็ดขาด”

ตัวอย่าง 5.3.

- $f(n) = 3n^4 + 50n^2 + 7 = O(n^4)$ เลือก $c = 60$ และ $n_0 = 1$ ได้ เพราะทุกพจน์ไม่เกิน n^4 เท่าของสัมประสิทธิ์
- $3n^4 + 50n^2 + 7 = O(n^5)$ ก็จริง แต่บอกข้อมูลน้อยกว่า; ในทางกลับกัน $3n^4 \neq O(n^3)$
- $\log_2 n$ กับ $\log_{10} n$ ต่างกันเพียงตัวคูณคงที่ จึงเขียน $O(\log n)$ โดยไม่ระบุฐาน
- ขอบเขตแบบพหุนาม (polynomial bound) คือ $n^{O(1)}$; ขอบเขตแบบเลขชี้กำลัง (exponential bound) คือ 2^{n^δ} สำหรับ $\delta > 0$ เช่น $2^{O(n)}$

ตัวอย่าง 5.4. ตัวอย่างของ o: $\sqrt{n} = o(n)$, $n \log n = o(n^2)$, $n^{10} = o(2^n)$, $\log\log n = o(\log n)$ แต่ $5n \neq o(n)$ เพราะอัตราส่วนเข้าใกล้ 5 ไม่ใช่ 0 และ $f(n) \neq o(f(n))$ เสมอ

5.1.2 ตัวอย่างการวิเคราะห์: ภาษา $\{0^k1^k\}$

ให้ $A = \{0^k1^k \mid k \geq 0\}$ พิจารณาเครื่องเทปเดียว

M_1 = "เมื่อได้รับอินพุต w:

1. กวาดผ่านเทปหนึ่งรอบ ถ้าพบ 0 อยู่ทางขวาของ 1 ให้*ปฏิเสธ*
2. ขณะที่ยังมีทั้ง 0 และ 1 เหลืออยู่ กวาดผ่านเทปแล้วขีดฆ่า 0 หนึ่งตัวและ 1 หนึ่งตัว
3. ถ้ายังเหลือ 0 หรือ 1 อยู่ฝั่งเดียว ให้*ปฏิเสธ* มิฉะนั้น*ยอมรับ*"

วิเคราะห์: ขั้นที่ 1 ใช้ $O(n)$ ก้าว (กวาดไปและกลับ) ขั้นที่ 2 แต่ละรอบใช้ $O(n)$ ก้าวและทำไม่เกิน $n/2$ รอบ รวม $O(n^2)$ ขั้นที่ 3 ใช้ $O(n)$ รวมทั้งหมด $O(n) + O(n^2) + O(n) = O(n^2)$

เราทำได้ดีกว่านี้ด้วยการขีดฆ่าทีละครึ่ง

M_2 = "เมื่อได้รับอินพุต w:

1. ตรวจรูปแบบ 0^*1^* เหมือน M_1
2. ทำซ้ำตราบที่ยังมี 0 และ 1 เหลือบนเทป:
   (a) ตรวจภาวะคู่คี่ (parity) ของจำนวน 0 ที่เหลือและจำนวน 1 ที่เหลือ ถ้าไม่ตรงกัน ให้*ปฏิเสธ*
   (b) ขีดฆ่า 0 ตัวเว้นตัวเริ่มจากตัวแรก และขีดฆ่า 1 ตัวเว้นตัวเริ่มจากตัวแรก
3. ถ้าเหลือ 0 หรือ 1 ฝั่งเดียว ให้*ปฏิเสธ* มิฉะนั้น*ยอมรับ*"

ถ้าเหลืออยู่ c ตัว หลังขีดฆ่าตัวเว้นตัวจะเหลือ $\lfloor c/2 \rfloor$ ตัว ภาวะคู่คี่ที่ตรวจในแต่ละรอบจึงเป็นเลขโดดฐานสองของจำนวน 0 และจำนวน 1 ทีละหลักจากหลักขวาสุด การผ่านทุกรอบจึงแปลว่าจำนวนทั้งสองมีเลขฐานสองเหมือนกันทุกหลัก เช่นอินพุต $0^{12}1^{10}$: $12 = 1100_2$ และ $10 = 1010_2$ รอบแรกทั้งคู่เป็นเลขคู่ (หลักขวาสุดเป็น 0 ทั้งคู่) เหลือ 6 กับ 5 รอบที่สองภาวะคู่คี่ต่างกัน เครื่องจึงปฏิเสธ จำนวนรอบไม่เกิน $1 + \log_2 n$ รอบ แต่ละรอบ $O(n)$ ก้าว ดังนั้น M_2 ใช้เวลา $O(n \log n)$

ถ้าใช้เครื่อง*สองเทป* ทำได้ในเวลา $O(n)$: คัดลอก 0 ทั้งหมดไปเทปที่สอง แล้วจับคู่กับ 1 ทีละตัวขณะอ่านต่อ (อาจพิสูจน์ได้ว่าเครื่องเทปเดียวทำได้ไม่ดีกว่า $O(n \log n)$ สำหรับภาษานี้) ข้อสังเกตนี้สำคัญ: *ความซับซ้อนขึ้นกับแบบจำลองการคำนวณ* ต่างจากความตัดสินได้ซึ่งไม่ขึ้นกับแบบจำลอง

นิยาม 5.5. ให้ $t : \mathbb{N} \to \mathbb{R}^+$ **คลาสความซับซ้อนเชิงเวลา** $\mathsf{TIME}(t(n))$ คือเซตของภาษาทั้งหมดที่ตัดสินได้ด้วยเครื่องจักรทัวริง (เทปเดียว เชิงกำหนด) เวลา $O(t(n))$

จากการวิเคราะห์ข้างต้น $A \in \mathsf{TIME}(n \log n)$ และแน่นอนว่า $A \in \mathsf{TIME}(n^2)$ ด้วย

5.1.3 ความสัมพันธ์ระหว่างแบบจำลอง

ทฤษฎีบท 5.6. *ให้ $t(n) \geq n$ เครื่องจักรทัวริงหลายเทปเวลา $t(n)$ ใด ๆ มีเครื่องเทปเดียวที่สมมูลกันซึ่งใช้เวลา $O(t^2(n))$*

แนวคิดการพิสูจน์. ใช้การจำลองแบบเดิมจากบทเครื่องจักรทัวริง: เก็บเนื้อหาของ k เทปเรียงต่อกันบนเทปเดียวคั่นด้วย # และทำเครื่องหมายตำแหน่งหัวอ่านของแต่ละเทป ภายใน $t(n)$ ก้าว แต่ละเทป

ยาวไม่เกิน $t(n)$ ส่วนที่ใช้งานบนเทปเดียวจึงยาว $O(t(n))$ การจำลองหนึ่งก้าวต้องกวาดอ่านหนึ่งรอบ กวาดเขียนอีกรอบ และอาจต้องเลื่อนข้อมูลเพื่อเพิ่มช่องว่าง รวมเป็น $O(t(n))$ ก้าว ทำ $t(n)$ ครั้ง ได้ $O(t^2(n))$ □

นิยาม 5.7. ให้ N เป็นเครื่องจักรทัวริงไม่เชิงกำหนด (nondeterministic) ที่เป็นตัวตัดสิน (ทุกกิ่งหยุด) เวลาทำงานของ N คือ $f(n)$ ซึ่งเป็นจำนวนก้าวมากที่สุดที่*กิ่งใดกิ่งหนึ่ง*ของการคำนวณใช้บนอินพุตความยาว n

ทฤษฎีบท 5.8. *ให้ $t(n) \geq n$ เครื่องไม่เชิงกำหนดเวลา $t(n)$ ใด ๆ มีเครื่องเชิงกำหนดเทปเดียวที่สมมูลกันซึ่งใช้เวลา $2^{O(t(n))}$*

แนวคิดการพิสูจน์. ต้นไม้การคำนวณของ N มีความลึกไม่เกิน $t(n)$ และแต่ละโหนดแตกกิ่งไม่เกิน b กิ่ง (b คงที่ขึ้นกับฟังก์ชันการเปลี่ยนสถานะ) จึงมีใบไม่เกิน $b^{t(n)}$ ใบ การค้นแบบกว้างก่อน (breadth-first) ด้วยเครื่องสามเทปไล่ทุกโหนด แต่ละโหนดเริ่มจำลองใหม่จากรากใช้ $O(t(n))$ ก้าว รวมเป็น $O(t(n)\, b^{t(n)}) = 2^{O(t(n))}$ แปลงเป็นเทปเดียวด้วยทฤษฎีบท 5.6 ได้ $\left(2^{O(t(n))}\right)^2 = 2^{O(t(n))}$ □

สองทฤษฎีบทนี้บอกความแตกต่างสำคัญ: ระหว่างแบบจำลอง*เชิงกำหนด*ด้วยกัน เวลาต่างกันอย่างมากแค่ยกกำลังสอง (พหุนาม) แต่จากไม่เชิงกำหนดมาเป็นเชิงกำหนด เท่าที่เรารู้ เวลาอาจเพิ่มขึ้นแบบเลขชี้กำลัง

5.2 คลาส P

นิยาม 5.9.

$$\mathsf{P} = \bigcup_{k \geq 1} \mathsf{TIME}(n^k)$$

คือคลาสของภาษาที่ตัดสินได้ด้วยเครื่องจักรทัวริงเทปเดียวเชิงกำหนดในเวลาพหุนาม (polynomial time)

5.2.1 ทำไมจึงใช้พหุนาม

1. **ความทนทาน** (robustness): แบบจำลองเชิงกำหนดที่สมเหตุสมผลทุกแบบ (เครื่องหลายเทป เครื่องเข้าถึงสุ่ม ฯลฯ) จำลองกันได้โดยเวลาเพิ่มขึ้นเพียงแบบพหุนาม และพหุนามประกอบกับพหุนามยังคงเป็นพหุนาม ดังนั้นคลาส P ไม่เปลี่ยนไม่ว่าจะใช้แบบจำลองใด
2. **ความหมายเชิงปฏิบัติ**: P เป็นตัวแทนคร่าว ๆ ของปัญหาที่ “แก้ได้จริง” เพราะช่องว่างระหว่างพหุนามกับเลขชี้กำลังนั้นมหาศาล เช่นที่ $n = 60$ จะได้ $n^3 = 216{,}000$ แต่ $2^n \approx 1.15 \times 10^{18}$
3. **การค้นหาแบบถึก** (brute force) ซึ่งไล่ทุกคำตอบที่เป็นไปได้มักใช้เวลาเลขชี้กำลัง การพบอัลกอริทึมเวลาพหุนามมักหมายถึงการเข้าใจโครงสร้างของปัญหาอย่างลึกซึ้ง

หมายเหตุ. เวลาวัดเทียบกับความยาวของ*การเข้ารหัส*อินพุต ต้องใช้การเข้ารหัสที่สมเหตุสมผล เช่น เขียนจำนวนเต็มในฐานสอง (หรือฐานใดก็ได้ที่ ≥ 2) ไม่ใช่ฐานหนึ่ง (unary) เพราะจำนวน m ในฐานหนึ่งยาว m ตัว แต่ในฐานสองยาวเพียงประมาณ $\log_2 m$ ตัว กราฟเข้ารหัสด้วยรายการโหนดและเส้นเชื่อม หรือเมทริกซ์ประชิด (adjacency matrix) ซึ่งยาวต่างกันแค่พหุนาม

5.2.2 ตัวอย่างปัญหาใน P

ปัญหาเส้นทาง

$$\mathit{PATH} = \{\langle G, s, t\rangle \mid G \text{ เป็นกราฟระบุทิศทางที่มีเส้นทางจาก } s \text{ ไปยัง } t\}$$

การไล่ทุกเส้นทางที่เป็นไปได้อาจใช้เวลาประมาณ m^m เมื่อ G มี m โหนด แต่อัลกอริทึมต่อไปนี้ใช้เวลาพหุนาม

M = "เมื่อได้รับ $\langle G, s, t\rangle$:
1. ทำเครื่องหมายที่ s
2. ทำซ้ำจนไม่มีโหนดใหม่ถูกทำเครื่องหมาย: สำหรับทุกเส้นเชื่อม (a, b) ที่ a มีเครื่องหมายแต่ b ยังไม่มี ให้ทำเครื่องหมายที่ b
3. ถ้า t มีเครื่องหมาย ให้*ยอมรับ* มิฉะนั้น*ปฏิเสธ*"

ขั้นที่ 2 ทำไม่เกิน m รอบ (แต่ละรอบเพิ่มโหนดอย่างน้อยหนึ่งโหนด) และแต่ละรอบตรวจทุกเส้นเชื่อมได้ในเวลาพหุนาม ดังนั้น $\mathit{PATH} \in$ P ตัวอย่างเช่น กราฟที่มีโหนด $1, \ldots, 6$ และเส้นเชื่อม $1 \to 3,\ 3 \to 2,\ 2 \to 5,\ 4 \to 1,\ 6 \to 4$ เมื่อ $s = 1$ โหนดที่ถูกทำเครื่องหมายคือ $\{1\} \to \{1, 3\} \to \{1, 2, 3\} \to \{1, 2, 3, 5\}$ แล้วหยุด ดังนั้นมีเส้นทางจาก 1 ไป 5 แต่ไม่มีเส้นทางจาก 1 ไป 6

ปัญหาจำนวนเฉพาะสัมพัทธ์ จำนวนเต็มบวกสองจำนวน **เป็นจำนวนเฉพาะสัมพัทธ์** (relatively prime) ถ้าตัวหารร่วมมาก (gcd) เท่ากับ 1 ให้ $\mathit{RELPRIME} = \{\langle x, y\rangle \mid x, y \text{ เป็นจำนวนเฉพาะสัมพัทธ์}\}$ การลองหารด้วยทุกจำนวนตั้งแต่ 2 ถึง $\min(x, y)$ ใช้เวลาเลขชี้กำลังเทียบกับ*จำนวนบิต*ของอินพุต แต่ **ขั้นตอนวิธีของยุคลิด** (Euclidean algorithm) ทำได้เร็ว:

E = "เมื่อได้รับ $\langle x, y\rangle$: ทำซ้ำจนกว่า $y = 0$: กำหนด $x \leftarrow x \bmod y$ แล้วสลับ x กับ y; คืนค่า x"

ตัวอย่าง $\gcd(84, 55)$:

$$(84, 55) \to (55, 29) \to (29, 26) \to (26, 3) \to (3, 2) \to (2, 1) \to (1, 0)$$

ได้ $\gcd = 1$ ดังนั้น $\langle 84, 55\rangle \in \mathit{RELPRIME}$ (ตรวจสอบได้จาก $84 = 2^2 \cdot 3 \cdot 7$ และ $55 = 5 \cdot 11$) ทุก ๆ สองรอบค่า x ลดลงอย่างน้อยครึ่งหนึ่ง (ถ้า $y \le x/2$ แล้ว $x \bmod y < y \le x/2$ ถ้า $y > x/2$ แล้ว $x \bmod y = x - y < x/2$) จำนวนรอบจึงเป็น $O(\log x)$ ซึ่งเป็นเชิงเส้นในจำนวนบิต และแต่ละรอบหารได้ในเวลาพหุนาม จึงได้ $\mathit{RELPRIME} \in$ P

ภาษาไม่พึ่งบริบททุกภาษาอยู่ใน P

ทฤษฎีบท 5.10. *ทุกภาษาไม่พึ่งบริบทเป็นสมาชิกของ* P

แนวคิดการพิสูจน์. อัลกอริทึมจากบทก่อนที่ลองทุกการแปลงยาว $2n - 1$ ก้าวใช้เวลาเลขชี้กำลัง แทนที่จะเป็นเช่นนั้นเราใช้ **กำหนดการพลวัต** (dynamic programming): ให้ไวยากรณ์ G อยู่ในรูปแบบปกติของชอมสกี (Chomsky normal form) และสำหรับอินพุต $w = w_1 \cdots w_n$ สร้างตาราง $T[i, j]$ เก็บเซตของตัวแปรที่สร้างสตริงย่อย $w_i \cdots w_j$ ได้ เติมตารางจากสตริงย่อยสั้นไปยาว: $T[i, i]$ ได้จากกฎ $X \to w_i$ และ $X \in T[i, j]$ ถ้ามีกฎ $X \to YZ$ และจุดแบ่ง k ที่ $Y \in T[i, k]$, $Z \in T[k + 1, j]$ ยอมรับเมื่อตัวแปรเริ่มต้นอยู่ใน $T[1, n]$ มีช่องตาราง $O(n^2)$ ช่อง แต่ละช่องลองจุดแบ่ง $O(n)$ จุด จึงใช้เวลา $O(n^3)$ □

ตัวอย่าง 5.11. ไวยากรณ์ $S \to AB \mid AC,\ C \to SB,\ A \to \mathtt{a},\ B \to \mathtt{b}$ สร้างภาษา $\{\mathtt{a}^k\mathtt{b}^k \mid k \geq 1\}$ ตารางของ $w = \mathtt{aabb}$ (แถวคือความยาวสตริงย่อย คอลัมน์คือตำแหน่งเริ่มต้น i) เป็นดังนี้

ความยาว	$i=1$	$i=2$	$i=3$	$i=4$
1	$\{A\}$	$\{A\}$	$\{B\}$	$\{B\}$
2	$\emptyset$	$\{S\}$	$\emptyset$	
3	$\emptyset$	$\{C\}$		
4	$\{S\}$			

เช่น $T[2,4]$ (สตริง $\mathtt{abb}$) มี C เพราะแบ่งเป็น $\mathtt{ab}\,|\,\mathtt{b}$ ได้ S กับ B และมีกฎ $C \to SB$ สุดท้าย $T[1,4]$ มี S จากการแบ่ง $\mathtt{a}\,|\,\mathtt{abb}$ และกฎ $S \to AC$ จึงยอมรับ

5.3 คลาส NP

5.3.1 ตัวตรวจสอบและใบรับรอง

มีปัญหาจำนวนมากที่ยังไม่มีใครพบอัลกอริทึมเวลาพหุนาม แต่มีลักษณะร่วมที่น่าสนใจ คือ *ถ้ามีคนบอกคำตอบมาให้ เราตรวจได้เร็ว* เช่น ปัญหาเส้นทางแฮมิลตัน

$$\mathit{HAMPATH} = \{\langle G, s, t\rangle \mid G \text{ เป็นกราฟระบุทิศทางที่มีเส้นทางแฮมิลตันจาก } s \text{ ไป } t\}$$

โดย **เส้นทางแฮมิลตัน** (Hamiltonian path) คือเส้นทางที่ผ่านทุกโหนดของกราฟครั้งเดียวพอดี การหาเส้นทางดูเหมือนต้องไล่ลำดับโหนดแบบถึก แต่ถ้ามีใครยื่นลำดับโหนดมาให้ เราตรวจว่าถูกต้องได้ในเวลาพหุนาม

นิยาม 5.12. ตัวตรวจสอบ (verifier) ของภาษา A คืออัลกอริทึม V ที่

$$A = \{w \mid V \text{ ยอมรับ } \langle w, c\rangle \text{ สำหรับบางสตริง } c\}.$$

ตัวตรวจสอบ*เวลาพหุนาม* คือตัวที่ใช้เวลาพหุนามในความยาวของ w สตริง c เรียกว่า **ใบรับรอง** (certificate) หรือหลักฐาน (proof) ของการเป็นสมาชิก และคลาส NP คือคลาสของภาษาที่มีตัวตรวจสอบเวลาพหุนาม

สังเกตว่าเมื่อ V ใช้เวลาพหุนาม ใบรับรองที่ V อ่านได้ทันก็ยาวไม่เกินพหุนามโดยอัตโนมัติ และนิยามไม่สมมาตร: สำหรับ $w \notin A$ ไม่มีใบรับรองใดทำให้ V ยอมรับ แต่เราไม่ได้เรียกร้องให้มี “หลักฐานของการไม่เป็นสมาชิก” คลาสของภาษาที่ส่วนเติมเต็มอยู่ใน NP เรียกว่า coNP และยังไม่รู้ว่าเท่ากับ NP หรือไม่

นิยาม 5.13. $\mathrm{NTIME}(t(n))$ คือเซตของภาษาที่ตัดสินได้ด้วยเครื่องจักรทัวริงไม่เชิงกำหนดเวลา $O(t(n))$

ทฤษฎีบท 5.14. *ภาษาหนึ่งอยู่ใน* NP *ก็ต่อเมื่อตัดสินได้ด้วยเครื่องจักรทัวริงไม่เชิงกำหนดเวลาพหุนาม นั่นคือ* $\mathrm{NP} = \bigcup_k \mathrm{NTIME}(n^k)$

แนวคิดการพิสูจน์. ($\Rightarrow$) ถ้ามีตัวตรวจสอบ V เวลา n^k ให้เครื่องไม่เชิงกำหนด “เดา” ใบรับรอง c ยาวไม่เกิน n^k แล้วรัน V บน $\langle w, c\rangle$ ($\Leftarrow$) ถ้ามีเครื่องไม่เชิงกำหนด N ใช้ลำดับการเลือกในกิ่งที่ยอมรับเป็นใบรับรอง ตัวตรวจสอบจำลอง N แบบเชิงกำหนดตามลำดับการเลือกนั้น □

ชื่อ NP มาจาก *nondeterministic polynomial time* ไม่ได้ย่อมาจาก “non-polynomial”

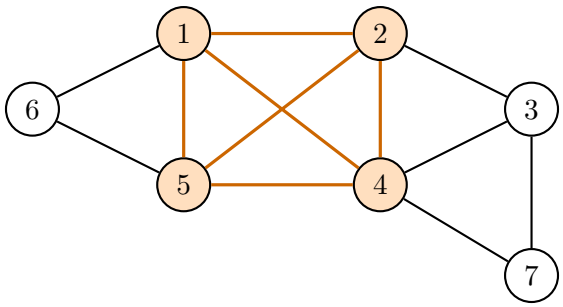
5.3.2 ตัวอย่างปัญหาใน NP

เส้นทางแฮมิลตัน ใบรับรองคือลำดับโหนด $v_1, \ldots, v_m$ ตัวตรวจสอบเช็กว่าไม่มีโหนดซ้ำ ครบทุกโหนด $v_1 = s$, $v_m = t$ และมีเส้นเชื่อม (v_i, v_{i+1}) ในกราฟทุกคู่ ทั้งหมดทำได้ในเวลาพหุนาม เช่น กราฟที่มีเส้นเชื่อม $s \to a,\ a \to b,\ s \to b,\ b \to c,\ c \to a,\ c \to t$ ใบรับรอง $s, b, c, a, \ldots$ ใช้ไม่ได้เพราะ a ไปต่อ t ไม่ได้ แต่ s, a, b, c, t เป็นเส้นทางแฮมิลตัน

กลุ่มพรรคพวก **กลุ่มพรรคพวก** (clique) ในกราฟไม่ระบุทิศทางคือเซตของโหนดที่ทุกคู่มีเส้นเชื่อมถึงกัน k-clique คือกลุ่มพรรคพวกขนาด k ให้

$$\mathit{CLIQUE} = \{\langle G, k\rangle \mid G \text{ เป็นกราฟไม่ระบุทิศทางที่มี } k\text{-clique}\}$$

ใบรับรองคือเซตของ k โหนด ตัวตรวจสอบเช็กว่าทุกคู่ในเซต (มี $\binom{k}{2}$ คู่) มีเส้นเชื่อม หรือมองแบบเครื่องไม่เชิงกำหนด: เลือก k โหนดแบบไม่เชิงกำหนดแล้วตรวจ รูปที่ 5.1 แสดงกราฟเจ็ดโหนดที่มี 4-clique แต่ไม่มี 5-clique



รูปที่ 5.1: กราฟที่มี 4-clique คือ $\{1, 2, 4, 5\}$ (โหนดและเส้นเชื่อมที่เน้น) ใบรับรองของ $\langle G, 4\rangle \in \mathit{CLIQUE}$ คือเซตนี้

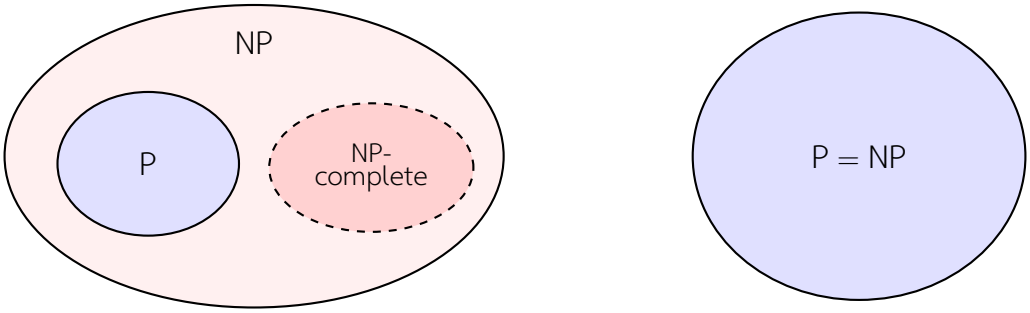
ผลรวมเซตย่อย

$$\mathit{SUBSET\text{-}SUM} = \left\{\langle S, t\rangle \;\middle|\; \text{มี } T \subseteq S \text{ ที่ } \textstyle\sum_{x \in T} x = t\right\}$$

ในที่นี้ S เป็นพหุเซต (multiset) ของจำนวนเต็มบวก ตัวอย่างเช่น $S = \{4, 11, 16, 21, 27\}$ และ $t = 47$ เป็นสมาชิก เพราะ $4 + 16 + 27 = 47$ ใบรับรองคือเซตย่อย $\{4, 16, 27\}$ ซึ่งตรวจได้ด้วยการบวกเลข แต่ถ้า $t = 6$ จะไม่มีเซตย่อยใดเลย การยืนยันว่า “ไม่มี” ดูเหมือนต้องตรวจทั้ง 2^5 เซตย่อย (ตัวเลขเขียนในฐานสอง อัลกอริทึมกำหนดการพลวัตที่ใช้เวลาพหุนามใน*ค่า* t จึงไม่ใช่เวลาพหุนามในความยาวอินพุต)

5.3.3 คำถาม P เทียบกับ NP

ทุกภาษาใน P อยู่ใน NP ด้วย (ตัวตรวจสอบไม่ต้องใช้ใบรับรองเลย) จึงได้ $\mathrm{P} \subseteq \mathrm{NP}$ คำถามว่า $\mathrm{P} = \mathrm{NP}$ หรือไม่ คือคำถามว่า “การตรวจคำตอบได้เร็ว” เพียงพอให้ “หาคำตอบได้เร็ว” หรือไม่ นักวิจัยส่วนใหญ่เชื่อว่า $\mathrm{P} \neq \mathrm{NP}$ แต่ยังไม่มีใครพิสูจน์ได้ ขอบเขตที่ดีที่สุดที่รู้สำหรับการแก้ปัญหาใน NP ด้วยเครื่องเชิงกำหนดคือ $\mathrm{NP} \subseteq \mathrm{EXPTIME} = \bigcup_k \mathrm{TIME}(2^{n^k})$ (จากทฤษฎีบท 5.8)



กรณี P $\neq$ NP กรณี P $=$ NP

รูปที่ 5.2: สองความเป็นไปได้ ถ้า P $\neq$ NP ปัญหาเอ็นพีบริบูรณ์ทั้งหมดอยู่นอก P

5.4 ความบริบูรณ์เอ็นพี

ในช่วงต้นทศวรรษ 1970 สตีเฟน คุก (Stephen Cook) และลีโอนิด เลวิน (Leonid Levin) ค้นพบโดยอิสระว่า มีปัญหาบางปัญหาใน NP ที่ “ยากที่สุด” ในความหมายว่าถ้าปัญหานั้นมีอัลกอริทึมเวลาพหุนาม ทุกปัญหาใน NP ก็จะมีด้วย ปัญหาเหล่านี้เรียกว่า **เอ็นพีบริบูรณ์** (NP-complete) ผลนี้สำคัญทั้งในเชิงทฤษฎี (เพื่อพิสูจน์ P $\neq$ NP ก็พอจะพิจารณาปัญหาเดียว) และในเชิงปฏิบัติ (ถ้าพิสูจน์ได้ว่าปัญหาที่เจอเป็นเอ็นพีบริบูรณ์ ก็ไม่ควรเสียเวลาหาอัลกอริทึมเวลาพหุนามที่แม่นยำ)

5.4.1 ปัญหาความสอดคล้องได้

สูตรบูลีน (Boolean formula) สร้างจากตัวแปรบูลีนด้วยตัวดำเนินการ $\wedge$ (และ), $\vee$ (หรือ), $\neg$ (นิเสธ) สูตรเป็น **สูตรที่สอดคล้องได้** (satisfiable) ถ้ามีการกำหนดค่าจริงเท็จให้ตัวแปรที่ทำให้สูตรเป็นจริง

$$SAT = \{\langle\phi\rangle \mid \phi \text{ เป็นสูตรบูลีนที่สอดคล้องได้}\}$$

เช่น $\phi = (x \vee \neg y) \wedge (\neg x \vee z) \wedge (y \vee \neg z)$ สอดคล้องได้ด้วย $x = y = z = 1$ แต่ $(x \vee y) \wedge (\neg x) \wedge (\neg y)$ สอดคล้องไม่ได้

5.4.2 การลดรูปเวลาพหุนาม

นิยาม 5.15. ฟังก์ชัน f เป็น **ฟังก์ชันที่คำนวณได้ในเวลาพหุนาม** (polynomial time computable function) ถ้ามีเครื่องจักรทัวริงเวลาพหุนามที่คำนวณ f ได้ ภาษา A **ลดรูปเวลาพหุนามได้** ไปยัง B เขียนว่า $A \leq_{\mathrm{P}} B$ ถ้ามี f ที่คำนวณได้ในเวลาพหุนามซึ่ง $w \in A \iff f(w) \in B$ สำหรับทุก w

นี่คือการลดรูปแบบส่งแผนที่จากบทที่แล้วที่เพิ่มเงื่อนไขว่าต้องคำนวณได้ *เร็ว*

ทฤษฎีบท 5.16. *ถ้า* $A \leq_{\mathrm{P}} B$ *และ* $B \in \mathrm{P}$ *แล้ว* $A \in \mathrm{P}$

พิสูจน์. คำนวณ $f(w)$ ในเวลา $O(n^k)$ ผลลัพธ์จึงยาวไม่เกิน $O(n^k)$ แล้วรันตัวตัดสินของ B เวลา $O(m^\ell)$ บน $f(w)$ รวมเป็น $O(n^k) + O(n^{k\ell})$ ซึ่งเป็นพหุนาม □

5.4.3 ปัญหา 3SAT และการลดรูปไปยัง CLIQUE

ลิเทอรัล (literal) คือตัวแปรหรือนิเสธของตัวแปร **อนุประโยค** (clause) คือลิเทอรัลหลายตัวเชื่อมด้วย $\vee$ สูตรอยู่ใน **รูปแบบปกติเชิงเชื่อม** (conjunctive normal form, CNF) ถ้าเป็นอนุประโยคหลายอนุประโยคเชื่อมด้วย $\wedge$ และเป็น **สูตร 3cnf** ถ้าทุกอนุประโยคมีลิเทอรัลพอดีสามตัว ให้ $3SAT = \{\langle\phi\rangle \mid \phi$ เป็นสูตร 3cnf ที่สอดคล้องได้$\}$

ทฤษฎีบท 5.17. *$3SAT \leq_{\mathrm{P}} CLIQUE$*

พิสูจน์. ให้ ϕ เป็นสูตร 3cnf ที่มี k อนุประโยค สร้างกราฟไม่ระบุทิศทาง G ดังนี้

- **โหนด**: แต่ละอนุประโยคให้โหนดสามโหนดเป็น "กลุ่มสาม" (triple) หนึ่งกลุ่ม แต่ละโหนดมีป้ายเป็นลิเทอรัลหนึ่งตัวของอนุประโยคนั้น รวม $3k$ โหนด
- **เส้นเชื่อม**: เชื่อมโหนดสองโหนด*ทุกคู่* ยกเว้น (ก) คู่ที่อยู่ในกลุ่มสามเดียวกัน และ (ข) คู่ที่มีป้ายขัดแย้งกัน เช่น x_2 กับ $\neg x_2$

แล้วให้ $f(\langle\phi\rangle) = \langle G, k\rangle$ ซึ่งคำนวณได้ในเวลาพหุนามอย่างชัดเจน

ถ้า ϕ สอดคล้องได้ : ภายใต้การกำหนดค่าที่ทำให้ ϕ จริง แต่ละอนุประโยคมีลิเทอรัลที่จริงอย่างน้อยหนึ่งตัว เลือกโหนดของลิเทอรัลนั้นจากแต่ละกลุ่มสาม ได้ k โหนดจากกลุ่มต่างกัน และไม่มีคู่ใดขัดแย้งกัน (เพราะลิเทอรัลที่จริงพร้อมกันจะเป็น x กับ $\neg x$ ไม่ได้) ทุกคู่จึงมีเส้นเชื่อม เป็น k-clique

ถ้า G มี k-clique: โหนดในกลุ่มเดียวกันไม่มีเส้นเชื่อม clique จึงมีโหนดจากแต่ละกลุ่มสามพอดีหนึ่งโหนด และไม่มีป้ายขัดแย้งกัน จึงกำหนดค่าให้ทุกลิเทอรัลใน clique เป็นจริงได้พร้อมกัน (ตัวแปรที่เหลือกำหนดค่าใดก็ได้) ทุกอนุประโยคจึงจริง ϕ สอดคล้องได้ □

ตัวอย่าง 5.18. ให้ $\phi = (x_1 \vee x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \neg x_2 \vee x_3)$ ซึ่ง $k = 3$ กราฟที่ได้แสดงในรูปที่ 5.3 ระหว่างกลุ่มสามแต่ละคู่มีเส้นเชื่อม $9 - 2 = 7$ เส้น (ตัดคู่ที่ขัดแย้งออกสองคู่) รวม 21 เส้น การกำหนดค่า $x_1 = x_2 = x_3 = 1$ ทำให้ ϕ จริง เลือก x_1 จากอนุประโยคแรก x_2 จากอนุประโยคที่สอง และ x_1 จากอนุประโยคที่สาม ได้ 3-clique ที่เน้นไว้ในรูป

5.4.4 นิยามของความบริบูรณ์เอ็นพี

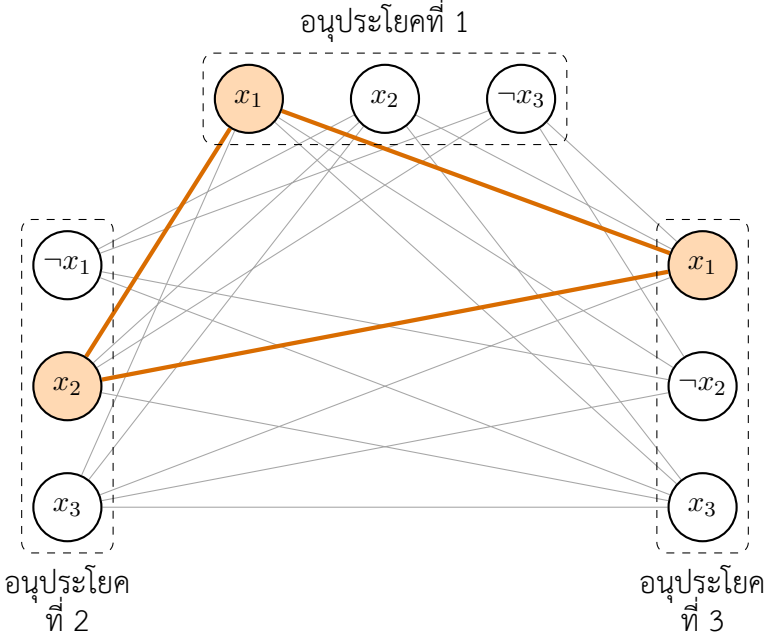
นิยาม 5.19. ภาษา B เป็น **เอ็นพีบริบูรณ์** (NP-complete) ถ้า (1) $B \in$ NP และ (2) ทุกภาษา $A \in$ NP ลดรูปเวลาพหุนามได้ไปยัง B คือ $A \leq_{\mathrm{P}} B$ ถ้า B ผ่านเฉพาะเงื่อนไข (2) เรียกว่า **เอ็นพียาก** (NP-hard)

ทฤษฎีบท 5.20. *ถ้า B เป็นเอ็นพีบริบูรณ์ และ $B \in$ P แล้ว* P = NP

ทฤษฎีบท 5.21. *ถ้า B เป็นเอ็นพีบริบูรณ์ และ $B \leq_{\mathrm{P}} C$ โดยที่ $C \in$ NP แล้ว C เป็นเอ็นพีบริบูรณ์*

พิสูจน์. สำหรับ $A \in$ NP ใด ๆ มี $A \leq_{\mathrm{P}} B$ และ $B \leq_{\mathrm{P}} C$ การประกอบฟังก์ชันเวลาพหุนามสองตัวยังเป็นเวลาพหุนาม จึงได้ $A \leq_{\mathrm{P}} C$ □

ทฤษฎีบท 5.21 ทำให้งานง่ายลงมาก: เมื่อมีปัญหาเอ็นพีบริบูรณ์ปัญหาแรกแล้ว ปัญหาถัดไปเพียงต้องแสดงว่าอยู่ใน NP และลดรูปจากปัญหาเอ็นพีบริบูรณ์ที่รู้แล้วสักปัญหาหนึ่ง



รูปที่ 5.3: กราฟจากการลดรูป *3SAT* $\leq_{\mathrm{P}}$ *CLIQUE* ของสูตรในตัวอย่าง เส้นสีเทาเชื่อมลิเทอรัลที่ไม่ขัดแย้งกันในต่างอนุประโยค เส้นหนาคือ 3-clique ที่สอดคล้องกับ $x_1 = x_2 = x_3 = 1$

5.4.5 ทฤษฎีบทคุก–เลวิน

ทฤษฎีบท 5.22 (คุก–เลวิน (Cook–Levin))**.** *SAT เป็นเอ็นพีบริบูรณ์*

แนวคิดการพิสูจน์. *SAT* $\in$ NP ชัดเจน (ใบรับรองคือการกำหนดค่าที่ทำให้สูตรจริง) ส่วนที่ยากคือแสดงว่า $A \leq_{\mathrm{P}}$ *SAT* สำหรับ*ทุก* $A \in$ NP ให้ N เป็นเครื่องไม่เชิงกำหนดที่ตัดสิน A ในเวลา n^k การคำนวณหนึ่งกิ่งของ N บน w เขียนได้เป็น **ตาราง** (tableau) ขนาด $n^k \times n^k$ ซึ่งแถวที่ i คือการจัดรูปแบบที่ก้าวที่ i จาก w สร้างสูตร ϕ ที่มีตัวแปร $x_{i,j,s}$ หมายถึง “ช่อง (i,j) ของตารางมีสัญลักษณ์ s” และ

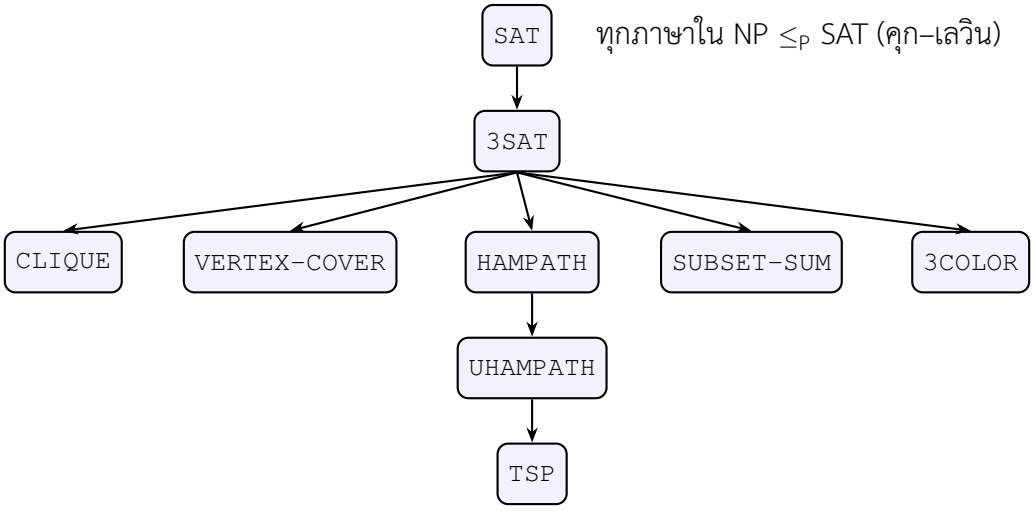
$$\phi = \phi_{\text{cell}} \wedge \phi_{\text{start}} \wedge \phi_{\text{move}} \wedge \phi_{\text{accept}}$$

โดย ϕ_{cell} บังคับให้ทุกช่องมีสัญลักษณ์เดียวพอดี ϕ_{start} บังคับให้แถวแรกเป็นการจัดรูปแบบเริ่มต้นบน w ϕ_{accept} บังคับให้มี q_{accept} อยู่ที่ใดที่หนึ่ง และ ϕ_{move} บังคับให้ทุก “หน้าต่าง” (window) ขนาด 2×3 ของตารางสอดคล้องกับฟังก์ชันการเปลี่ยนสถานะ ซึ่งเป็นเงื่อนไขเฉพาะที่ (local) จึงเพียงพอให้ทุกแถวต่อจากแถวก่อนอย่างถูกต้อง ϕ สอดคล้องได้ก็ต่อเมื่อ N มีกิ่งที่ยอมรับ w และ ϕ มีขนาด $O(n^{2k})$ จึงสร้างได้ในเวลาพหุนาม □

บทแทรก 5.23. *3SAT เป็นเอ็นพีบริบูรณ์ (แปลงสูตรจากบทพิสูจน์ข้างต้นให้เป็น 3cnf ได้ในเวลาพหุนาม) และจากทฤษฎีบท 5.17 กับ 5.21 จะได้ว่า CLIQUE เป็นเอ็นพีบริบูรณ์*

5.4.6 ปัญหาเอ็นพีบริบูรณ์อื่น ๆ

ปัจจุบันรู้จักปัญหาเอ็นพีบริบูรณ์หลายพันปัญหาในแทบทุกสาขา รูปที่ 5.4 แสดงสายการลดรูปที่มักใช้พิสูจน์ความบริบูรณ์ของปัญหาเหล่านี้



รูปที่ 5.4: ต้นไม้การลดรูปเวลาพหุนาม ลูกศร $X \to Y$ หมายถึง $X \leq_{\mathrm{P}} Y$

ตัวอย่างที่ควรรู้จัก ได้แก่

- *VERTEX-COVER*: มีเซตโหนดขนาด k ที่ทุกเส้นเชื่อมแตะอย่างน้อยหนึ่งโหนดในเซตหรือไม่
- *HAMPATH* และรุ่นไม่ระบุทิศทาง *UHAMPATH*
- *SUBSET-SUM* และปัญหาการแบ่งส่วน (partition) ปัญหาเป้ (knapsack) รุ่นตัดสินใจ
- *3COLOR*: ระบายสีโหนดด้วยสามสีโดยโหนดที่ติดกันสีต่างกันได้หรือไม่
- ปัญหาพนักงานขายเดินทาง (traveling salesman) รุ่นตัดสินใจ: มีวงทัวร์ที่ระยะรวมไม่เกิน d หรือไม่
- การจัดตารางเวลา และการเขียนโปรแกรมเชิงเส้นจำนวนเต็ม (integer linear programming) รุ่นตัดสินใจ

สรุปบทที่ 5

- เวลาทำงานวัดแบบกรณีเลวร้ายที่สุดเป็นฟังก์ชันของความยาวอินพุต ใช้ O และ o ละทิ้งตัวคูณคงที่
- หลายเทปเวลา $t \to$ เทปเดียว $O(t^2)$; ไม่เชิงกำหนดเวลา $t \to$ เชิงกำหนด $2^{O(t)}$
- $\mathrm{P} = \bigcup_k \mathrm{TIME}(n^k)$ ทนทานต่อการเปลี่ยนแบบจำลอง เช่น *PATH*, *RELPRIME* และทุกภาษาไม่พึ่งบริบท
- NP = ภาษาที่มีตัวตรวจสอบเวลาพหุนาม $= \bigcup_k \mathrm{NTIME}(n^k)$ เช่น *HAMPATH*, *CLIQUE*, *SUBSET-SUM*
- $\mathrm{P} \subseteq \mathrm{NP}$ แต่ยังไม่รู้ว่าเท่ากันหรือไม่
- B เอ็นพีบริบูรณ์ ถ้า $B \in \mathrm{NP}$ และทุก $A \in \mathrm{NP}$ มี $A \leq_{\mathrm{P}} B$; *SAT* (คุก–เลวิน), *3SAT*, *CLIQUE* เป็นเอ็นพีบริบูรณ์